

AY 2025/2026 S1

EE6427 Video Signal Processing

Part 5 Video Analysis and Understanding

Dr Yap Kim Hui

Room: S2-B2b-53

Tel: 6790 4339

Email: [ekhyap@ntu.edu.sg](mailto:ekhyap@ntu.edu.sg)



# References

- Xiong et. al., “Recent Advances in Deep Learning for Object Detection,” Neurocomputing 2020.
- Zaidi et. al., “Survey of Modern Deep Learning based Object Detection Models,” CoRR 2021. <https://arxiv.org/abs/2104.11892>
- Gioele Ciaparrone, “Deep Learning in Video Multi-Object Tracking: A Survey,” Neurocomputing, 2020.
- Ce Zheng et. al., “Deep Learning-Based Human Pose Estimation: A Survey,” CoRR 2020. <https://doi.org/10.48550/arXiv.2012.13392>
- Yi Zhu et. al., “A Comprehensive Study of Deep Video Action Recognition,” CoRR 2020. <https://doi.org/10.48550/arXiv.2012.06567>

# Part 5 Outline

- Video Analysis & Understanding
  - Object Detection & Tracking
  - Pose Estimation
  - Video / Human Action Recognition

# Object Detection

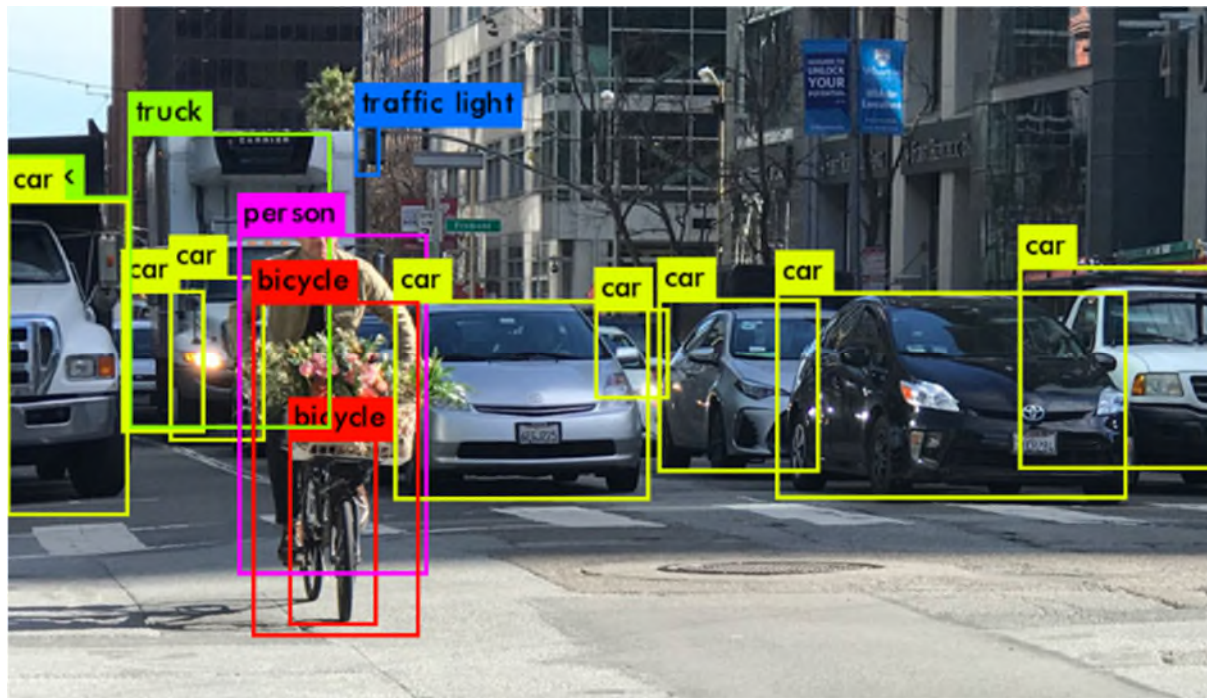


# Object Detection Overview

- Introduction
- Object Detection Methods
  - Two Stage Detectors
  - One Stage Detectors
  - Lightweight Detectors
- New / Emerging Methods

# Objective

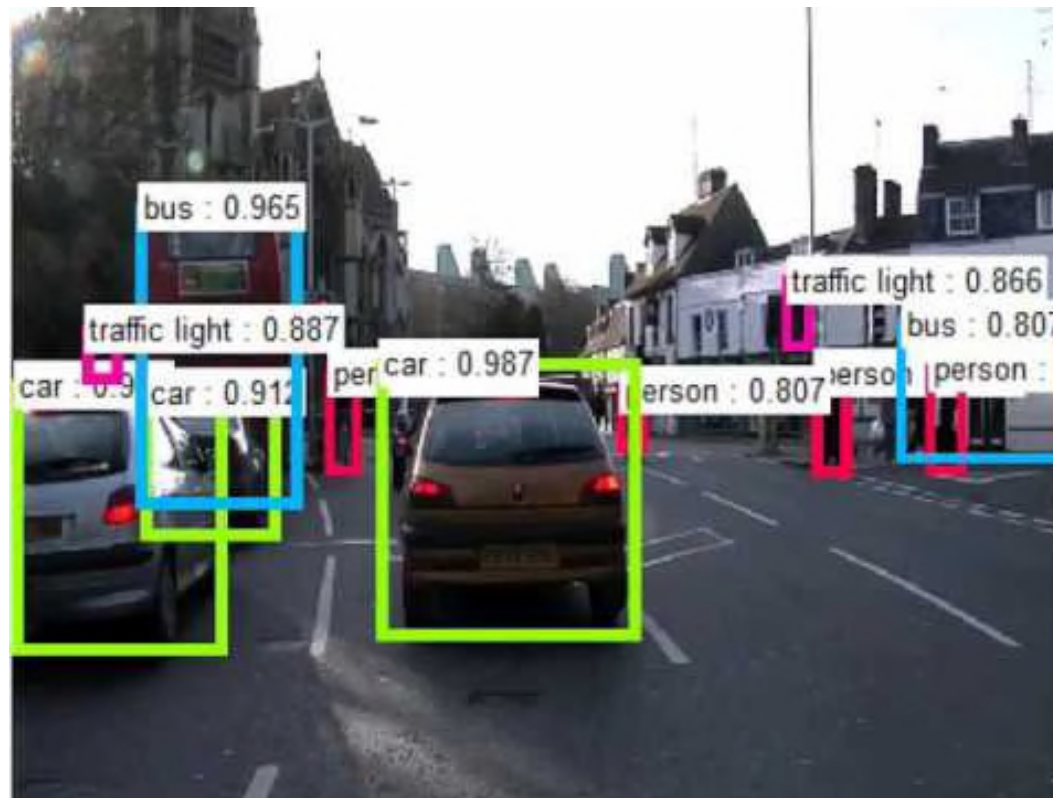
- To predict object classes in an image / video and localize them using bounding boxes.
- A regression + classification task.



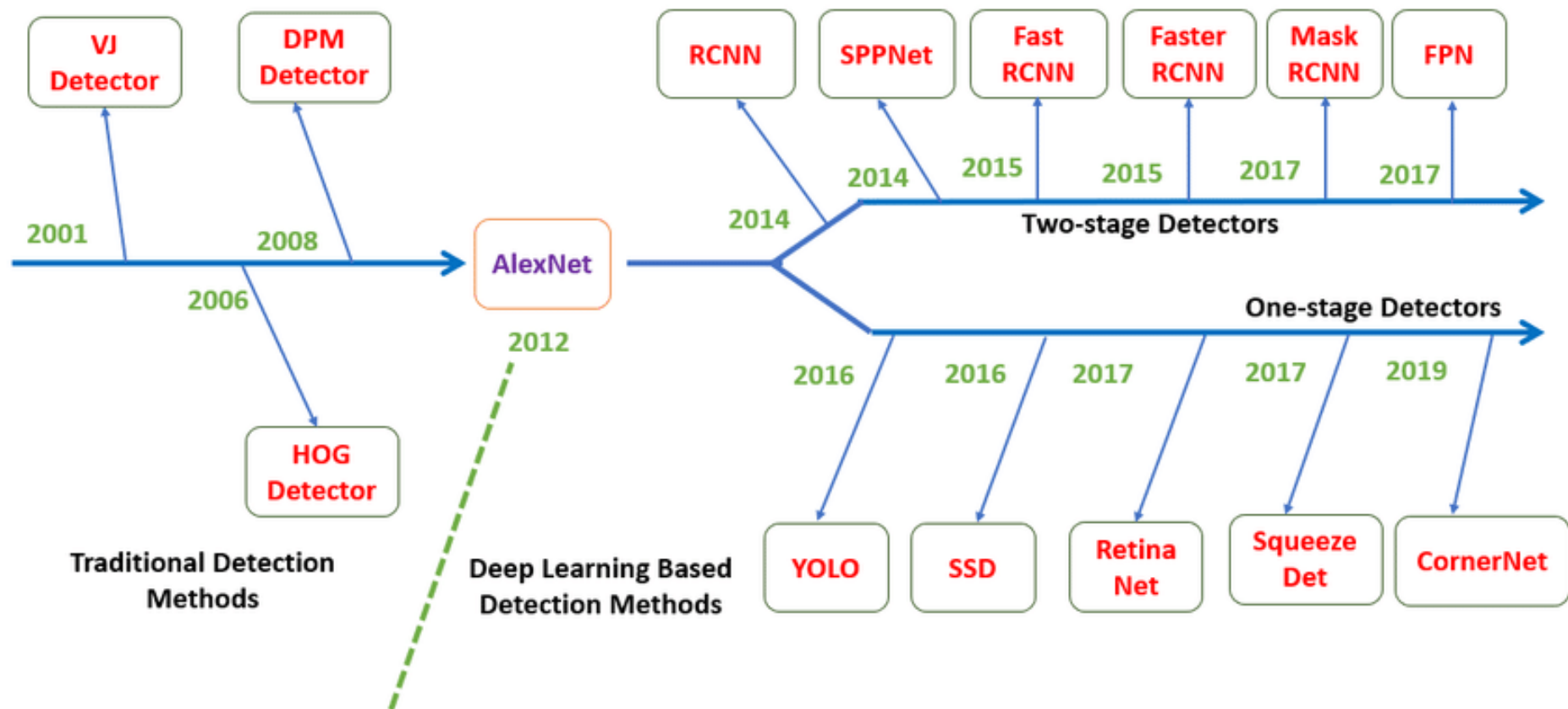
# Applications

- A key step for many downstream CV tasks:
  - Object tracking
  - Pose estimation
  - Action recognition
  - etc.
- Object detection has many real-life applications:
  - Autonomous driving
  - Video surveillance / monitoring
  - etc.

# Application: Autonomous Driving



# Brief History & Milestones



# Challenges

- Different imaging conditions including variations in viewpoints, scales, lighting, background clutter, occlusion, etc.
- Computational efficiency.
- Close-world vs open-world problem.
- Etc.

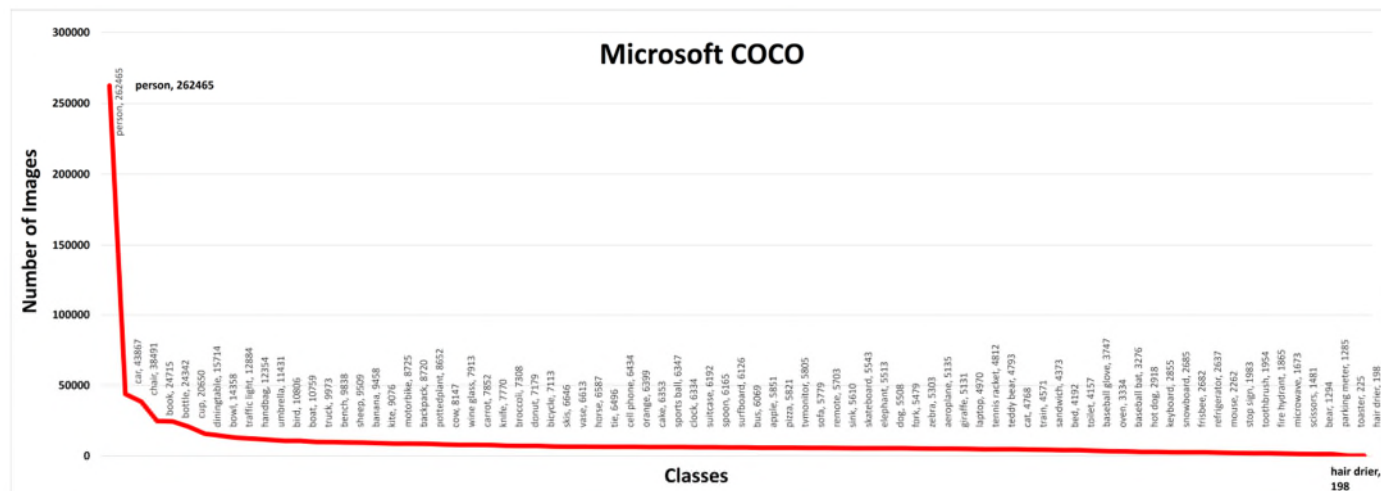
# Common Datasets

- Microsoft Common Objects in Context (MS-COCO)
- ImageNet
- Google Open Images

| Dataset     | Classes | Train     |            |               | Validation |         |               | Test    |
|-------------|---------|-----------|------------|---------------|------------|---------|---------------|---------|
|             |         | Images    | Objects    | Objects/Image | Images     | Objects | Objects/Image |         |
| MS-COCO     | 80      | 118,287   | 860,001    | 7.27          | 5,000      | 36,781  | 7.35          | 40,670  |
| ImageNet    | 200     | 456,567   | 478,807    | 1.05          | 20,121     | 55,501  | 2.76          | 40,152  |
| Open Images | 600     | 1,743,042 | 14,610,229 | 8.38          | 41,620     | 204,621 | 4.92          | 125,436 |

# MS COCO

- Key benchmark dataset for training and evaluation
- Train, validation & test sets contain more than 200,000 images and 80 object categories.





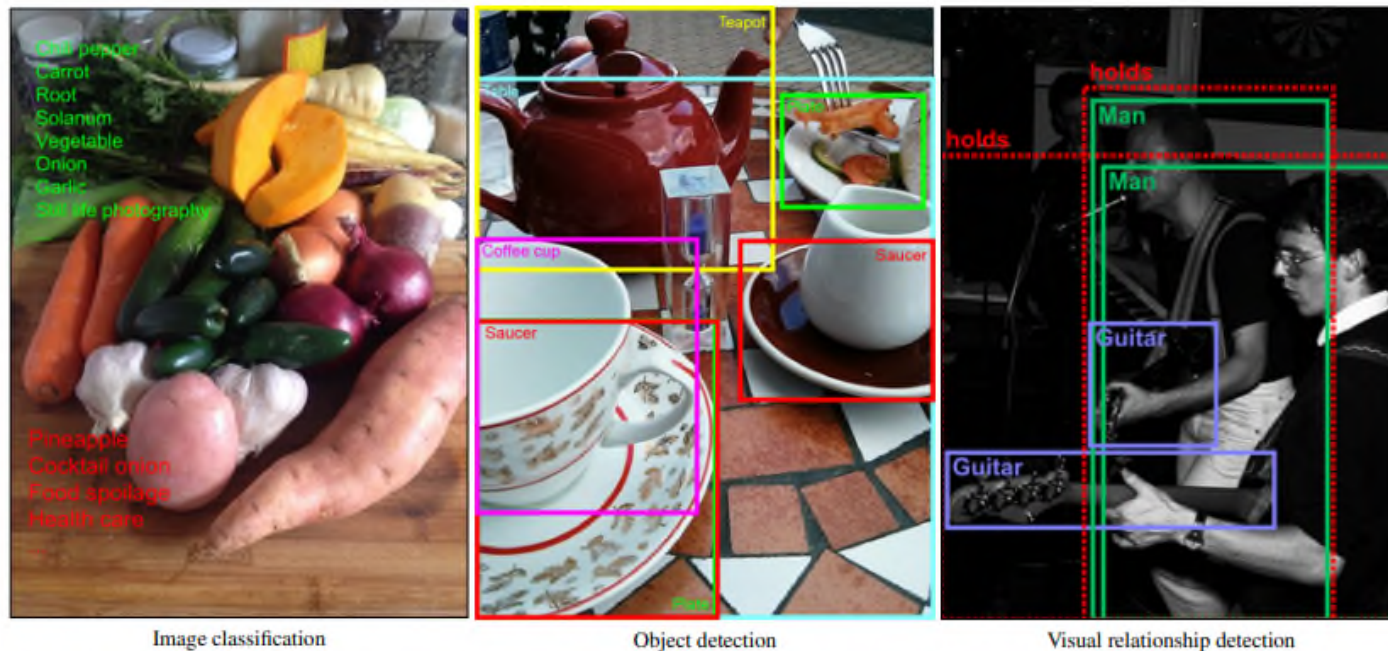
# ImageNet

- Consist of 1000 image classification classes.
- 200 classes hand-picked for object detection.
- More than 500k images for object detection.



# Google Open Images

- Contain 1.9 million images for object detection.
  - 16 million bounding boxes.
  - 600 categories.

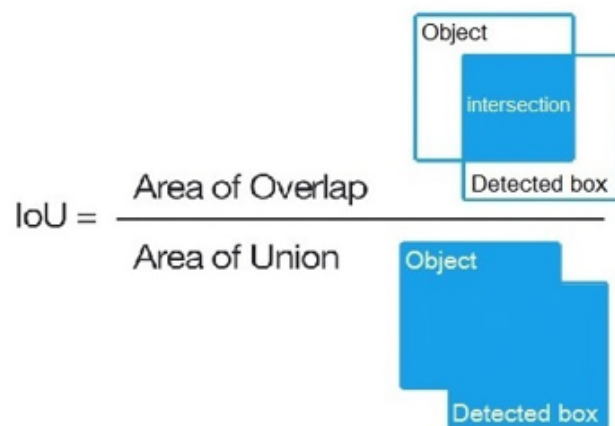


**Fig. 1** Example annotations in Open Images for image classification, object detection, and visual relationship detection. For image classification, positive labels (present in the image) are in green while negative labels (not present in the image) are in red. For visual relationship detection, the box with a dashed contour groups the two objects that hold a certain visual relationship.

# Performance Metrics

- mean Average Precision (mAP), also loosely known as AP.
  - A metric used to evaluate the accuracy of object detection models.
  - Dependent on chosen value of Intersection over Union (IoU).
  - A prediction is considered as True Positive (TP) if IoU > threshold, and False Positive (FP) if IoU < threshold.
  - Common AP: AP50 or  $AP_{0.5}$  ,  $AP_{0.50:0.05:0.95}$

$$mAP_{COCO} = \frac{mAP_{0.50} + mAP_{0.55} + \dots + mAP_{0.95}}{10}$$



# mean Average Precision (mAP)

- mAP is calculated by finding AP for each class and then average over all classes.
- Note: some papers call mAP loosely as AP and assume the difference is clear from context.
- Steps to calculate mAP:
  - Generate the predicted object bounding boxes and labels using model
  - Calculate the confusion matrix: True Positives (TP) , False Positives (FP), False Negatives (FN)
  - Calculate the Precision and Recall metrics
  - Calculate the AP for current class: the area under the precision-recall curve
  - Calculate mAP as the Average Precision (AP) over all classes.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

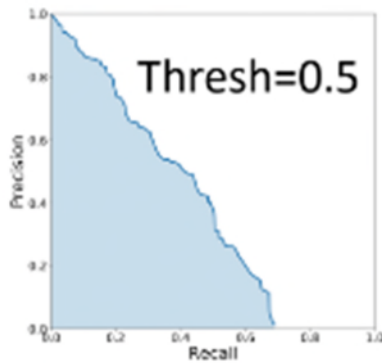
*Mean Average Precision Formula*

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\# \text{ ground truths}}$$

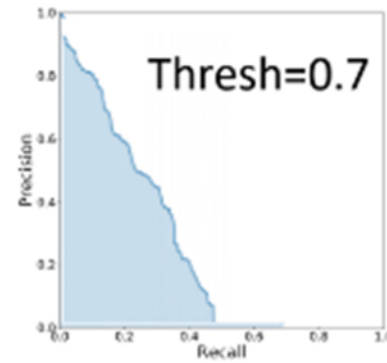
$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{\text{TP}}{\# \text{ predictions}}$$



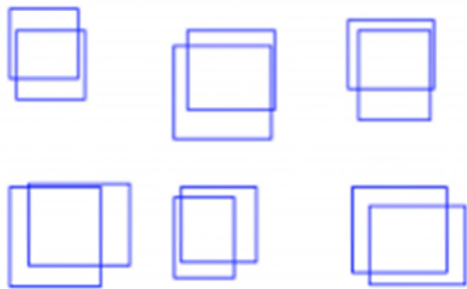
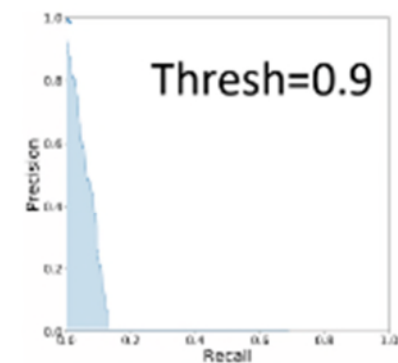
# mean Average Precision (mAP)



...

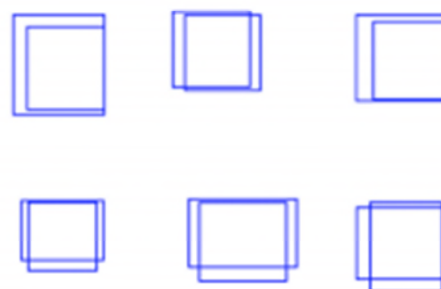


...

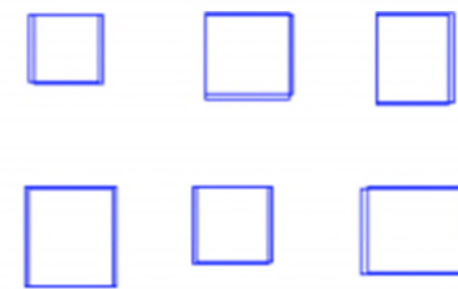


$\text{IoU} = 0.5$

Loose



$\text{IoU} = 0.7$



$\text{IoU} = 0.9$

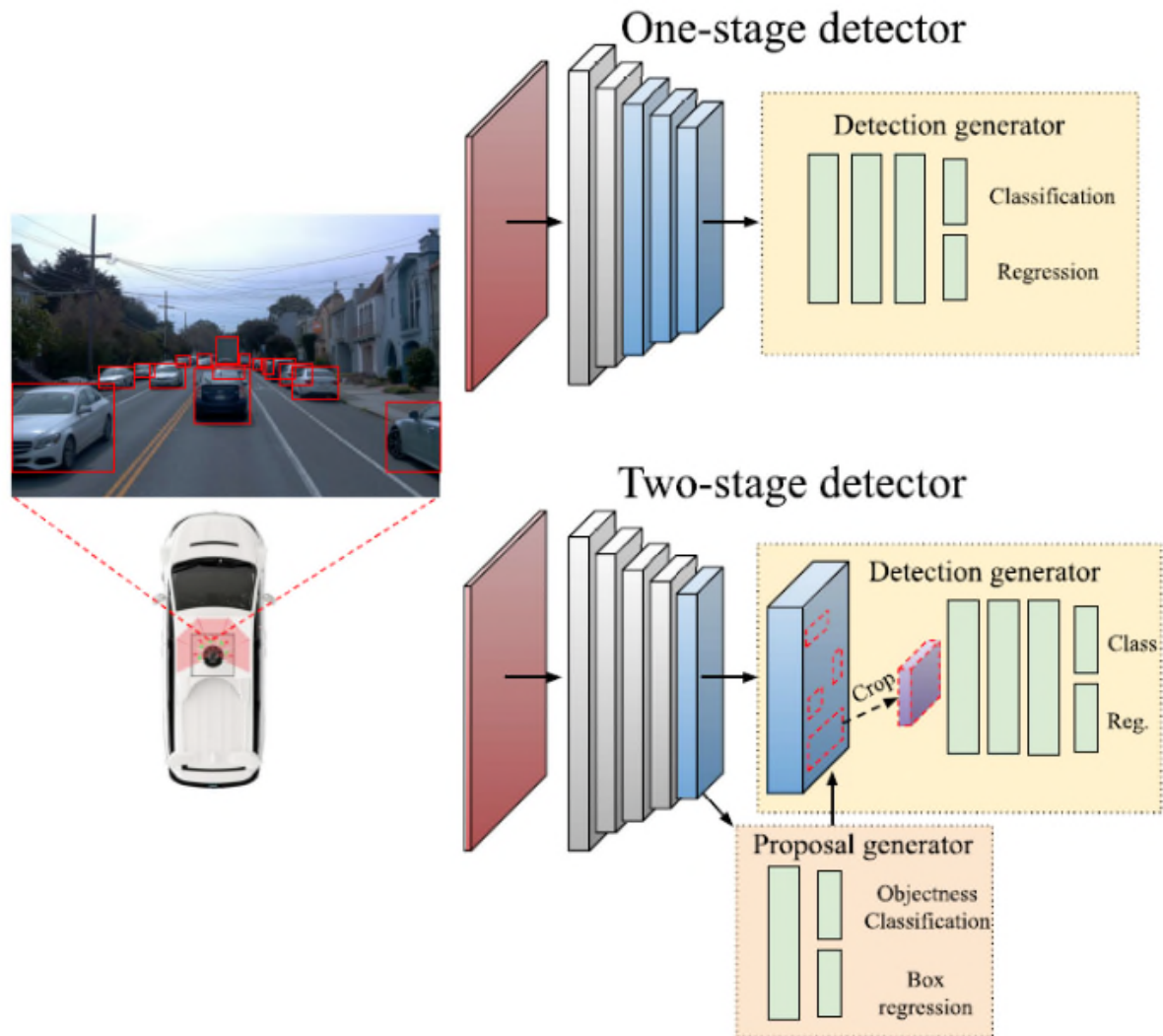
Tight

# Floating Point Operations (FLOPs)

- A metric used to evaluate computational complexity of any AI models.
- Often used to describe how many operations are required to run a single instance for a given model.
- Measure the total number of floating-point operations required for a single forward pass.
- Reflect the detection time of different models.

# Object Detector Categories

- Main detector categories:
  - One-stage detectors
  - Two-stage detectors
- Lightweight detectors



# Two-Stage Detectors



# Two-Stage Detectors

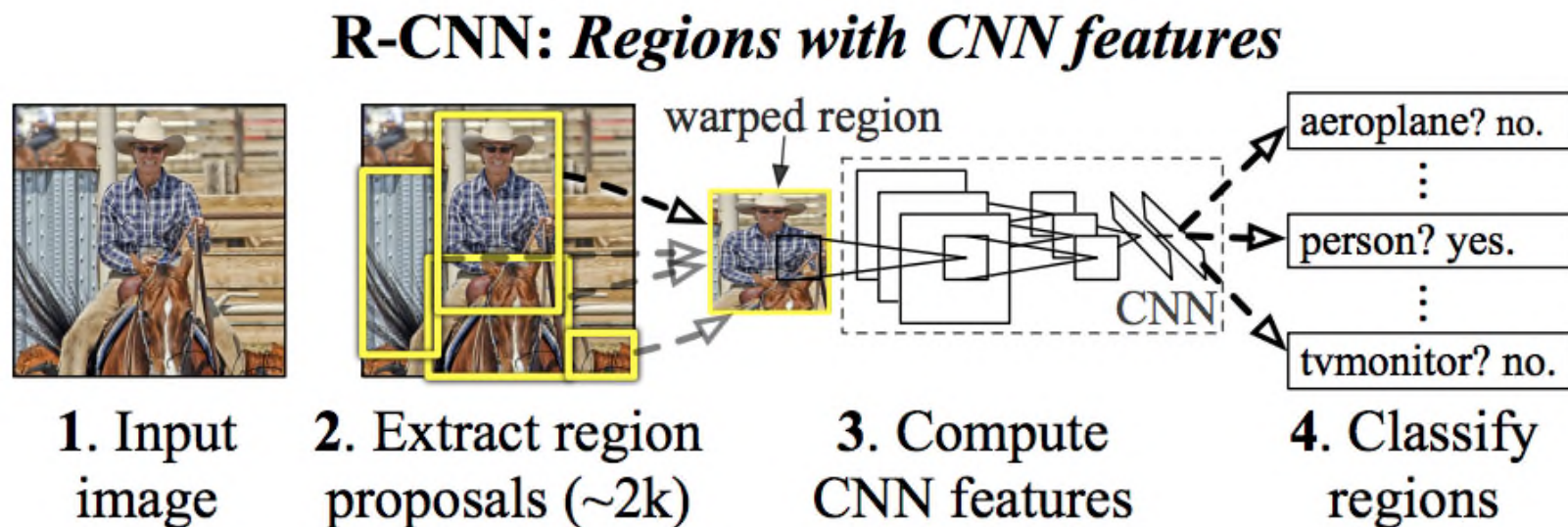
- Use techniques such as Region Proposal Network (RPN) to propose possible object regions.
- Representative methods: Faster-RCNN & Mask-RCNN.
- Other methods: SPP-Net, FPN, etc.
- Compared to one-stage detectors, generally:
  - Pros: higher accuracy (in the past).
  - Cons: more complex and slower.

# Region-based Convolutional Neural Network (RCNN) Series Detectors

- R-CNN
- Fast R-CNN
- Faster R-CNN
- Mask R-CNN

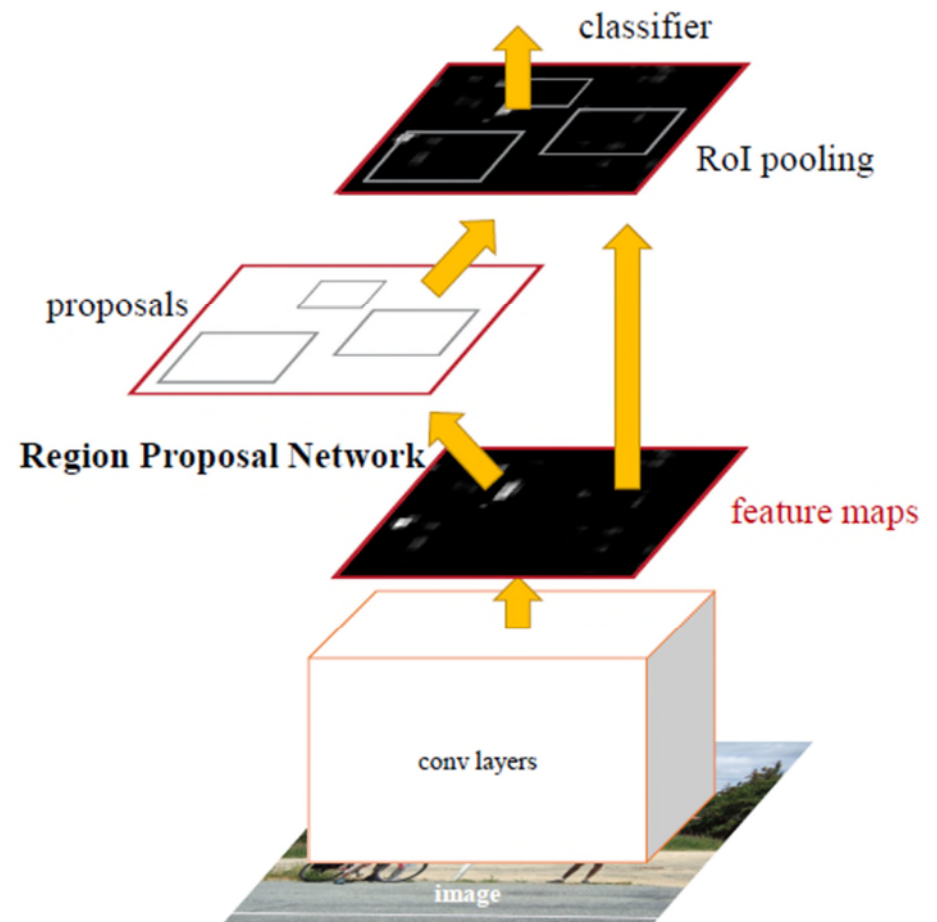
# Region-based Convolutional Neural Network (R-CNN)

- Generate region proposals (~2000 regions).
- Extract feature vector from each region using CNN.
- Classify each region using class-specific linear Support Vector Machine (SVM) and perform bounding-box regressions.

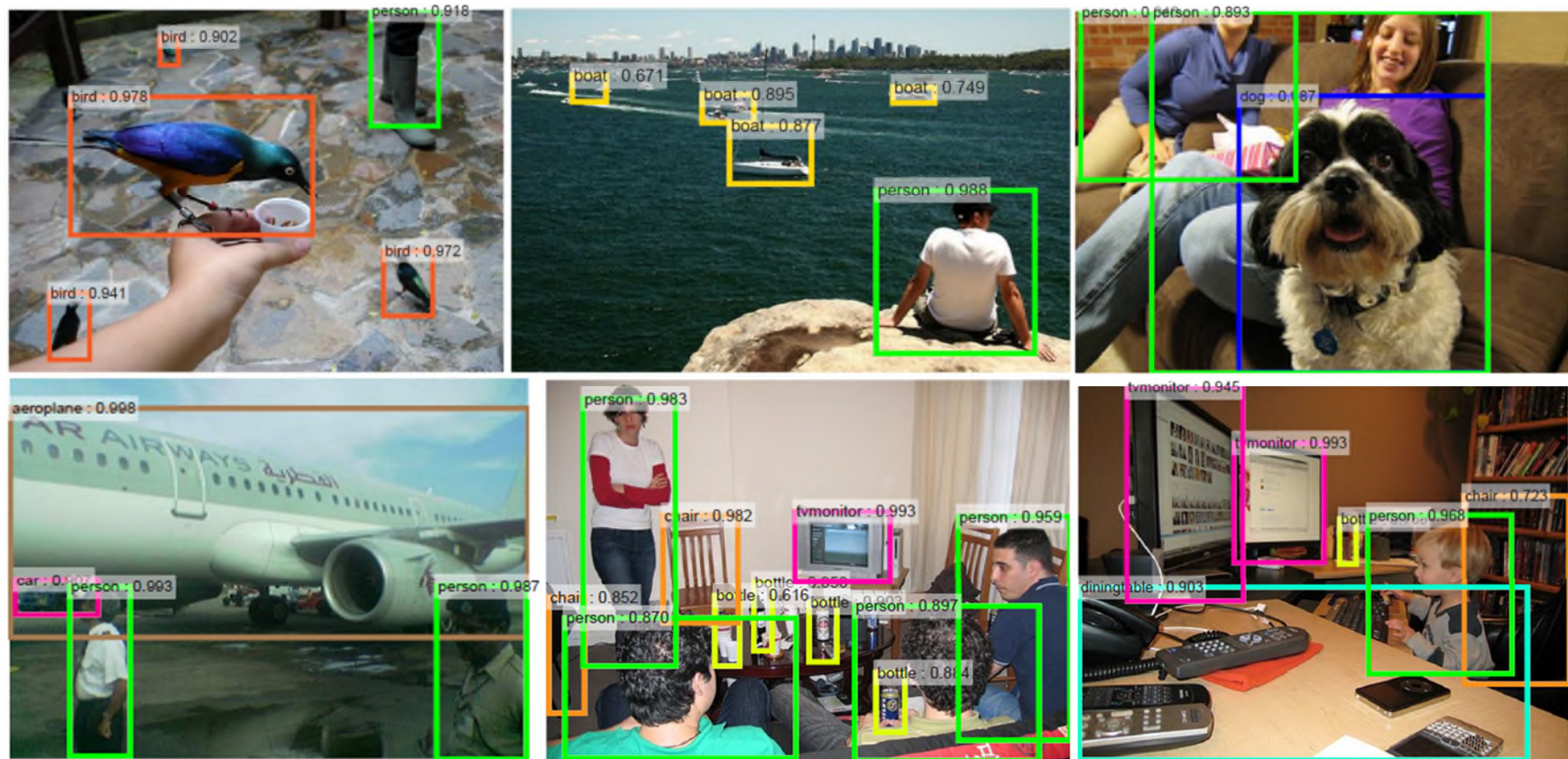


# Faster R-CNN

- Introduce a Region Proposal Network (RPN).
- RPN shares full-image convolutional features with detection branch, hence enabling nearly cost-free region proposals.
- RPN simultaneously predicts object boundaries and objectness scores.

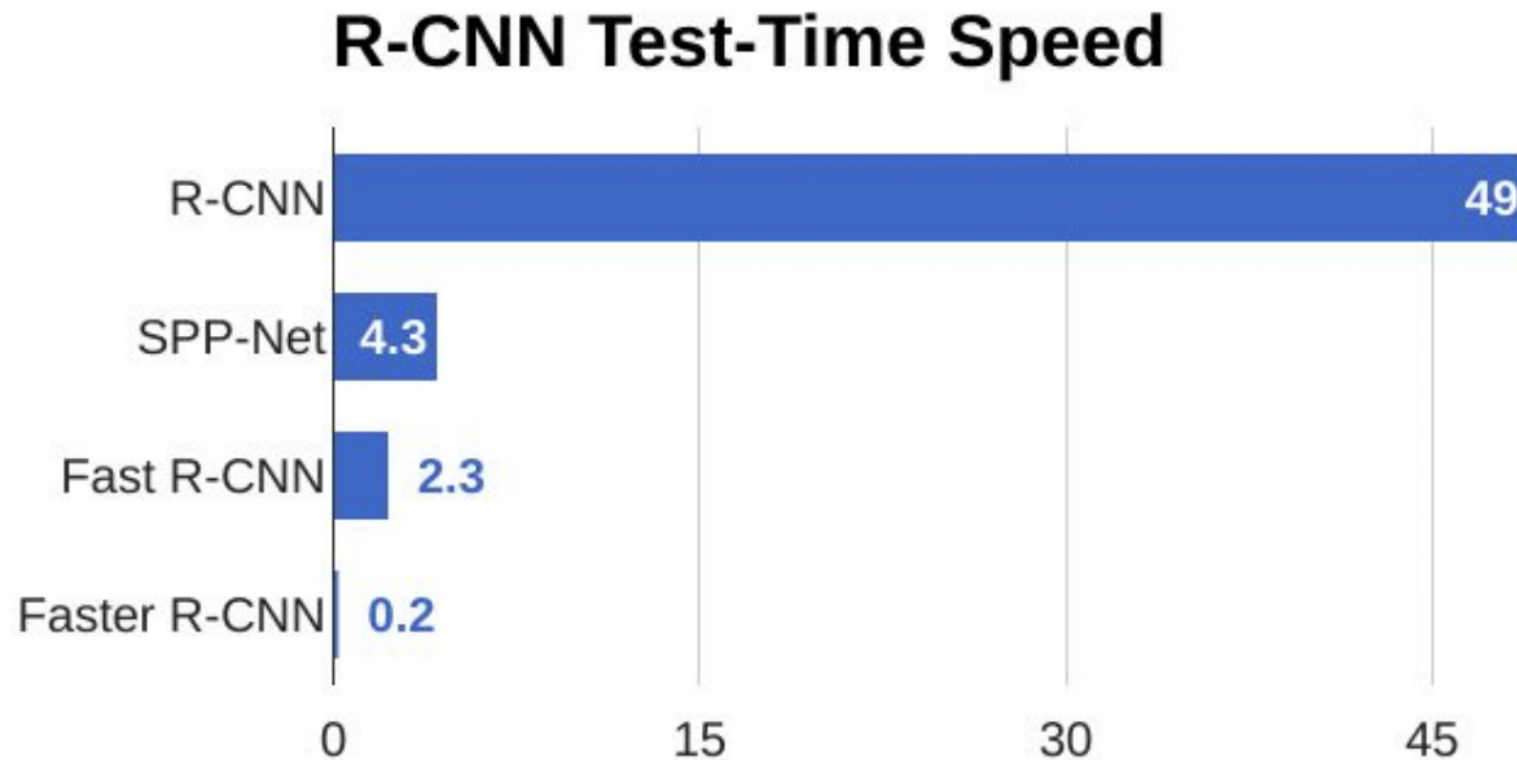


# Detection Results of Faster R-CNN





# Speed Comparison



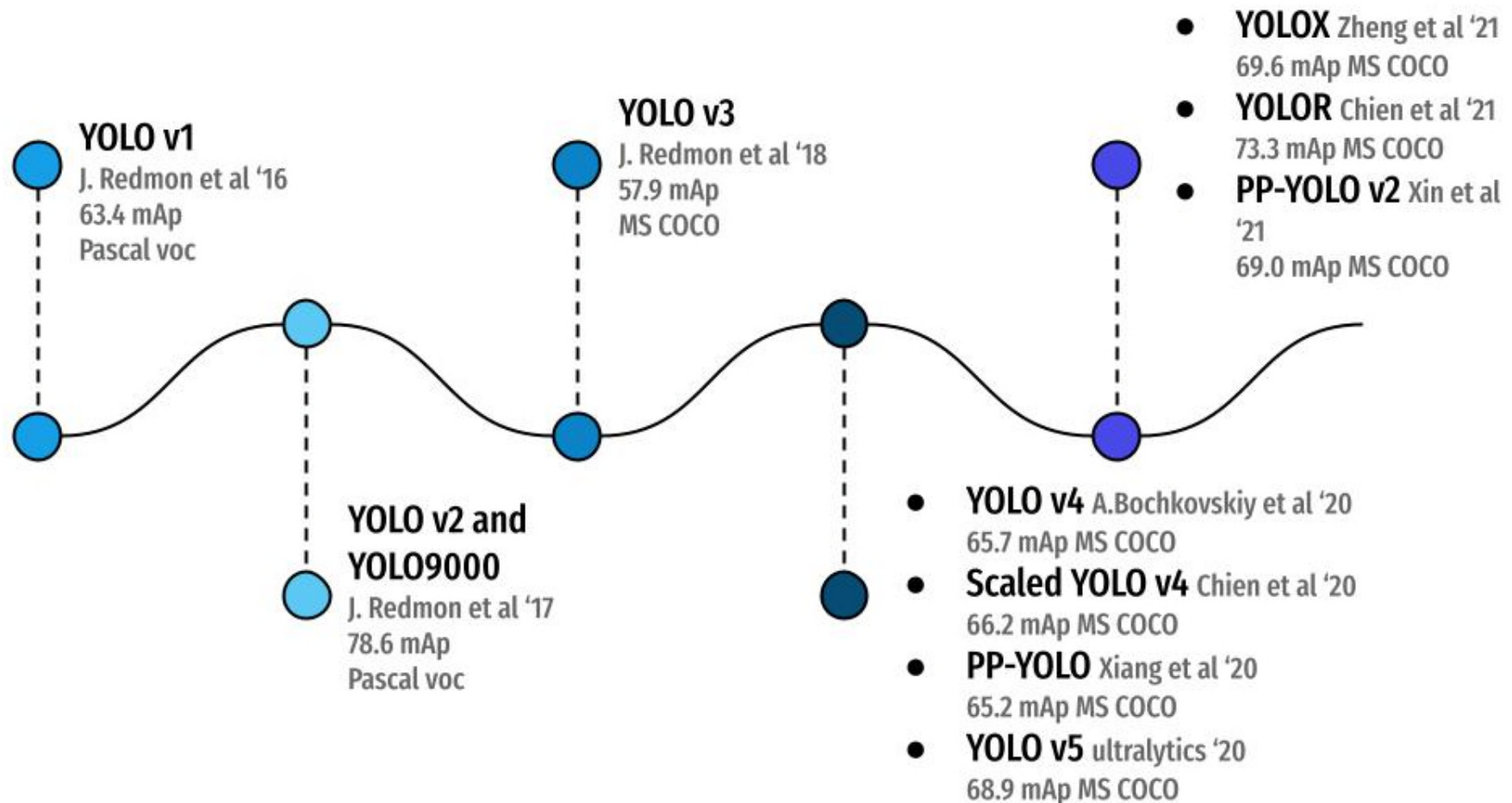
# One-Stage Detectors



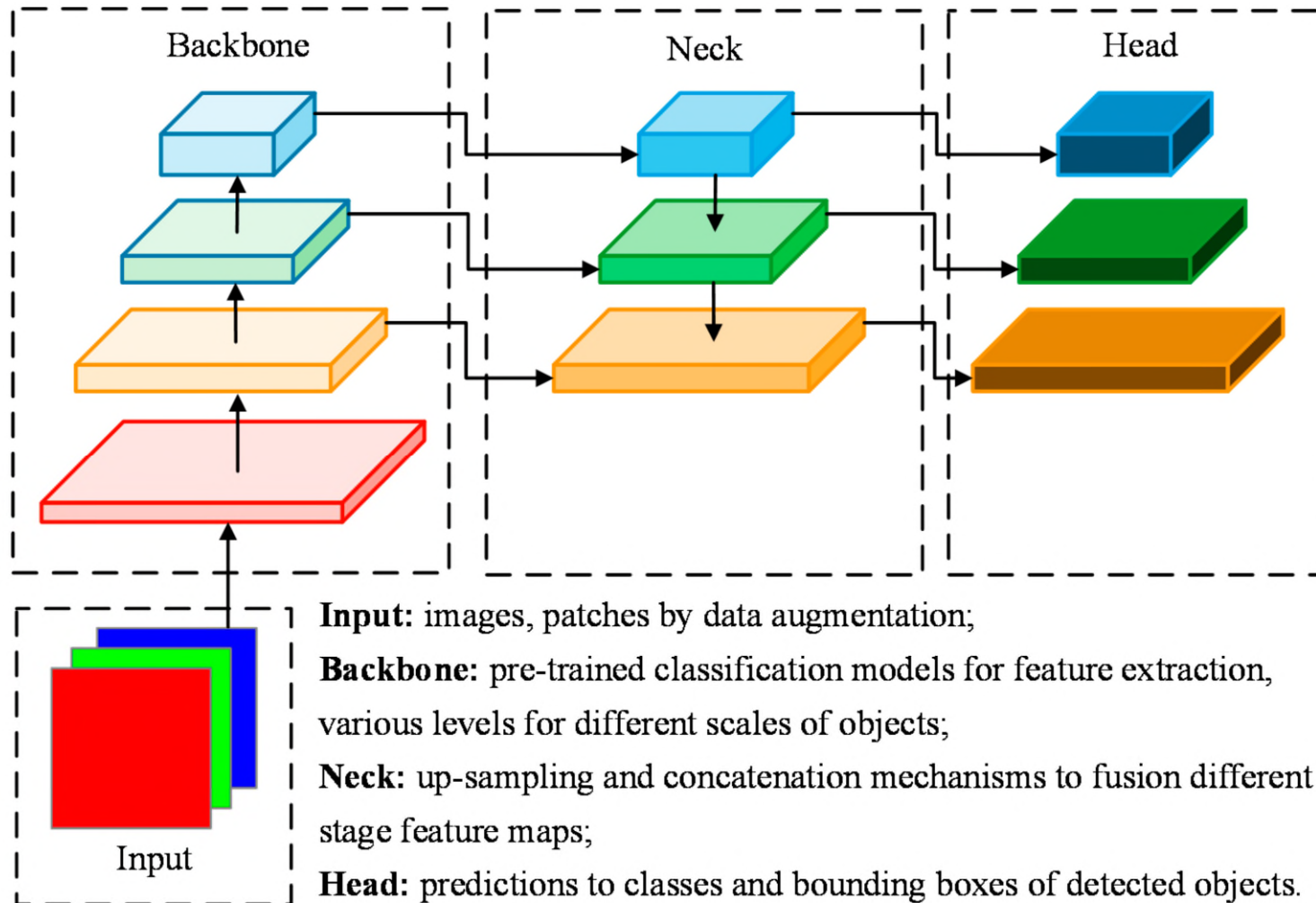
# One-Stage Detectors

- One stage detectors regress and classify from the feature map directly.
- Representative methods: Yolo, SSD.
- Other methods: CenterNet, EfficientDet, etc.
- Compared to two-stage detectors, generally:
  - Pros: lower complexity and faster speed
  - Cons: lower accuracy (for older methods)

# You Only Look Once (YOLO) Series Detectors



# YOLO General Architecture



# Recent YOLO Detectors

- Current SOTA object detection models.
- YOLOv7: 2022; YOLOv10: 2024, YOLOv12: 2025

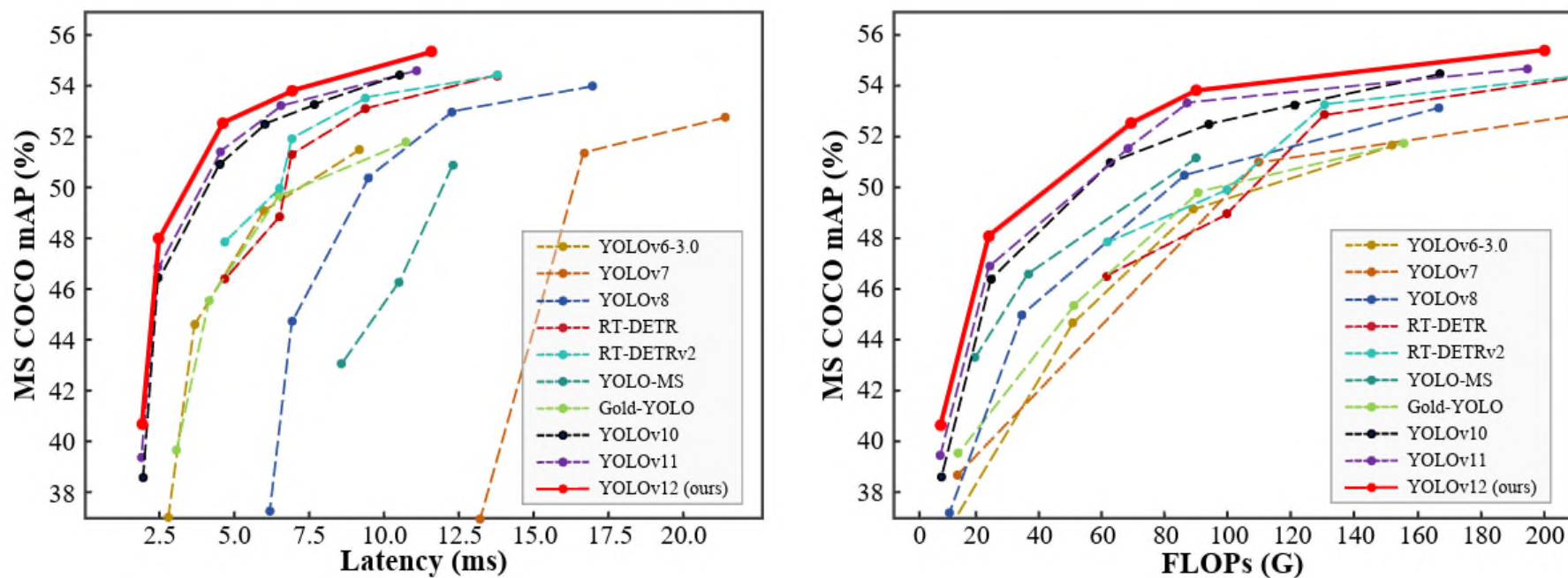


Figure 1. Comparisons with other popular methods in terms of latency-accuracy (left) and FLOPs-accuracy (right) trade-offs.

# YOLO Comparison





# Lightweight Detectors

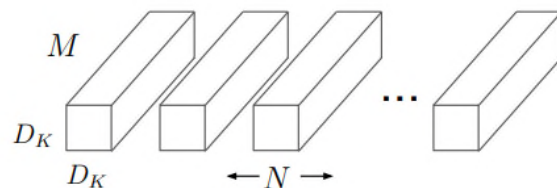
# Lightweight Detectors

- Designed for resource limited devices.
- Small, fast and efficient networks.
- Representative method: MobileNetV2-based SSD (MediaPipe).
- Other methods: SqueezeNet, ShuffleNet, etc.
- Pros: ultra fast, efficient.
- Cons: less accurate.

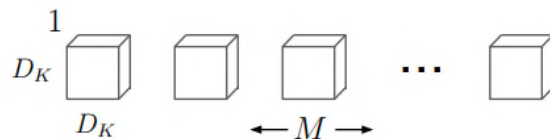


# MobileNet

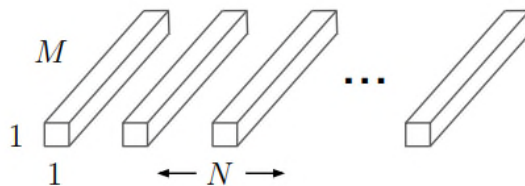
- Use depth-wise separable convolutions to reduce the computation.
  - Depth-wise convolution applies a single filter to each input channel.
  - Pointwise convolution is then applied using 1x1 convolution.



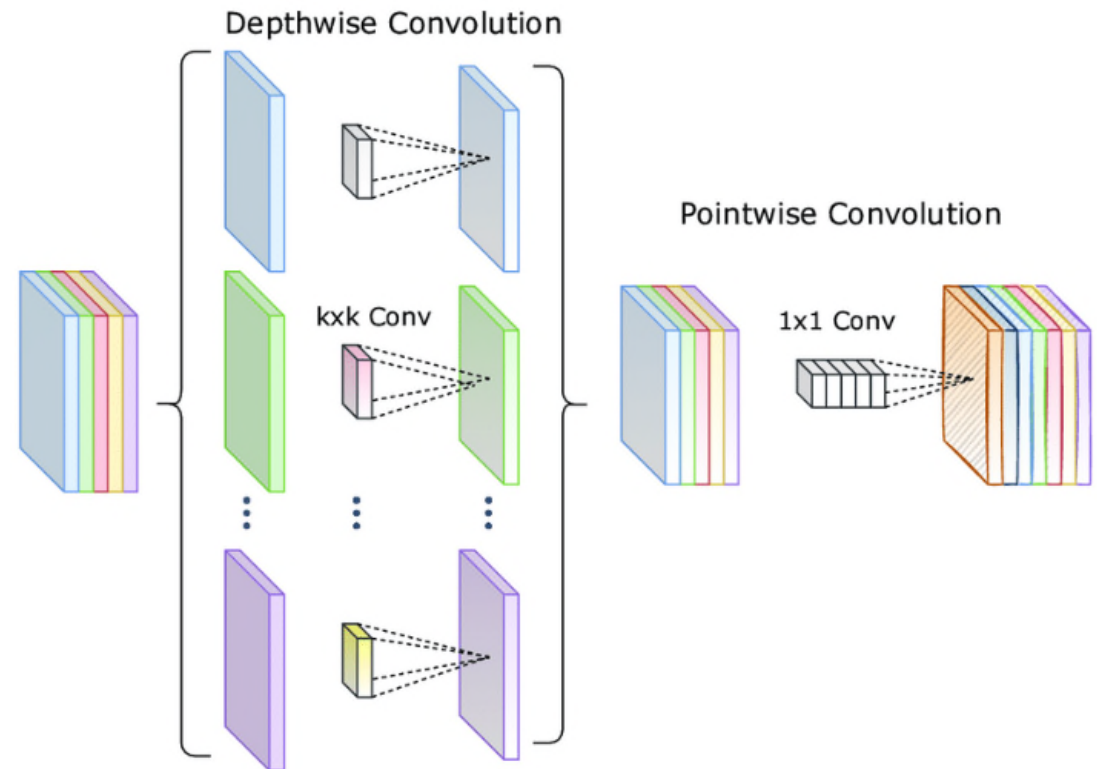
(a) Standard Convolution Filters



(b) Depthwise Convolutional Filters



(c)  $1 \times 1$  Convolutional Filters called Pointwise Convolution in the context of Depthwise Separable Convolution



Depthwise separable convolutions.

Emerging / New Methods

# Swin Transformer

- A popular recent image classification / object detection method.
- Serve as general-purpose backbone for computer vision.
- Hierarchical feature maps.

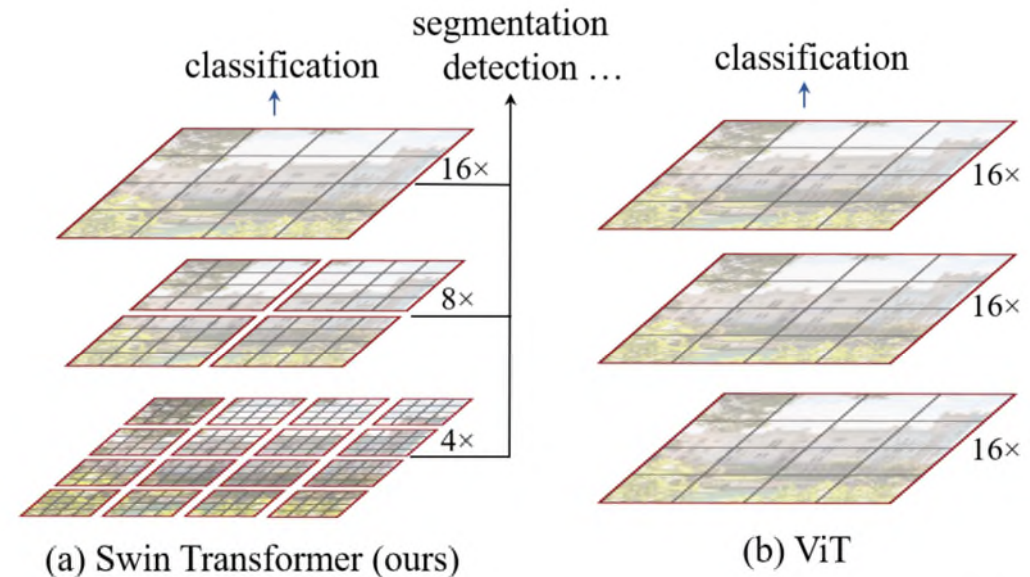
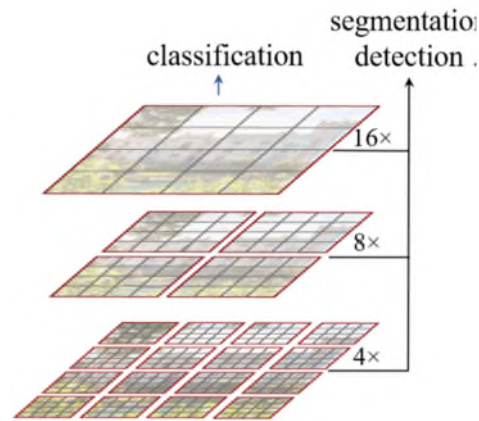


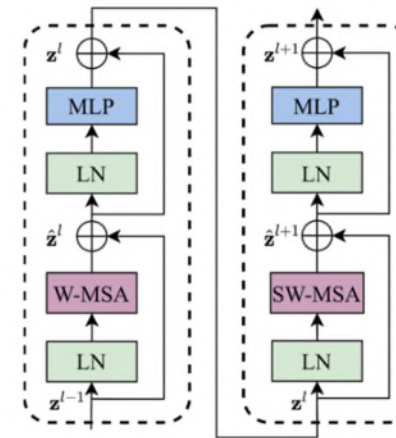
Figure 1. (a) The proposed Swin Transformer builds hierarchical feature maps by merging image patches (shown in gray) in deeper layers and has linear computation complexity to input image size due to computation of self-attention only within each local window (shown in red). It can thus serve as a general-purpose backbone for both image classification and dense recognition tasks. (b) In contrast, previous vision Transformers [19] produce feature maps of a single low resolution and have quadratic computation complexity to input image size due to computation of self-attention globally.

# Swin Transformer Architecture

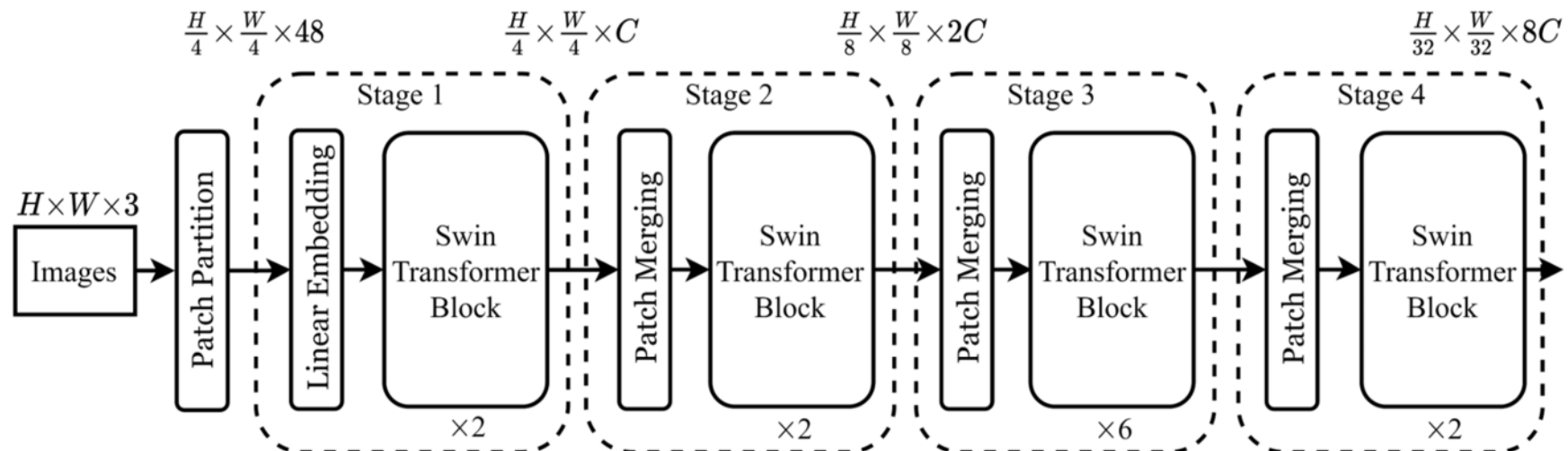
- Split an input RGB image into non-overlapping 4 x 4 patches.
- Propose Swin Transformer Block for learning.



(a) Swin Transformer



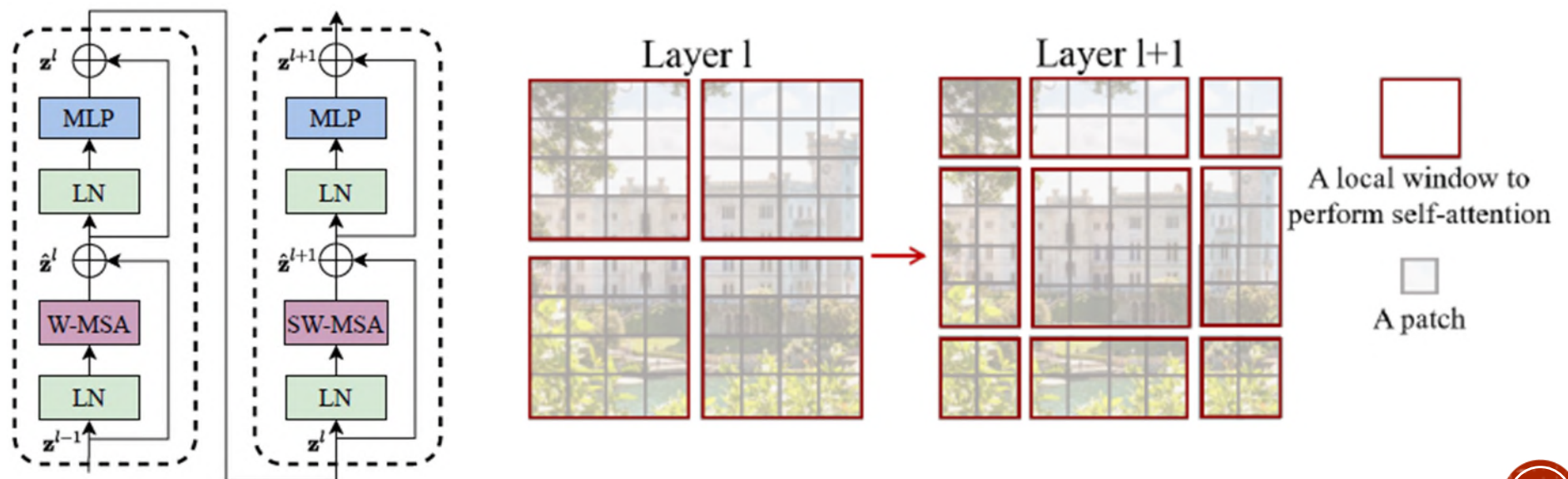
Two Successive Swin Transformer Blocks



(d) Architecture

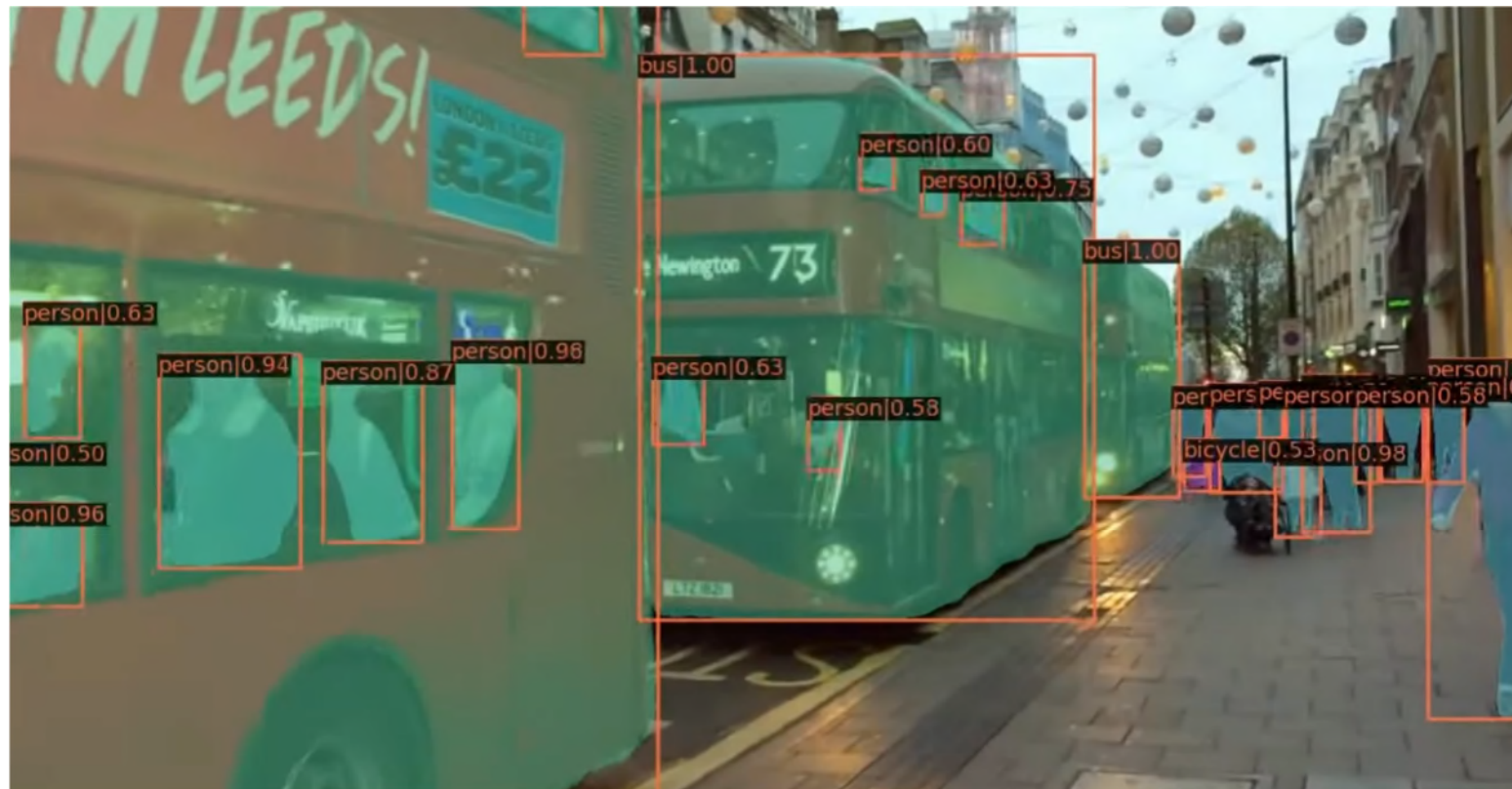
# Swin Transformer Block

- W-MSA: Window-based Multi-head Self-Attention
  - In layer  $l$ , a regular window partitioning scheme is adopted, and self-attention is computed within each window.
- SW-MSA: Shifted Window-based Multi-head Self-Attention
  - In the next layer  $l + 1$ , the window partitioning is shifted, resulting in new shifted windows.

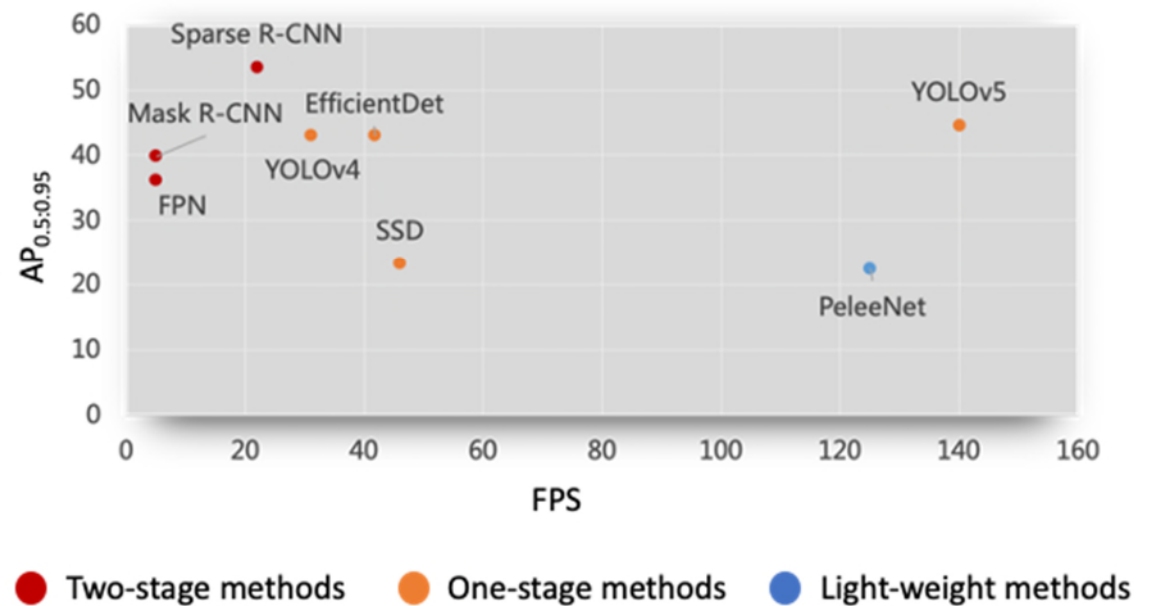
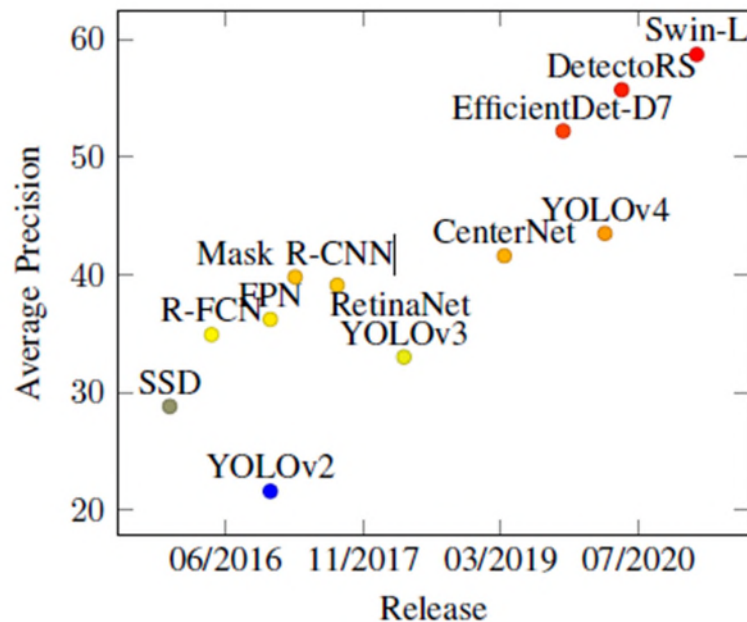




# Demo: Swin Transformer



# Performance Comparison on COCO Dataset





# Detection Transformer (DETR): End-to-End Object Detection With Transformers

- A popular recent object detection method.
- Predict in parallel the final set of detections by combining a common CNN with a transformer architecture.
- Input to decoder is a given sequence of positional encoding (object queries).

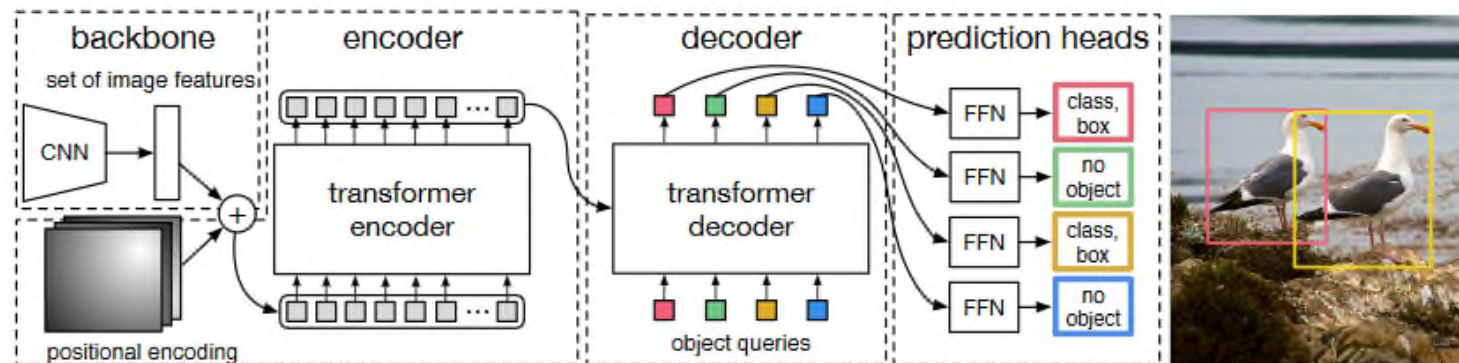
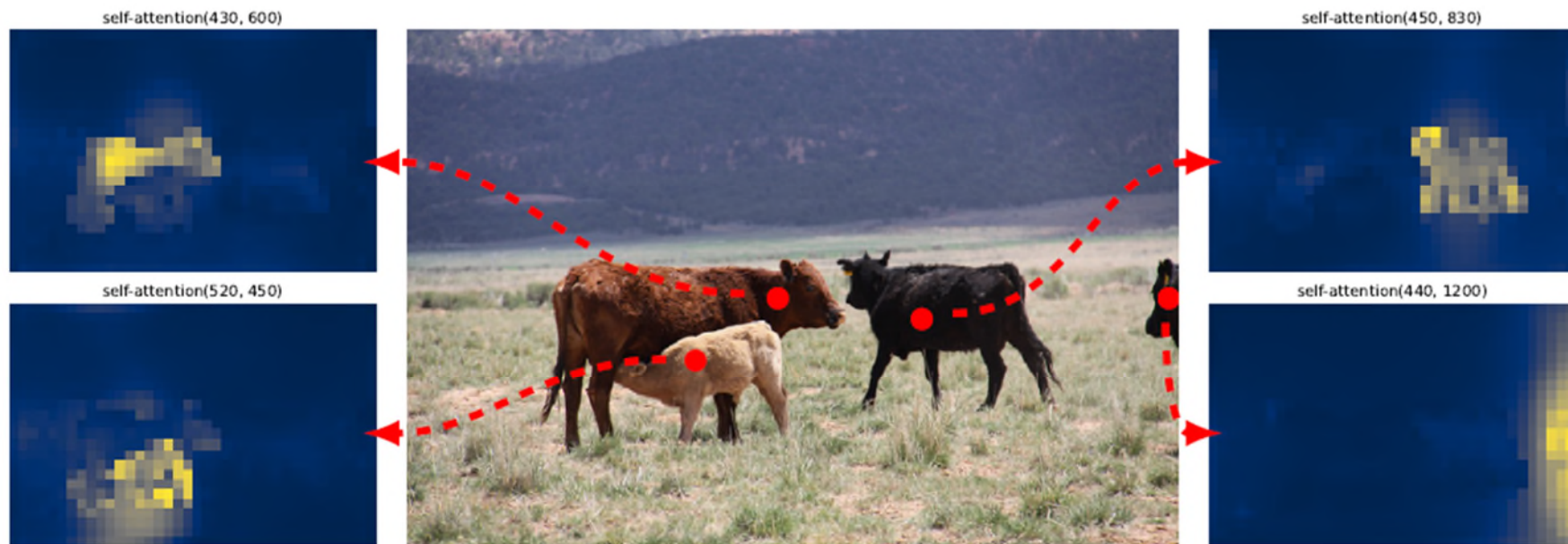


Fig. 2: DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call *object queries*, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a “no object” class.

# DETR: Experimental Results

| Model                 | GFLOPS/FPS | #params | AP          | AP <sub>50</sub> | AP <sub>75</sub> | AP <sub>S</sub> | AP <sub>M</sub> | AP <sub>L</sub> |
|-----------------------|------------|---------|-------------|------------------|------------------|-----------------|-----------------|-----------------|
| Faster RCNN-DC5       | 320/16     | 166M    | 39.0        | 60.5             | 42.3             | 21.4            | 43.5            | 52.5            |
| Faster RCNN-FPN       | 180/26     | 42M     | 40.2        | 61.0             | 43.8             | 24.2            | 43.5            | 52.0            |
| Faster RCNN-R101-FPN  | 246/20     | 60M     | 42.0        | 62.5             | 45.9             | 25.2            | 45.6            | 54.6            |
| Faster RCNN-DC5+      | 320/16     | 166M    | 41.1        | 61.4             | 44.3             | 22.9            | 45.9            | 55.0            |
| Faster RCNN-FPN+      | 180/26     | 42M     | 42.0        | 62.1             | 45.5             | 26.6            | 45.4            | 53.4            |
| Faster RCNN-R101-FPN+ | 246/20     | 60M     | 44.0        | 63.9             | <b>47.8</b>      | <b>27.2</b>     | 48.1            | 56.0            |
| DETR                  | 86/28      | 41M     | 42.0        | 62.4             | 44.2             | 20.5            | 45.8            | 61.1            |
| DETR-DC5              | 187/12     | 41M     | 43.3        | 63.1             | 45.9             | 22.5            | 47.3            | 61.1            |
| DETR-R101             | 152/20     | 60M     | 43.5        | 63.8             | 46.4             | 21.9            | 48.0            | 61.8            |
| DETR-DC5-R101         | 253/10     | 60M     | <b>44.9</b> | <b>64.7</b>      | 47.7             | 23.7            | <b>49.5</b>     | <b>62.3</b>     |

# DETR: Visualization



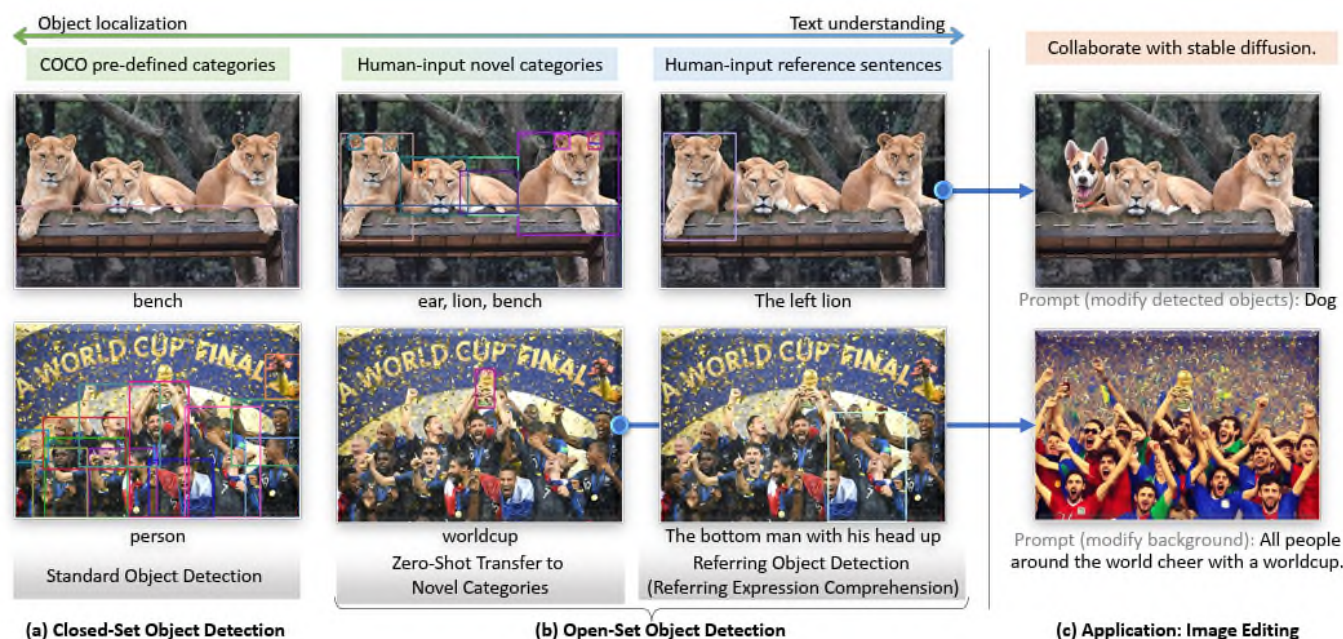
# Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection

- An open-set object detection model.
- Combine DINO (DETR with Improved DeNoising Anchor Boxes) with Grounding (aligning text and image).
- Grounding DINO = DETR-style detector + CLIP-style text grounding, enabling open-set detection with text prompts.



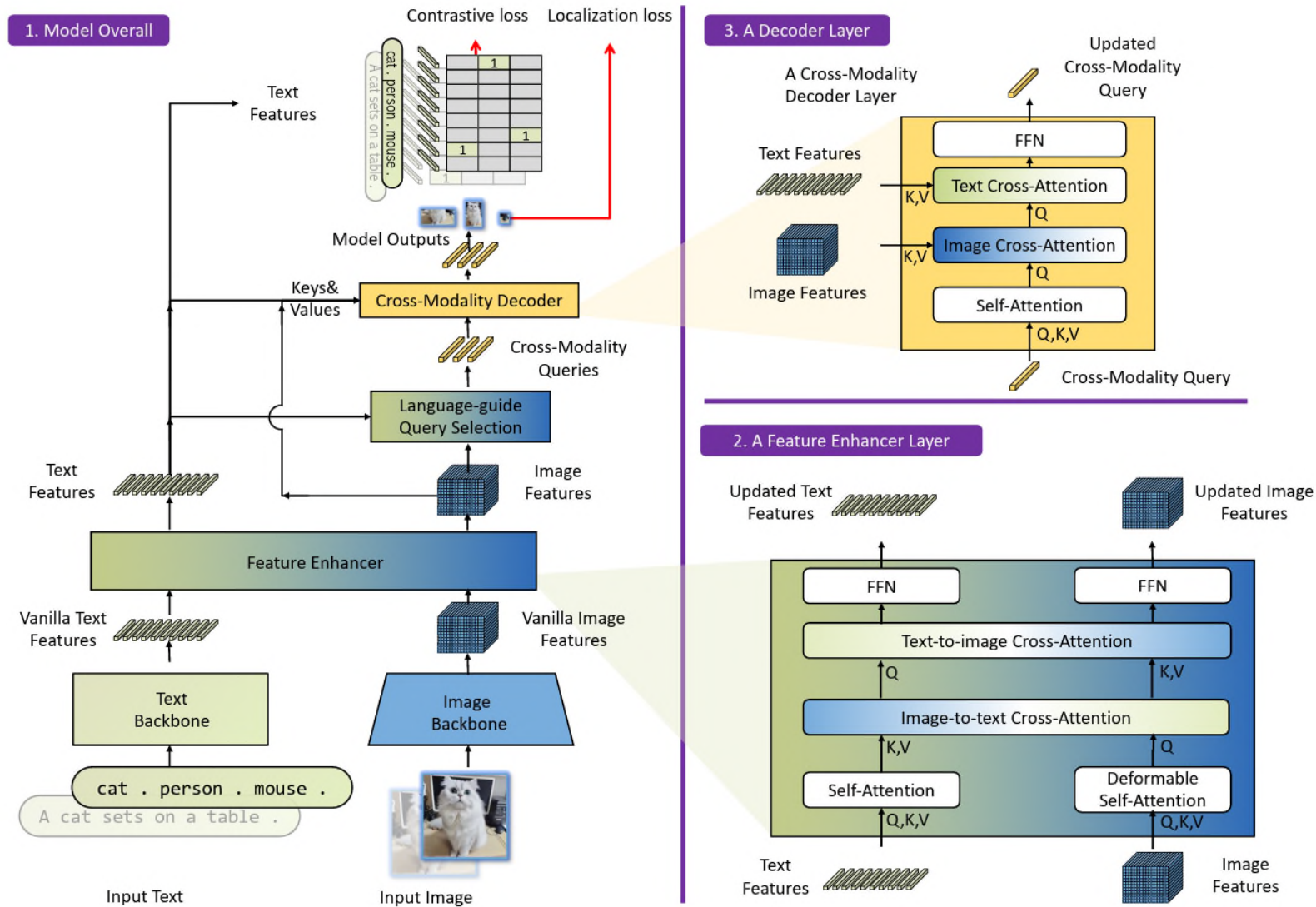
# Grounding DINO: Motivation

- Allow detection of arbitrary objects specified by natural language text prompts.
- Achieve state-of-the-art open-vocabulary detection performance.



**Fig. 1:** (a) Closed-set object detection requires models to detect objects of pre-defined categories. (b) We evaluate models on novel objects and standard Referring expression comprehension (REC) benchmarks for model generalizations on novel objects with attributes. (c) We present an image editing application by combining Grounding DINO and Stable Diffusion [41]. Best viewed in colors.

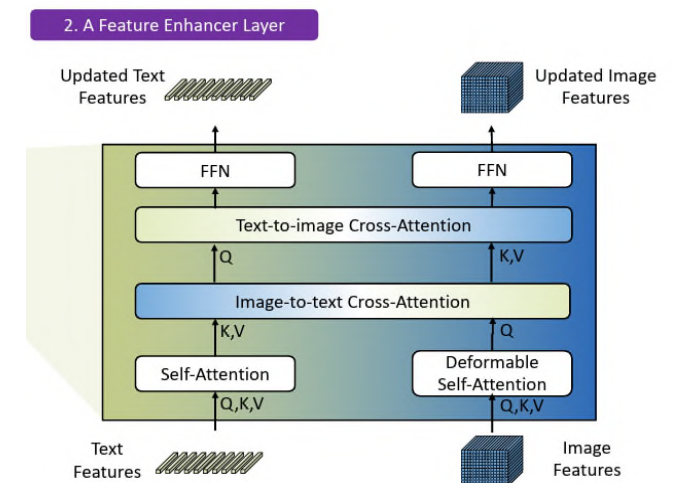
# Grounding DINO: Framework



**Fig. 3:** The framework of Grounding DINO. We present the overall framework, a feature enhancer layer, and a decoder layer in block 1, block 2, and block 3, respectively.

# Grounding DINO: Feature Extraction and Enhancer

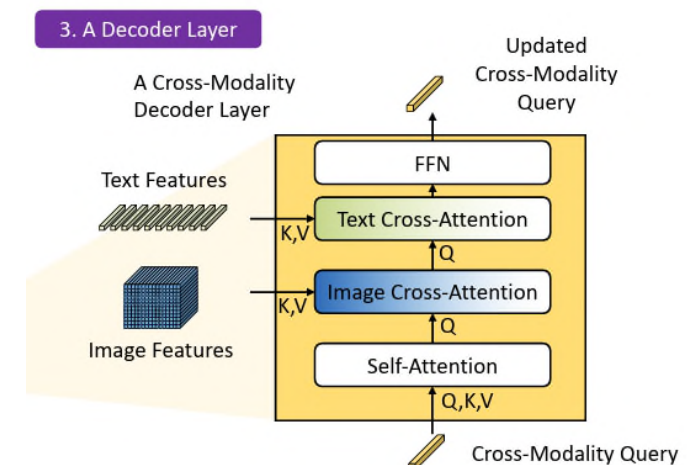
- For each (Image, Text) pair:
  - extract multi-scale image features with an image backbone like Swin Transformer.
  - extract text features with a text backbone like BERT.
- Feed vanilla image and text features into a feature enhancer for cross-modality feature fusion.
- Feature enhancer includes multiple feature enhancer layers.
- Leverage the Deformable self-attention to enhance image features and the vanilla self-attention for text feature enhancers.
- Perform image-to-text and text-to-image cross-attention for feature fusion. These modules help aligning features of different modalities.





# Grounding DINO: Language-Guided Query Selection & Cross-Modality Decoder

- Language-guided query selection
  - Select features that are more relevant to the input text as decoder queries.
- Cross-modality Decoder
  - Combine image and text modality features.
  - Each cross-modality query is fed into a self-attention layer, an image cross-attention layer to combine image features, a text cross-attention layer to combine text features, and an FFN layer in each cross-modality decoder layer.
  - Each decoder layer has an extra text cross-attention layer compared with the DINO decoder layer, as we need to inject text information into queries for better modality alignment.



# Grounding DINO: Results

| Model                   | Backbone  | Pre-Training Data             | Zero-Shot<br>2017val | Fine-Tuning<br>2017val/test-dev   |
|-------------------------|-----------|-------------------------------|----------------------|-----------------------------------|
| Faster R-CNN            | RN50-FPN  | -                             | -                    | 40.2 / -                          |
| Faster R-CNN            | RN101-FPN | -                             | -                    | 42.0 / -                          |
| DyHead-T [5]            | Swin-T    | -                             | -                    | 49.7 / -                          |
| DyHead-L [5]            | Swin-L    | -                             | -                    | 58.4 / 58.7                       |
| DyHead-L [5]            | Swin-L    | O365,ImageNet21K              | -                    | 60.3 / 60.6                       |
| SoftTeacher [50]        | Swin-L    | O365,SS-COCO                  | -                    | 60.7 / 61.3                       |
| DINO(Swin-L) [57]       | Swin-L    | O365                          | -                    | 62.5 / -                          |
| DyHead-T† [5]           | Swin-T    | O365                          | 43.6                 | 53.3 / -                          |
| GLIP-T (B) [25]         | Swin-T    | O365                          | 44.9                 | 53.8 / -                          |
| GLIP-T (C) [25]         | Swin-T    | O365,GoldG                    | 46.7                 | 55.1 / -                          |
| GLIP-L [25]             | Swin-L    | FourODs,GoldG,Cap24M          | 49.8                 | 60.8 / 61.0                       |
| DINO(Swin-T)† [57]      | Swin-T    | O365                          | 46.2                 | 56.9 / -                          |
| Grounding DINO T (Ours) | Swin-T    | O365                          | 46.7                 | 56.9 / -                          |
| Grounding DINO T (Ours) | Swin-T    | O365,GoldG                    | 48.1                 | 57.1 / -                          |
| Grounding DINO T (Ours) | Swin-T    | O365,GoldG,Cap4M              | 48.4                 | 57.2 / -                          |
| Grounding DINO L (Ours) | Swin-L    | O365,OI [19],GoldG            | <b>52.5</b>          | <b>62.6 / 62.7 (63.0 / 63.0)*</b> |
| Grounding DINO L (Ours) | Swin-L    | O365,OI,GoldG,Cap4M,COCO,RefC | <b>60.7‡</b>         | <b>62.6 / -</b>                   |

**Table 2:** Zero-shot domain transfer and fine-tuning on COCO. \* The results in brackets are trained with  $1.5\times$  image sizes, i.e., with a maximum image size of 2000. †The models map a subset of O365 categories to COCO for zero-shot evaluations. ‡This is not a real zero-shot performance as we add COCO data during model training. We list the result here for a reference.

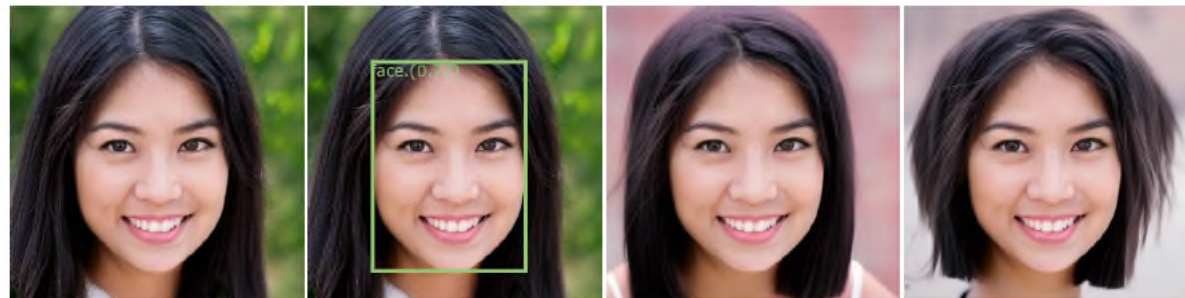
# Grounding DINO: Visualization



(c) Detection Prompt: black cat  
Generation Prompt: cats and apples



(d) Detection Prompt: the running girl  
Generation Prompt (modify background): The Wandering Earth



(e)\* Detection Prompt: face  
Generation Prompt (modify background): a girl with short hair

**Fig. 9:** Combination of Grounding DINO and Stable Diffusion. We first detect objects with Grounding DINO and then perform image inpainting with Stable Diffusion. “Detection Prompt” and “Generation Prompt” are inputs for Grounding DINO and Stable Diffusion, respectively. \*The input human face in the row (e) is generated by StyleGAN.



# Exercise: Object Detection

- (a) State clearly whether the following object detectors are one-stage detector or two-stage detector: (i) R-CNN and (ii) YOLOv7. Which of the two object detectors above is a more suitable choice if speed is the key consideration in an object detection application? Briefly justify your answer.

(7 Marks)

# Solution

# Object Detection Summary

- This section covers the following:
  - Introduction
  - Object Detection Methods
    - Two Stage Detectors
    - One Stage Detectors
    - Lightweight Detectors
  - New / Emerging Methods



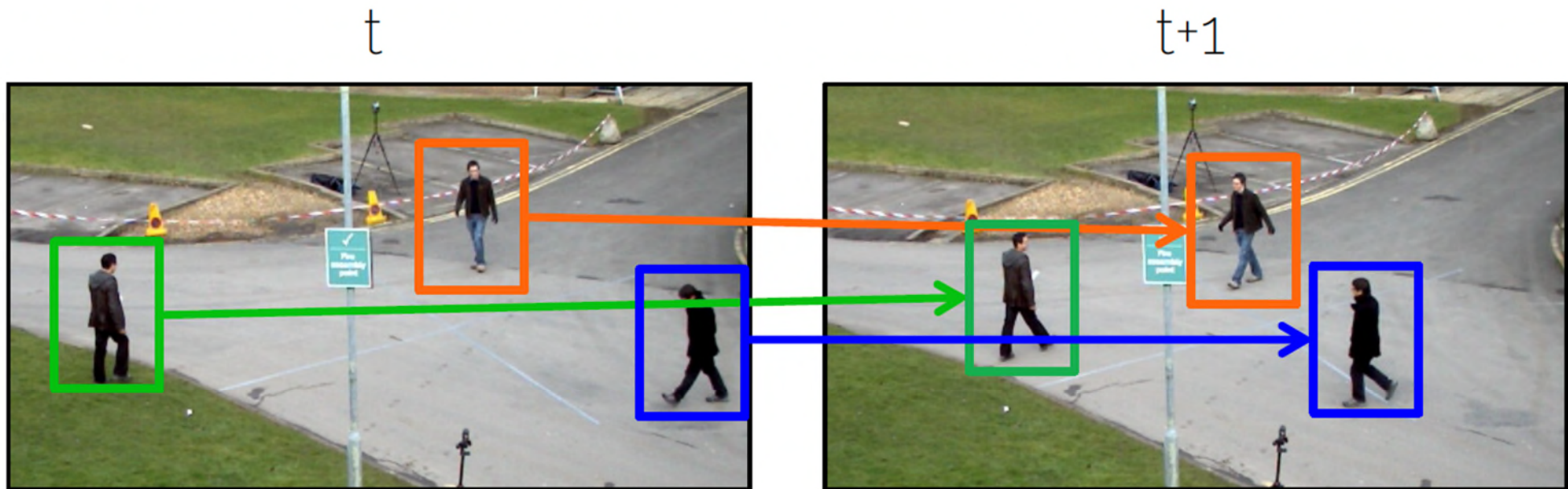
# Object Tracking

# Object Tracking Overview

- This section covers the following:
  - Introduction
  - Multiple Object Tracking (MOT)
  - Emerging / New Direction

# Objective

- Track the same persons/objects in continuous frames of a video.
- Objects are represented by the bounding boxes with IDs. Typically, the detection results are the input to the tracking algorithm.



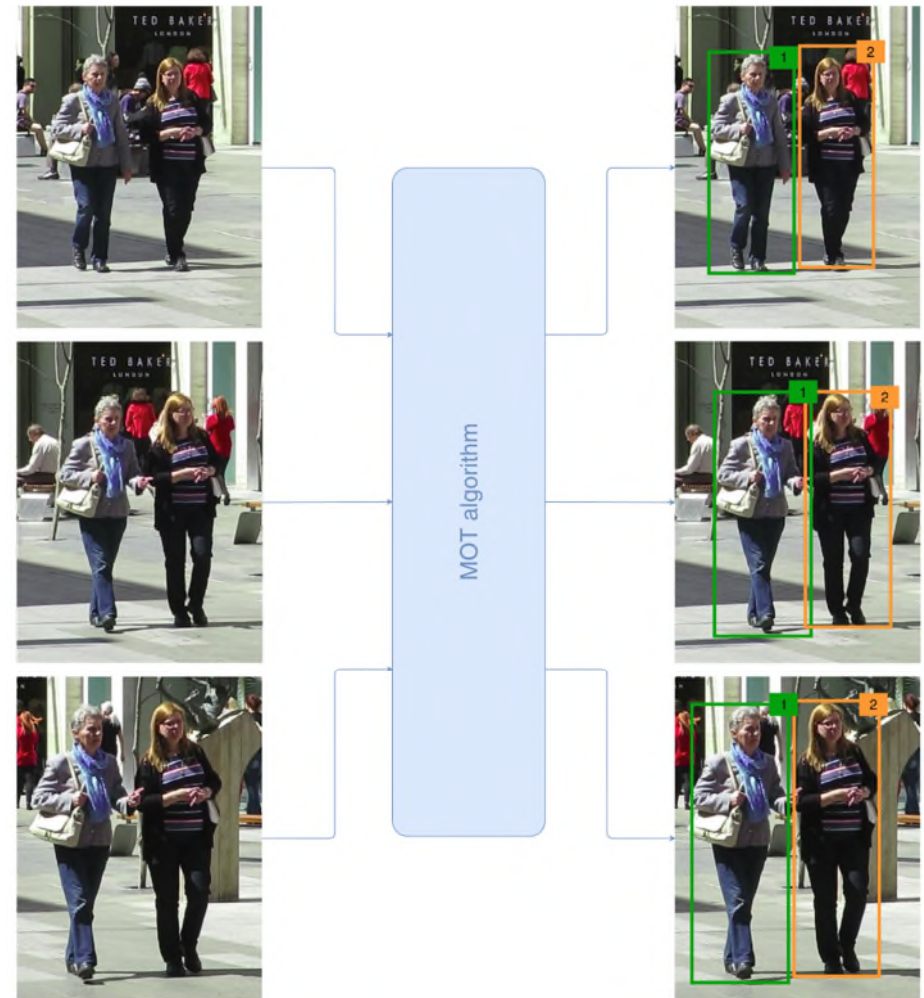
# Tracking Issues

- Two important factors:
  - Appearance features
    - How does the object look like?
  - Motion features
    - What is the movement of objects?
    - Where will they be located in the next frame?

# Multiple Object Tracking (MOT)

# Multiple Object Tracking (MOT)

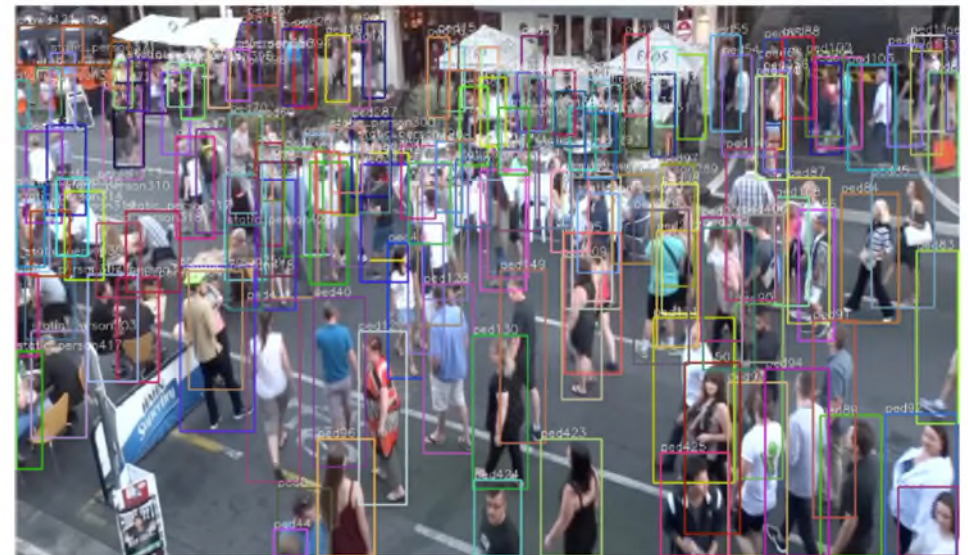
- Multiple object tracking (MOT) tracks all objects (e.g., persons) simultaneously.





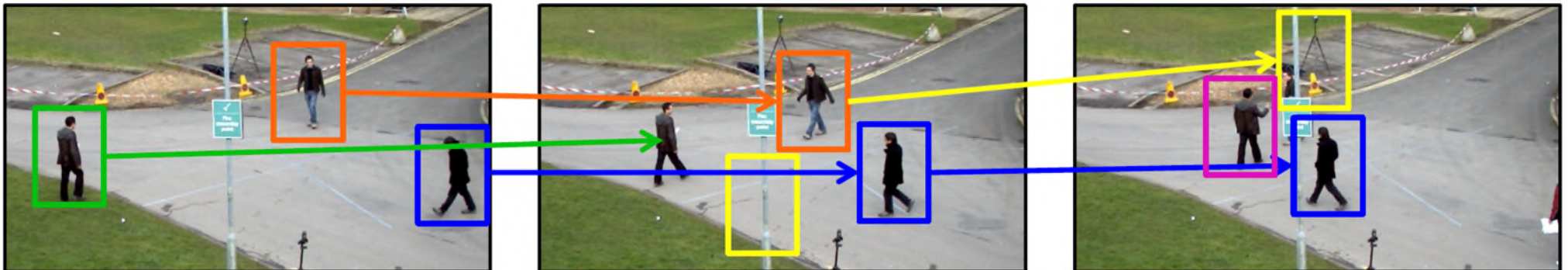
# Challenges of MOT

- Multiple objects with similar appearance, like pedestrians.
- Other uninterested objects (e.g., background, fire hydrants).
- Heavy occlusion.



# Tracking-by-Detection

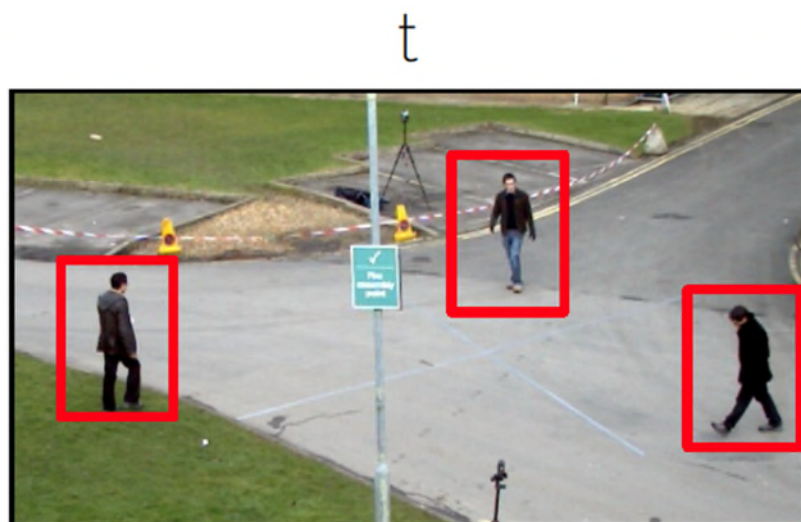
- Goal:
  - Detect objects in individual video frames.
  - Associate sets of detections between frames, thereby creating individual object tracks over time.
- Problem:
  - Detections are not always accurate, like false positive, missing detections.



# Tracking-by-Detection (1)



1. Detection: Use a detector to initialize tracks at frame  $t$ .

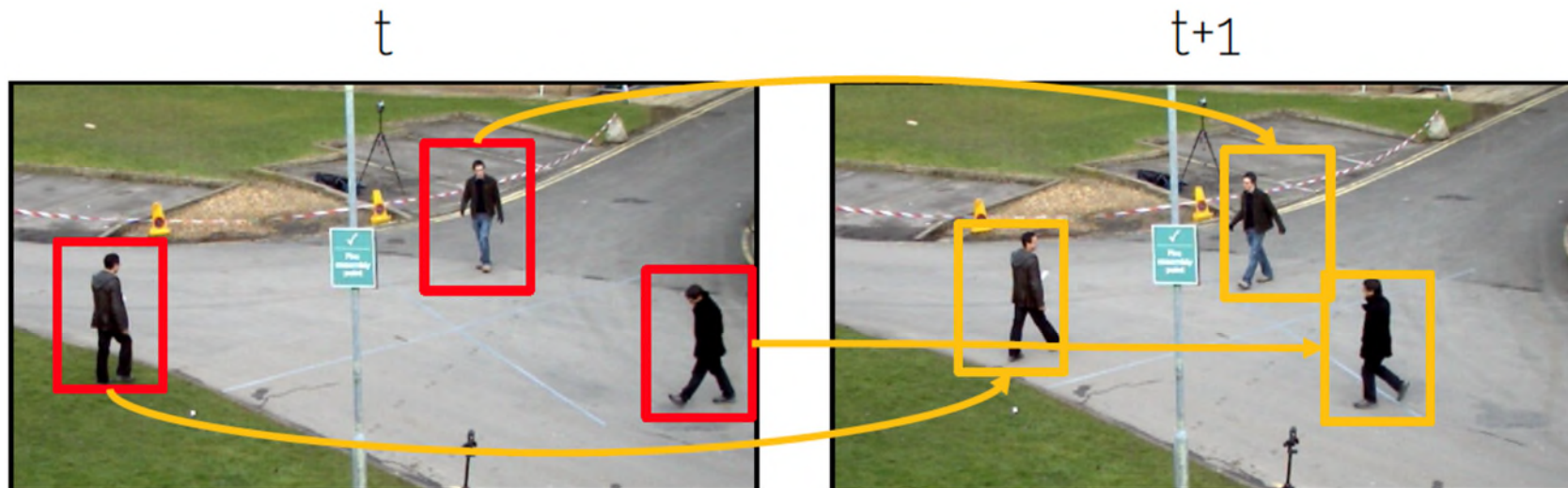




# Tracking-by-Detection (2)



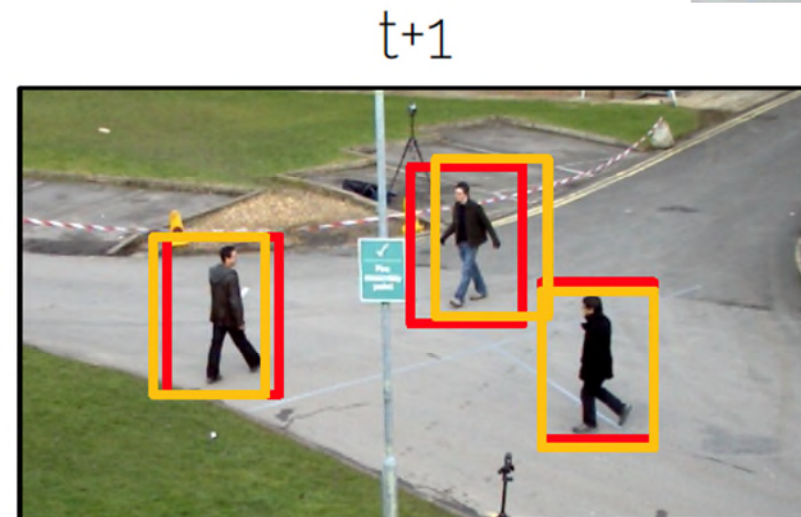
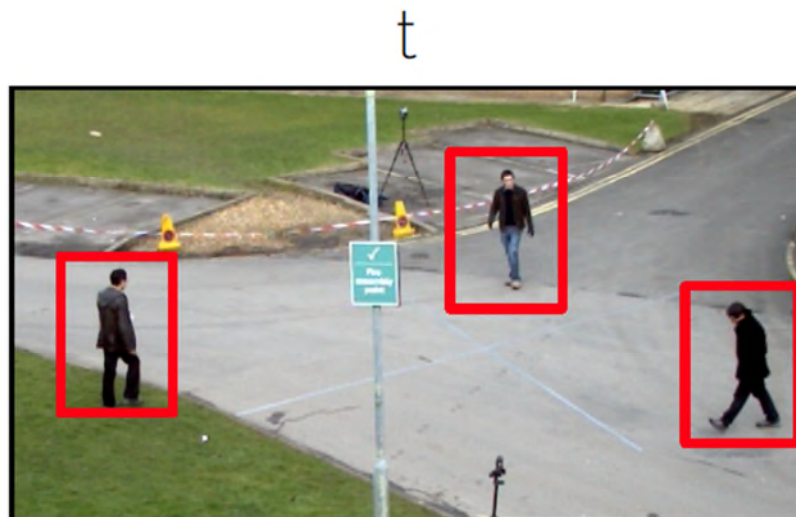
1. Detection: Use a detector to initialize tracks at frame  $t$ .
2. Motion prediction: predict the next positions of objects using motion model.



# Tracking-by-Detection (3)



1. Detection: Use a detector to initialize tracks at frame  $t$ .
2. Motion prediction: predict the next positions of objects using motion model.
3. Data association: Match predicted positions with detections at frame  $t+1$ .



# Motion Model

- Predictions are based on the movement of objects.
- Assumption: e.g., objects moves at a constant speed.
- Motion modelling
  - Kalman filtering
  - Particle filtering
  - Etc.



# Data Association

- Simple matching of detections with the predicted boxes by Kalman Filtering.
  - Calculate the **distances** between boxes (e.g., based on IOU, pixel-wise)
  - **Hungarian algorithm** solves the one-to-one linear assignment problem.
- Distance based on Intersection over Union (IoU)
  - Distance between each detection with all predicted bounding boxes using Kalman filtering.

Predictions

|                                                                                                   |                                                                                     |                                                                                     |                                                                                     |                                                                                     |
|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|                                                                                                   |  |  |  |  |
| <br>Detections | 0.9                                                                                 | 0.8                                                                                 | 0.8                                                                                 | 0.1                                                                                 |
|                | 0.5                                                                                 | 0.4                                                                                 | 0.3                                                                                 | 0.8                                                                                 |
|                | 0.2                                                                                 | 0.1                                                                                 | 0.4                                                                                 | 0.8                                                                                 |
|                | 0.1                                                                                 | 0.2                                                                                 | 0.5                                                                                 | 0.9                                                                                 |

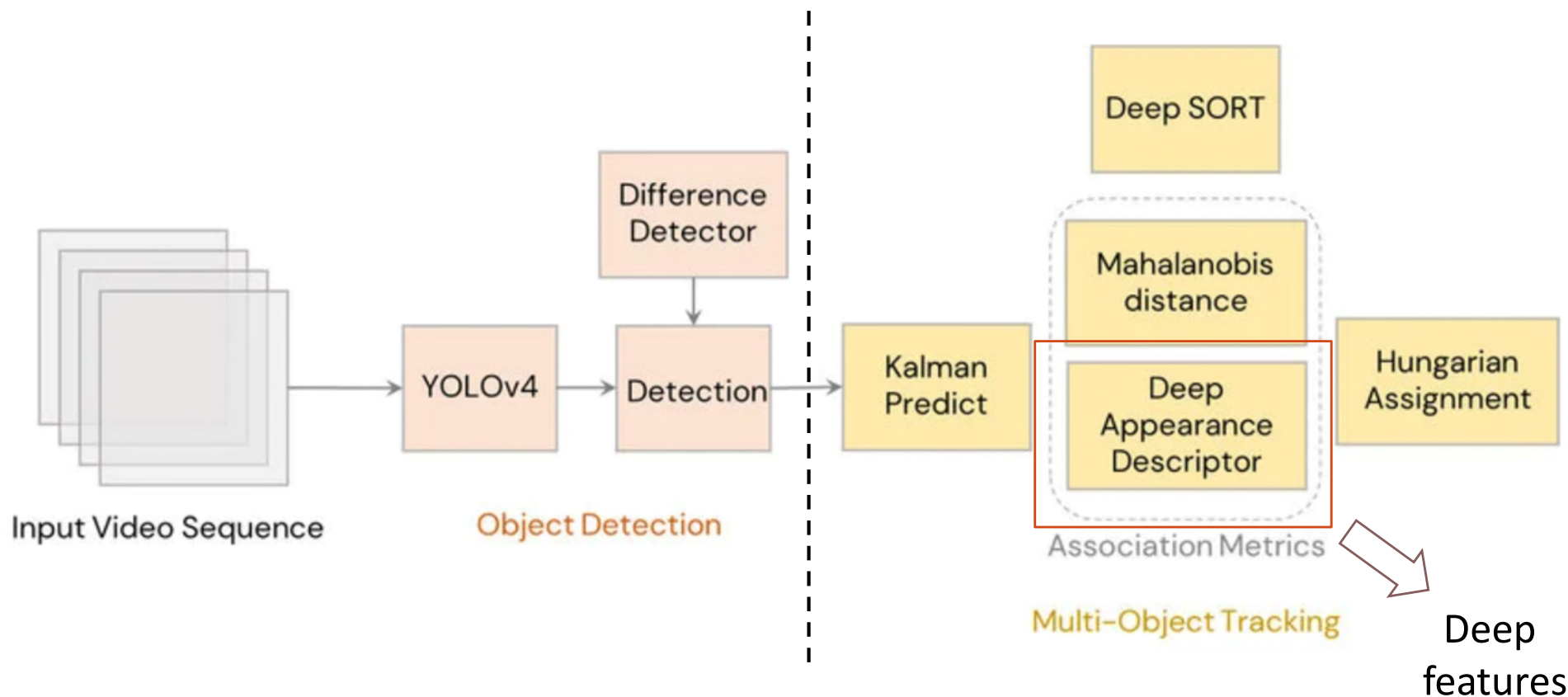


$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



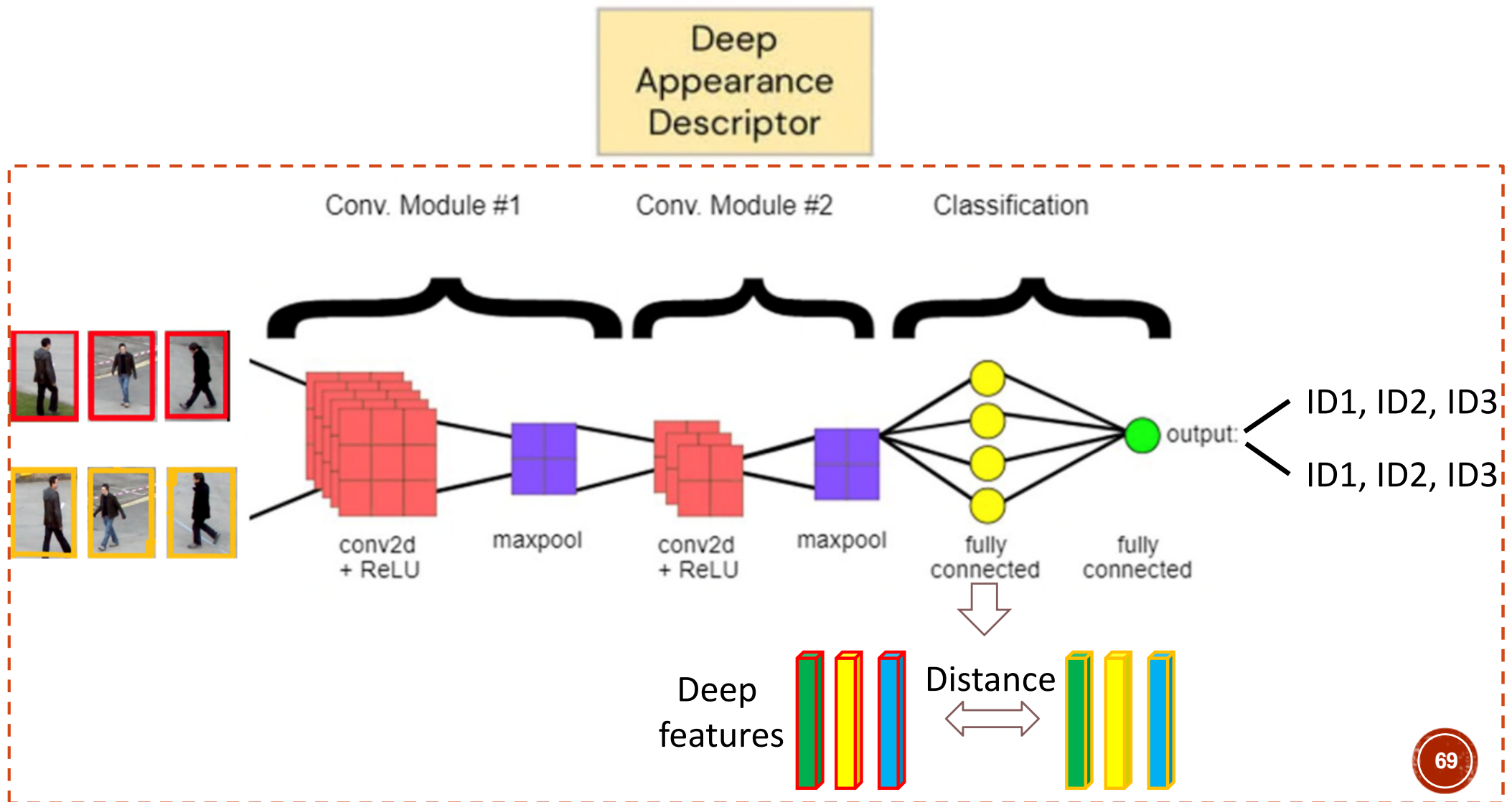
# Data Association

- Appearance model: Deep SORT (Simple Online and Real-time Tracking).
- Add deep appearance features to help data association between detections by object detector (e.g., YOLOv4) and predictions by Kalman filter.



# Data Association

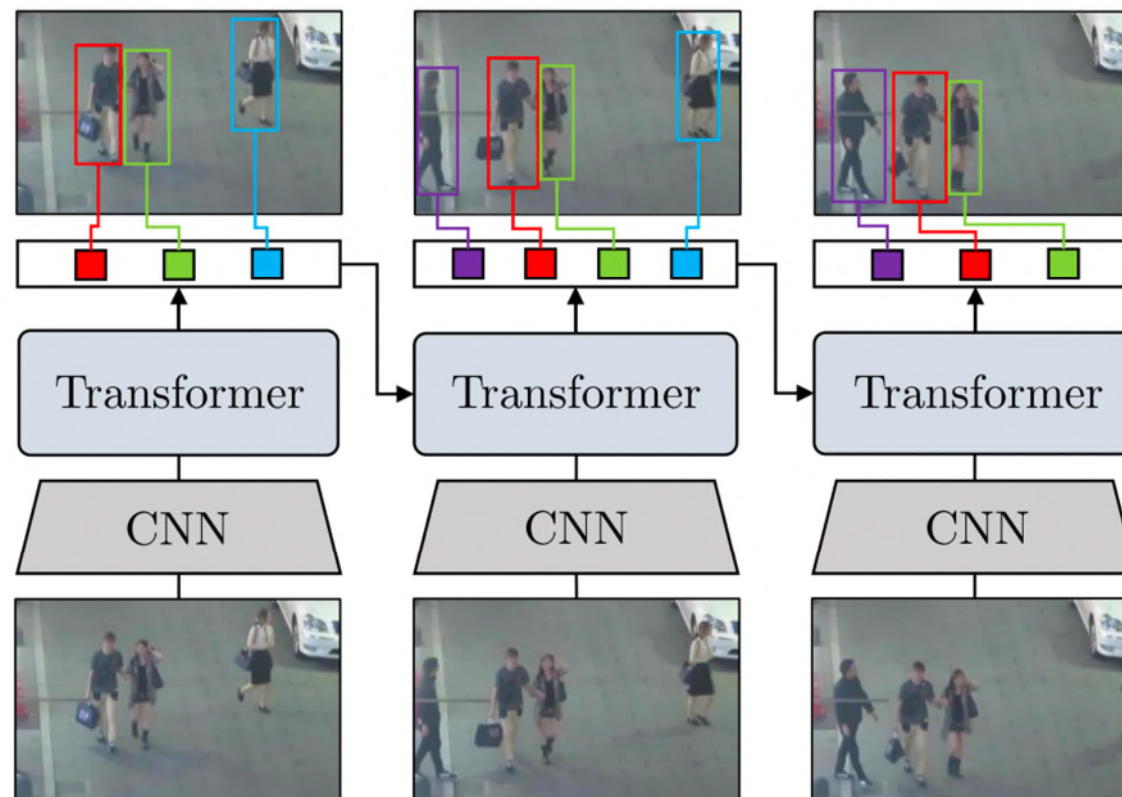
- Deep SORT: Deep appearance descriptor / feature / embedding (i.e., using CNN)



Emerging / New Methods in MOT

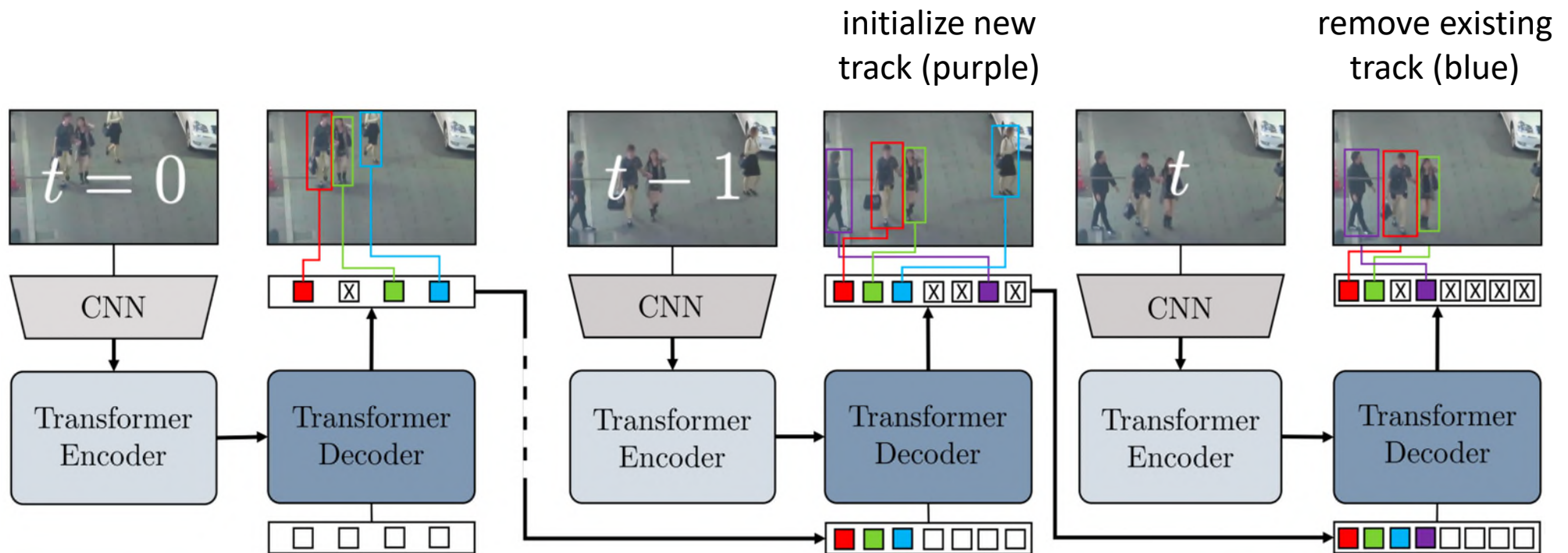
# Trackformer

- Perform joint object detection and tracking-by-attention with Transformer.
- Object and autoregressive track queries reason about track initialization, identity, and spatiotemporal trajectories.



# Trackformer Architecture

- Encoder: CNN + Transformer encoder.
- Decoder: Transformer decoder + object / track queries.
- Bounding box and class prediction: multilayer perceptron (MLP).



White boxes: object queries, Colored boxes: track queries, Crossed boxes: background classes



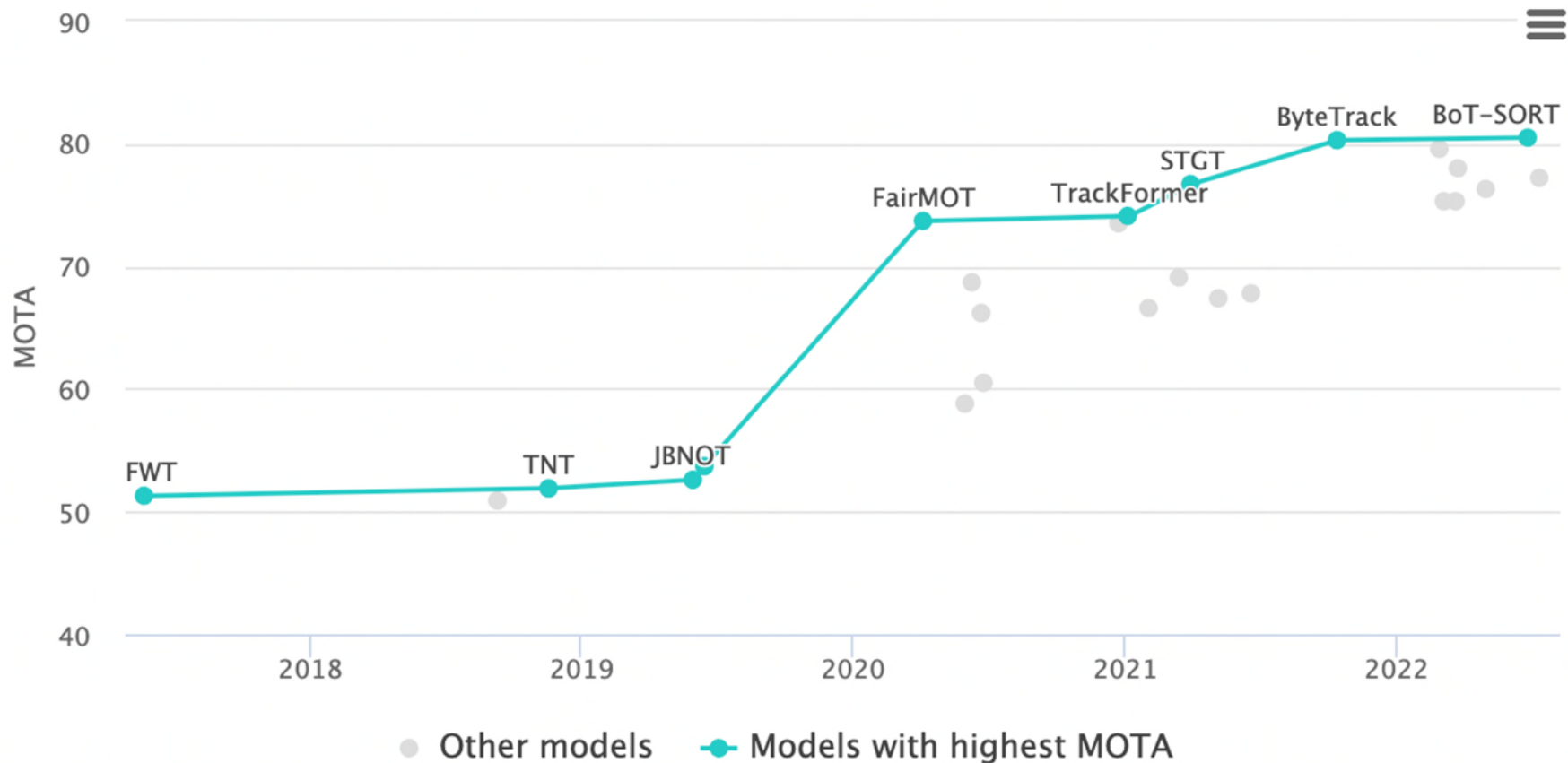
# Trackformer Queries

- Object queries:
  - Detect new objects in current frame and then form tracks for the next frame.
- Track queries:
  - Follow objects through a video by carrying their identity information while adapting to their changing position.
  - Associate tracked objects with the bounding box of the current frame.

# Demo: Trackformer



# Performance Comparison on MOT16



- Comparison of state-of-the-art methods by MOTA on the MOT16 dataset. (Date: Aug 2022)

MOTA (Multiple Object Tracking Accuracy)

$$MOTA = 1 - \frac{(FN + FP + IDSW)}{GT} \in (-\infty, 1]$$

GT is the number of ground truth boxes

# Object Tracking Summary

- This section covers the following:
  - Introduction
  - Multiple Object Tracking (MOT)
  - Emerging / New Direction

# Exercise: Multiple Object Tracking

(c) List the key steps in the tracking-by-detection multiple-object tracking in video.

(6 Marks)

# Solution



# Pose Estimation

# Pose Estimation Overview

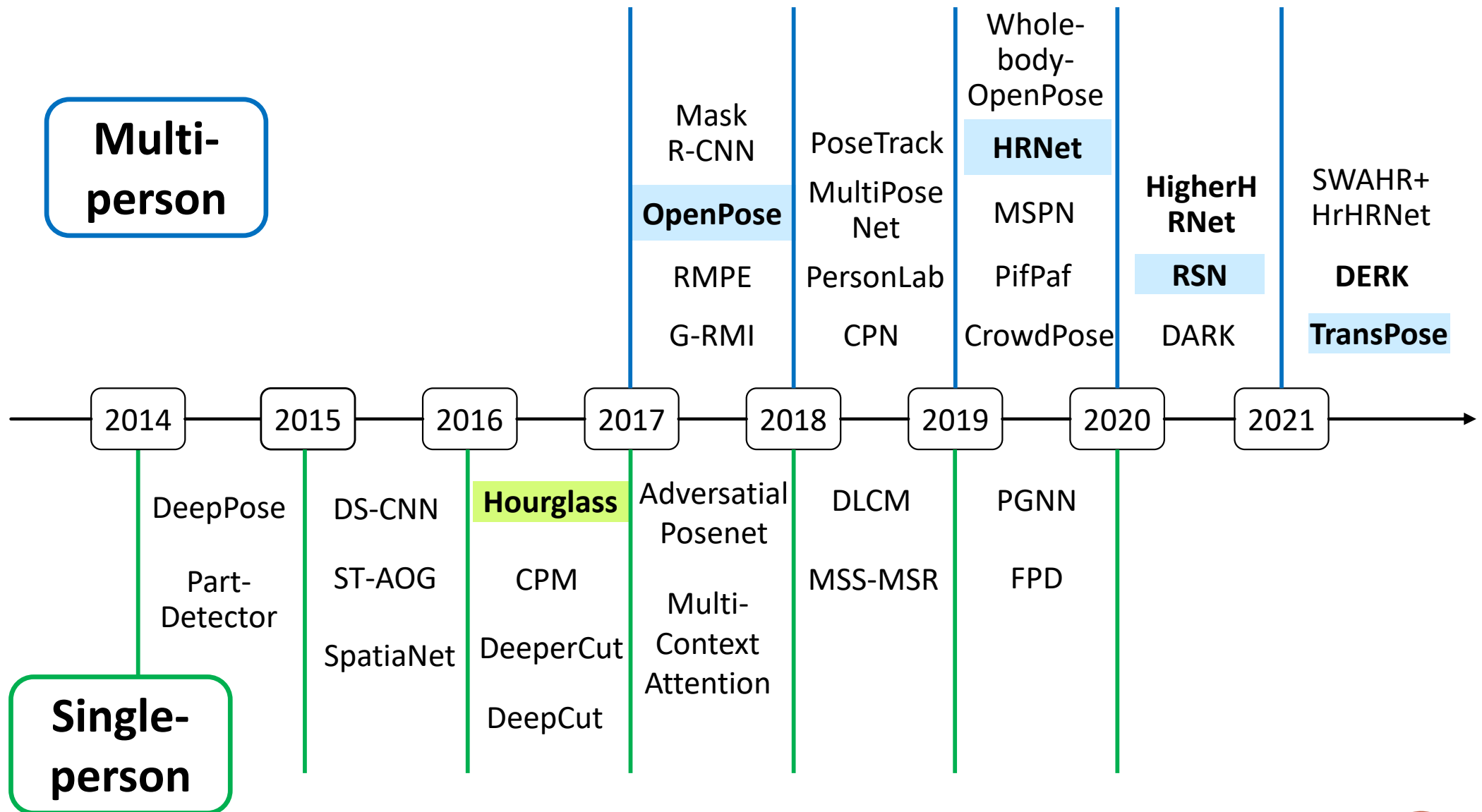
- Introduction
- Single-Person Pose Estimation
- Multi-Person Pose Estimation
- Emerging / New Methods

# Objective

- To locate human body / facial keypoints from images or videos

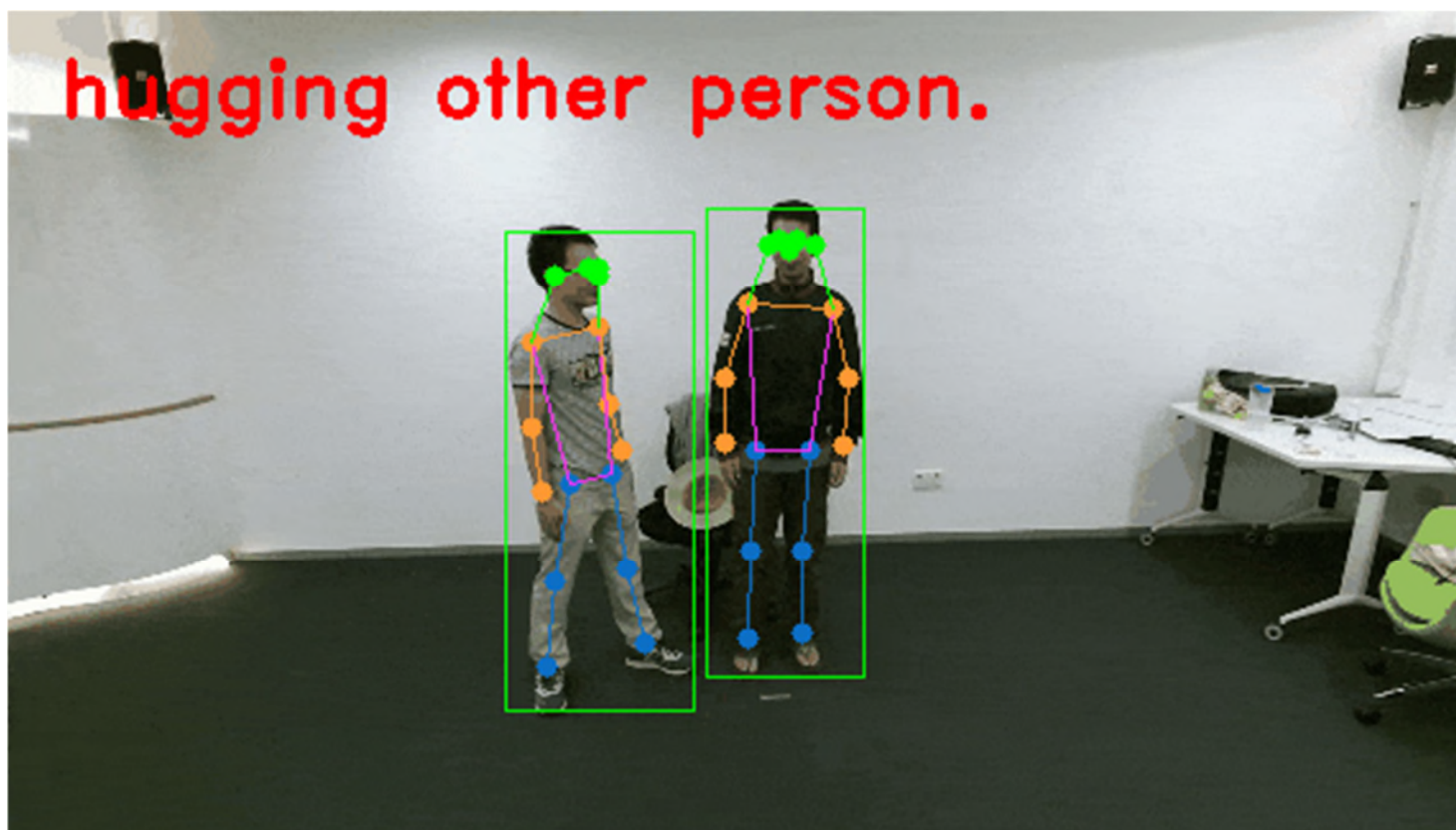


# Brief History & Milestones



# Applications

- Action Recognition: use body keypoints to classify human actions



# Applications

- Animation and Motion Capture: build virtual character based on human pose.





# Dataset: Single-Person Pose Estimation

| Dataset Name                                    | Year | Single-Person | Multi-Person | Upper Poses | Full Poses | Various Poses | Number of Joints | Evaluation Metric | Number of Images / Videos |     |        |
|-------------------------------------------------|------|---------------|--------------|-------------|------------|---------------|------------------|-------------------|---------------------------|-----|--------|
|                                                 |      |               |              |             |            |               |                  |                   | Train                     | Val | Test   |
| Image-Based Datasets for Human Pose Estimation. |      |               |              |             |            |               |                  |                   |                           |     |        |
| LSP [71]                                        | 2010 | ✓             |              |             | ✓          |               | 14               | PCP               | 1,000                     | -   | 1,000  |
| LSP-Extended [72]                               | 2011 | ✓             |              |             | ✓          |               | 14               | PCP               | 10,000                    | -   | -      |
| Flic [142]                                      | 2013 | ✓             |              | ✓           |            |               | 10               | PCP               | 5,000                     | -   | 1,016  |
| Flic-Full [142]                                 | 2013 | ✓             |              | ✓           |            |               | 10               | PCP               | 20,928                    | -   | -      |
| Flic-Plus [160]                                 | 2013 | ✓             |              | ✓           |            |               | 10               | PCP               | 17,380                    | -   | -      |
| MPII [1]                                        | 2014 | ✓             |              |             |            | ✓             | 16               | PCPm/PCKh         | 28,821                    | -   | 11,701 |
| Video-Based Datasets for Human Pose Estimation. |      |               |              |             |            |               |                  |                   |                           |     |        |
| Penn Action [195]                               | 2013 | ✓             |              |             | ✓          |               | 13               | mAP               | 1,000                     | -   | 1,000  |
| JHMDB [65]                                      | 2013 | ✓             |              |             | ✓          |               | 15               | mAP               | 600                       | -   | 300    |

# Dataset: Multi-Person Pose Estimation

| Dataset Name                                    | Year | Single-Person | Multi-Person | Upper Poses | Full Poses | Various Poses | Number of Joints | Evaluation Metric | Number of Images / Videos |        |        |
|-------------------------------------------------|------|---------------|--------------|-------------|------------|---------------|------------------|-------------------|---------------------------|--------|--------|
|                                                 |      |               |              |             |            |               |                  |                   | Train                     | Val    | Test   |
| Image-Based Datasets for Human Pose Estimation. |      |               |              |             |            |               |                  |                   |                           |        |        |
| MPII [1]                                        | 2014 |               | ✓            |             |            | ✓             | 16               | PCKh              | 3,800                     | -      | 1,700  |
| COCO [93]                                       | 2017 |               | ✓            |             |            | ✓             | 17               | AP                | 57,000                    | 5,000  | 20,000 |
| AIC-HKD [179]                                   | 2017 |               | ✓            |             |            | ✓             | 14               | mAP               | 210,000                   | 30,000 | 60,000 |
| CrowdedPose [84]                                | 2019 |               | ✓            |             |            | ✓             | 14               | mAP               | 10,000                    | 2,000  | 8,000  |
| Video-Based Datasets for Human Pose Estimation. |      |               |              |             |            |               |                  |                   |                           |        |        |
| PoseTrack2017 [63]                              | 2017 |               | ✓            |             |            | ✓             | 15               | mAP               | 250                       | 50     | 214    |
| PoseTrack2018 [2]                               | 2018 |               | ✓            |             |            | ✓             | 15               | mAP               | 593                       | 170    | 375    |
| HiEve [94]                                      | 2020 |               | ✓            |             |            | ✓             | 14               | mAP               | 19                        | -      | 13     |

# Sample Images in MS-COCO Dataset



# Performance Metrics

- PCK (Percentage of Correct Keypoint)
  - Measure the percentage of predicted keypoint that falls within a normalized distance of true joint, e.g.,
  - In Flic dataset, PCK@0.2: Distance between predicted and true joint  $< 0.2 \times$  torso diameter.
  - In MPII dataset, PCKh@0.5 refers to the threshold = 50% of the head segment line.
- PCP (Percentage of Correct Parts)
  - Measure the localization accuracy of limbs.
  - If the distance between the two predicted joints and the true limb is less than a fraction of the limb length (e.g., 0.1-0.5), then the limb is considered detected correctly.

# Performance Metrics

- AP and AR based on OKS (Object Keypoint Similarity)
  - Average Precision (AP), Average Recall (AR) and their variants are used in COCO, MPII and PoseTrack.
  - Object Keypoint Similarity (OKS) plays the same role as the IoU in detection.
  - OKS definition: the distance between predicted points and ground truth joints normalized by the scale of the person.

Metric of COCO dataset with OKS:

## Average Precision (AP):

|                |                                                    |
|----------------|----------------------------------------------------|
| AP             | % AP at OKS=.50:.05:.95 (primary challenge metric) |
| $AP^{OKS=.50}$ | % AP at OKS=.50 (loose metric)                     |
| $AP^{OKS=.75}$ | % AP at OKS=.75 (strict metric)                    |

## Average Recall (AR):

|                |                         |
|----------------|-------------------------|
| AR             | % AR at OKS=.50:.05:.95 |
| $AR^{OKS=.50}$ | % AR at OKS=.50         |
| $AR^{OKS=.75}$ | % AR at OKS=.75         |

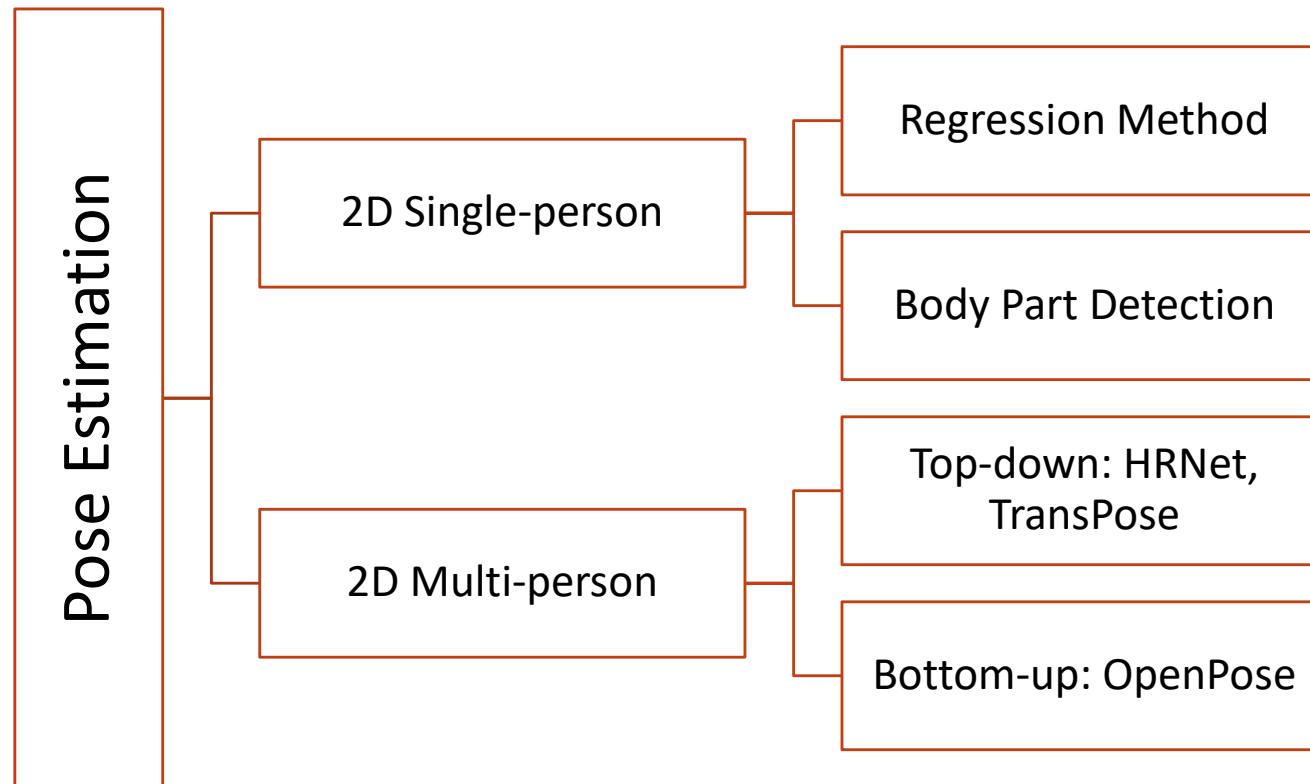


# Challenges

- Computational efficiency
- Limited data for rare poses
- Occlusion



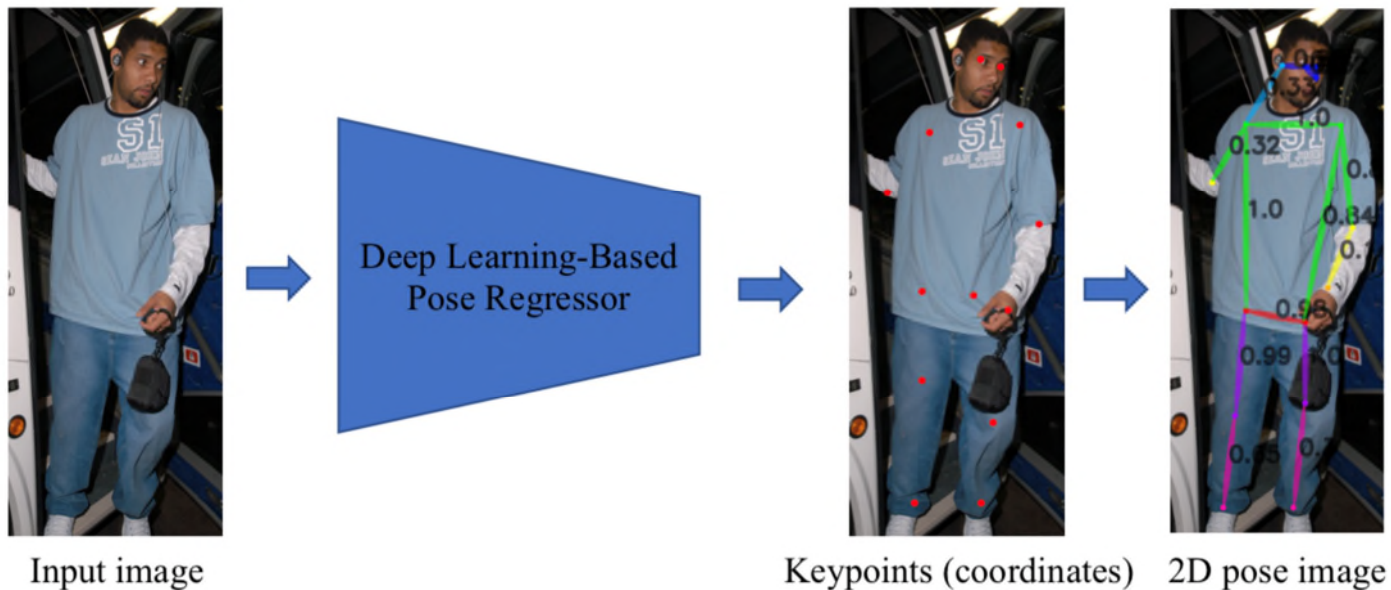
# Pose Estimation Methods



# Single-Person Pose Estimation

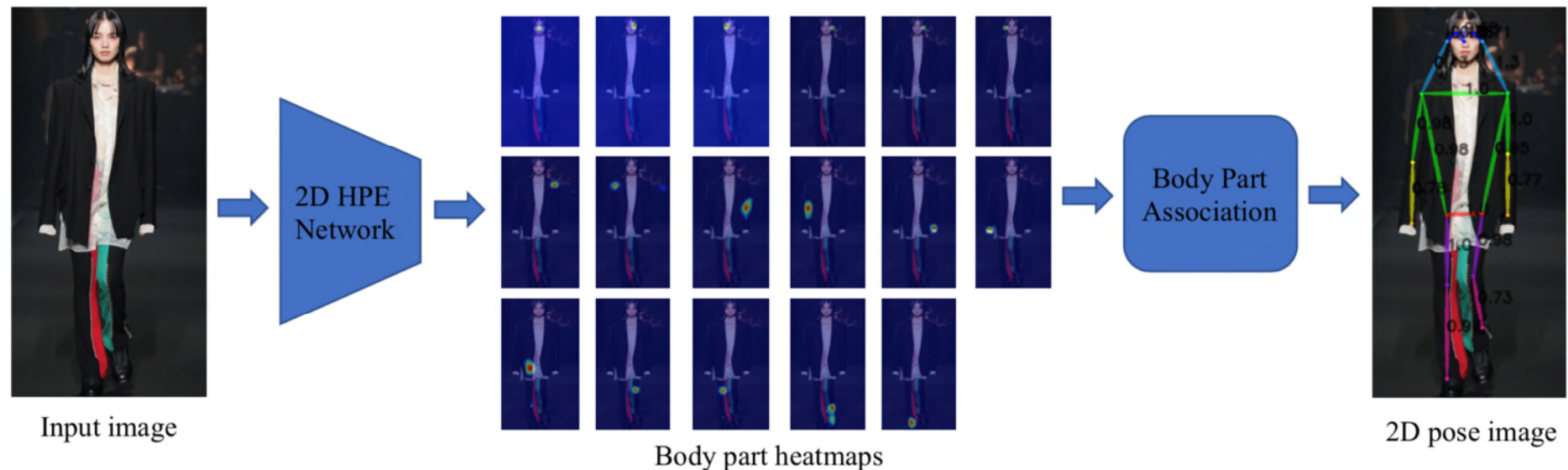
# Regression Method

- Approach:
  - Obtain feature map from CNN
  - Calculate the regressor from the feature map
- Pros: easy and fast
- Cons: less accurate



# Body Part Detection

- Approach:
  - Estimate K heatmaps for a total of K keypoints
  - Assemble these detected keypoints into whole body pose or skeleton.
- Pros: accuracy
- Cons: slower and complex
- Representative work: Stacked Hourglass Networks for Human Pose Estimation (2016 ECCV)



# Result Comparison on MPII dataset

Results based on AP Using OKS

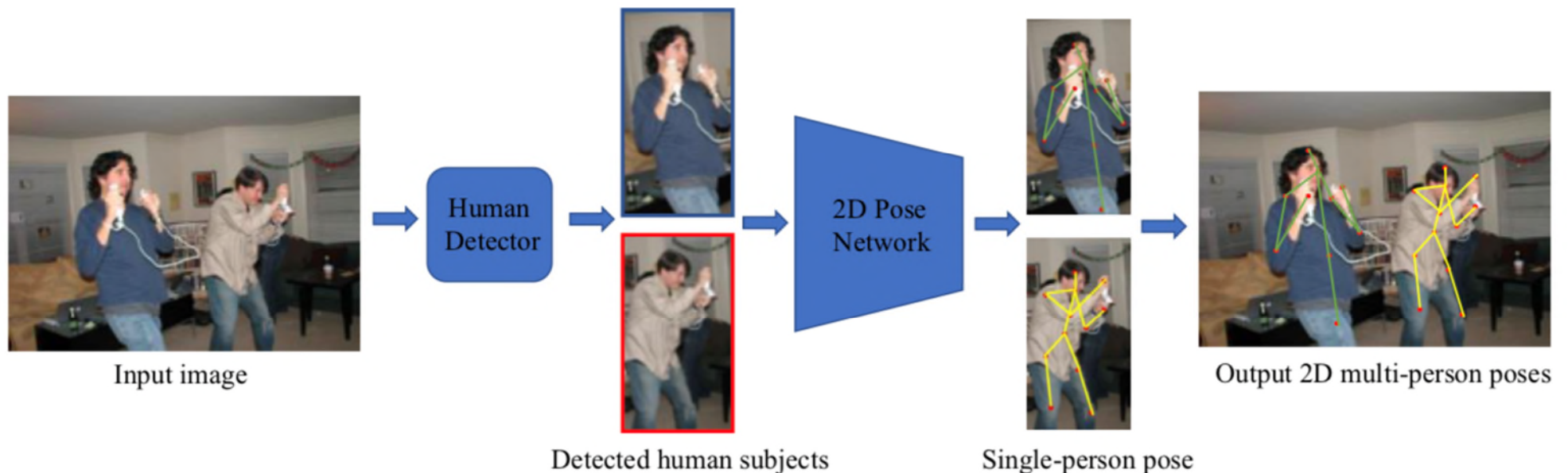
|                     | Year | Model                         | Head | Shoulder | Elbow | Wrist | Hip  | Knee | Ankle | Total |
|---------------------|------|-------------------------------|------|----------|-------|-------|------|------|-------|-------|
| Body Part Detection | 2016 | Deepercut                     | 96.8 | 95.2     | 89.3  | 84.4  | 88.4 | 83.4 | 78.0  | 88.5  |
|                     | 2016 | CPM                           | 97.8 | 95.0     | 88.7  | 84.0  | 88.4 | 82.8 | 79.4  | 88.5  |
|                     | 2016 | Stacked hourglass             | 98.2 | 96.3     | 91.2  | 87.1  | 90.1 | 87.4 | 83.6  | 90.9  |
|                     | 2017 | Multi-Context Attention       | 98.5 | 96.3     | 91.9  | 88.1  | 90.6 | 88.0 | 85.0  | 91.5  |
|                     | 2017 | Adversarial Posenet           | 98.1 | 96.5     | 92.5  | 88.5  | 90.2 | 89.6 | 86.0  | 91.9  |
|                     | 2017 | PRMs                          | 98.5 | 96.7     | 92.5  | 88.7  | 91.1 | 88.6 | 86.0  | 92.0  |
|                     | 2018 | MSS-MSR                       | 98.5 | 96.8     | 92.7  | 88.4  | 90.6 | 89.3 | 86.3  | 92.1  |
|                     | 2018 | DLCM                          | 98.4 | 96.9     | 92.6  | 88.7  | 91.8 | 89.4 | 86.2  | 91.5  |
|                     | 2019 | PGNN                          | 98.6 | 97.0     | 92.8  | 88.8  | 91.7 | 89.9 | 86.6  | 92.5  |
|                     | 2019 | MSPN                          | 98.4 | 97.1     | 93.2  | 89.2  | 92.0 | 90.1 | 85.5  | 92.6  |
|                     | 2020 | Unipose                       | -    | -        | -     | -     | -    | -    | -     | 92.7  |
| Regression          | 2016 | IEF                           | 95.7 | 91.7     | 81.7  | 72.4  | 82.8 | 73.2 | 66.4  | 81.3  |
|                     | 2017 | Compositional pose regression | 97.5 | 94.3     | 87.0  | 81.2  | 86.5 | 78.5 | 75.4  | 86.4  |
|                     | 2019 | FPD                           | 98.3 | 96.4     | 91.5  | 87.4  | 90.0 | 87.1 | 83.7  | 91.1  |

# Multi-Person Pose Estimation



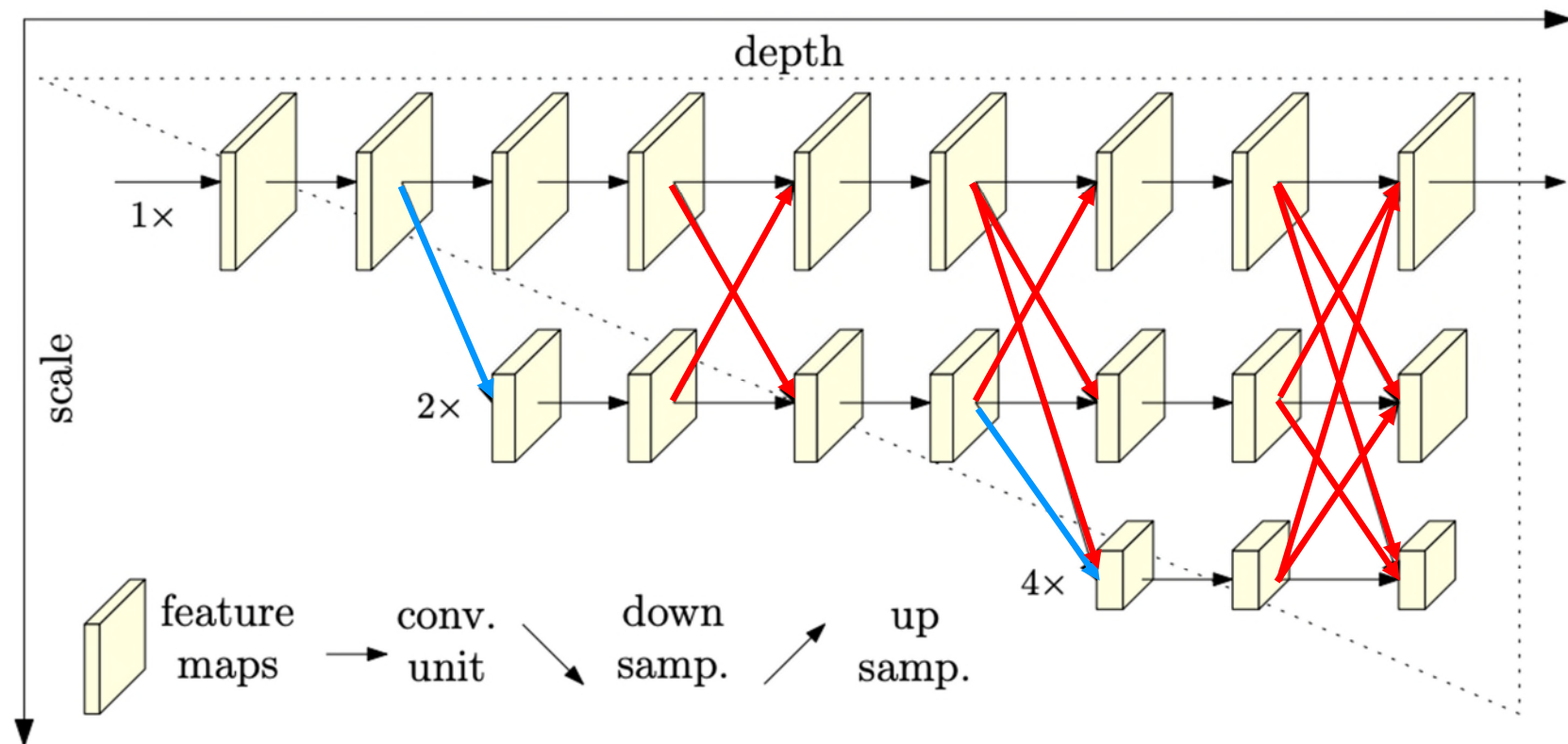
# Top-Down Methods

- Approach:
  - Detect persons in the image with off-the-shelf detector
  - Estimate the pose for every single person
- Representative work: HRNet (High-Resolution Net)
- Pros: less network complexity, accurate
- Cons: runtime increases with the increase of the number of detected persons.
- Suitable for the scenes with few people.



# HRNet (High-Resolution Net)

- Start from a high-resolution subnet, add high-to-low resolution subnet gradually.
- Connect multi-resolution subnets in parallel.
- Multi-scale fusions: exchanging information across parallel multi-resolution subnets.



# Bottom-up Methods

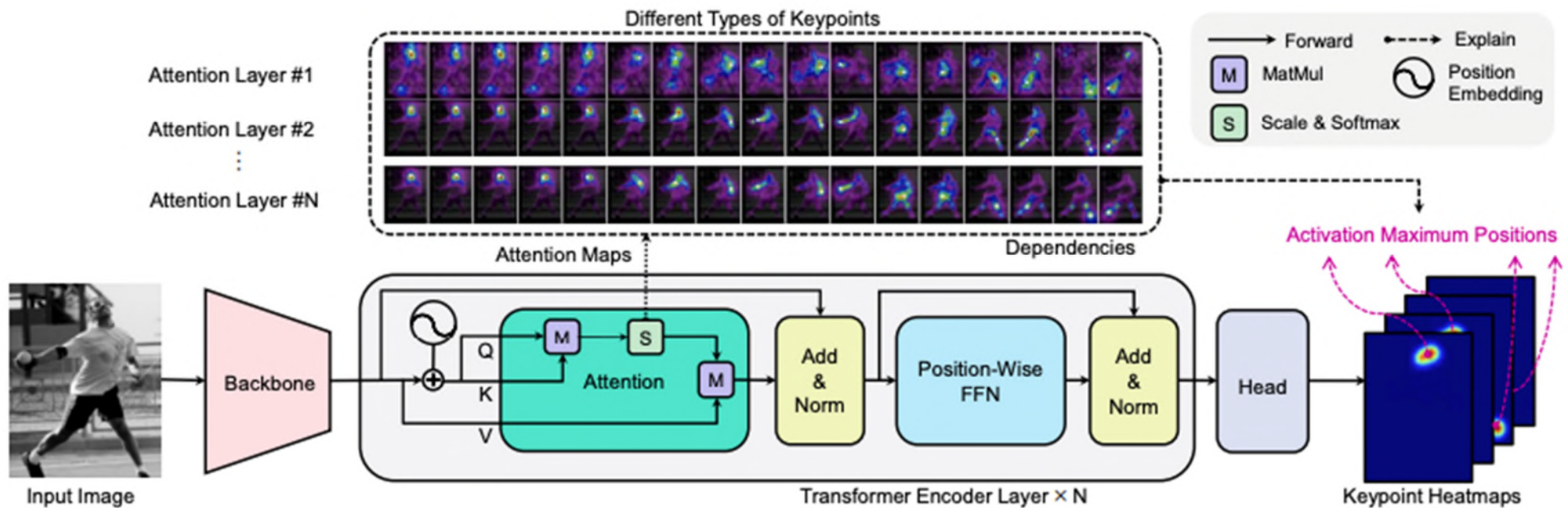
- Approach:
  - Detect all body parts in the image
  - Assemble the body parts into poses for different persons
- Representative work: OpenPose
- Pros: runtime is mostly constant with respect to different number of detected persons.
- Cons: complex
- Suitable for crowded scene



Emerging / New Methods

# TransPose

- A transformer-based pose estimation method
- Backbone: common CNNs to extract features, e.g., ResNet and HRNet, but only keep initial several layers
- Transformer: only use encoder, with N attention layers and feed-forward networks (FFN)
- Head: use convolution layers to predict keypoint heatmaps



# Comparison on MS-COCO for Multi-Person Pose Estimation

|           | Year | Model          | Backbone         | Input size | AP   | AP.5 | AP.75 | AP(M) | AP(L) |
|-----------|------|----------------|------------------|------------|------|------|-------|-------|-------|
| Top-down  | 2017 | Mask-RCNN      | ResNet50         | -          | 63.1 | 87.3 | 68.7  | 57.8  | 71.4  |
|           | 2017 | G-RMI          | ResNet101        | 353x257    | 68.5 | 87.1 | 75.5  | 65.8  | 73.3  |
|           | 2018 | CPN*           | ResNet-Inception | 384x288    | 72.1 | 91.4 | 80.0  | 68.7  | 77.2  |
|           | 2018 | SimpleBaseline | ResNet152        | 384x288    | 73.7 | 91.9 | 81.1  | 70.3  | 80.0  |
|           | 2019 | HRNet-W32      | HRNet-W32        | 384x288    | 74.9 | 92.5 | 82.8  | 71.3  | 80.9  |
|           | 2019 | HRNet-W48*     | HRNet-W48        | 384x288    | 77.0 | 92.7 | 84.5  | 73.4  | 83.1  |
|           | 2020 | DARK*          | HRNet-W48        | 384x288    | 77.4 | 92.6 | 84.6  | 73.6  | 83.7  |
|           | 2020 | RSN            | 4xRSN50          | 384x288    | 78.6 | 94.3 | 86.6  | 75.5  | 83.3  |
|           | 2021 | TransPose      | HRNet-Small-W48  | 256x192    | 75.0 | 92.2 | 83.6  | 72.5  | 82.4  |
| Bottom-up | 2017 | OpenPose       | -                | -          | 61.8 | 84.9 | 67.5  | 57.1  | 68.1  |
|           | 2017 | AE             | Hourglass        | 512x512    | 65.5 | 86.8 | 72.3  | 60.6  | 72.6  |
|           | 2018 | PersonLab      | ResNet152        | 1401x1401  | 68.7 | 89.0 | 75.4  | 64.1  | 75.5  |
|           | 2020 | HGG            | Hourglass        | 512x512    | 67.6 | 85.1 | 73.7  | 62.7  | 74.6  |
|           | 2020 | HrHRNet-W48+AE | HRNet-W48        | 640x640    | 70.5 | 89.3 | 77.2  | 66.6  | 75.8  |
|           | 2021 | DERK-W48       | HRNet-W48        | 640x640    | 71.0 | 89.2 | 78.0  | 67.1  | 76.9  |
|           | 2021 | SWAHR+HrHRNet  | HRNet-W48        | -          | 72.0 | 90.7 | 78.8  | 67.8  | 77.7  |

\* using extra data



# 3D Pose Estimation

# Limitations of 2D Pose Estimation

- Lack of depth information
  - 2D pose estimation provides only (x, y) coordinate information.
  - Depth ambiguity leads to poor understanding of real-world spatial arrangements.
  - No 3D scene understanding.
- Occlusion ambiguity
  - Without depth information, fundamentally challenging to distinguish body parts when people overlap or occlude each other.
- Inadequate for real-world applications
  - 2D outputs are not sufficient for applications such as Robotics, AR/VR, motion capture.

# Challenges in 3D Pose Estimation

- Depth Ambiguity
  - Hard to infer depth from monocular 2D views.
- Occlusion
  - Body parts may be occluded by other limbs, objects / people, etc.
- Variability in Appearance
  - Differences in clothing, body shapes, lighting, and camera viewpoints.
- Complex Poses & Interactions
  - Complex body articulations, multi-person interactions that are difficult to model.

# Challenges in 3D Pose Estimation

- Data Limitations
  - Annotated 3D datasets are scarce and expensive.
- Generalization Across Domains
  - Models trained on controlled datasets often perform poorly in in-the-wild conditions.
- Computational Complexity
  - High-dimensional output (3D coordinates for many joints) requires balancing accuracy with efficiency for real-time applications.
- Multi-view vs. Monocular Trade-off
  - Multi-view setups provide more reliable depth cues but are impractical for many real-world scenarios. Monocular setups are practical but much more ambiguous.

# Main 3D Human Pose Estimation Methods

- Direct 3D Pose Estimation (End-to-End)
  - The model directly predicts 3D joint coordinates from input images or videos.
  - Pros: Simple pipeline, no intermediate step.
  - Cons: Harder to train due to depth ambiguity and limited 3D data.
  - Examples: End-to-end CNNs, Transformers for 3D joint regression.
- Two-Stage (2D-to-3D) Methods
  - Stage 1: Estimate 2D keypoints (joints) in the image.
  - Stage 2: Lift the 2D keypoints into 3D space using regression models.
  - Pros: Leverages large 2D pose datasets; modular.
  - Cons: Errors in 2D estimation propagate to 3D.
  - Examples: 2D pose detectors (e.g., OpenPose) + 3D lifting networks (e.g., MotionBERT).

# MotionBERT: A Unified Perspective on Learning Human Motion Representations



# MotionBERT: Overview

- A leading method for 3D human pose estimation.
- Two stages:
  - Stage 1: pretraining a motion encoder to accomplish 2D-3D lifting.
  - Stage 2: finetuning of pretrained motion encoder for different downstream tasks.

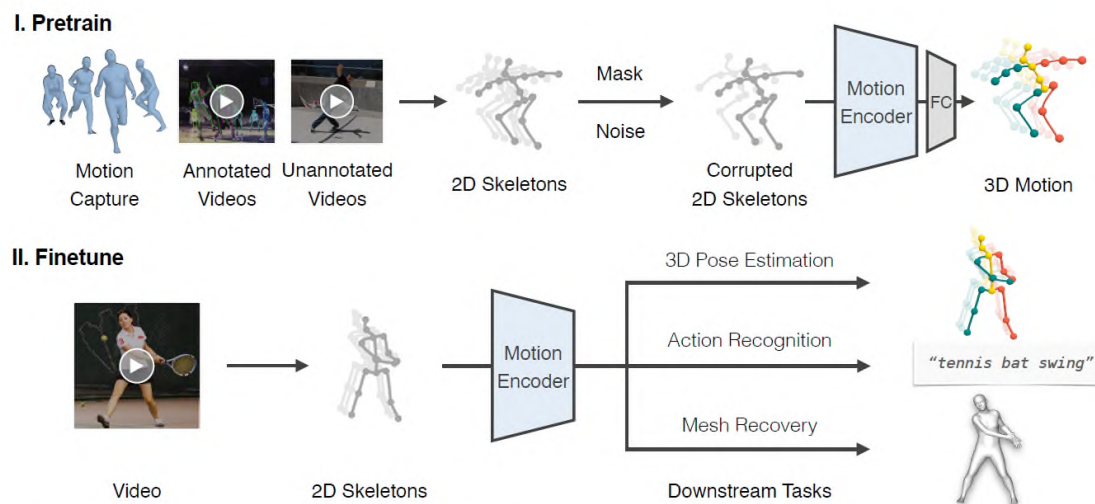


Figure 1. **Framework overview.** We utilize a motion encoder to learn human motion representations via recovering 3D human motion from corrupted 2D skeleton sequences. To adapt to different downstream tasks, we finetune the pretrained motion representations with a linear layer or a simple MLP.

# Architecture

- Dual-stream Spatio-temporal Transformer (DSTformer) consisting:
  - Spatial MHSA Block
    - model the relationship among the joints within the same time step.
  - Temporal MHSA Block
    - model the relationship across the time steps for a body joint.
- DSTformer fuses both spatial and temporal information in the flow.

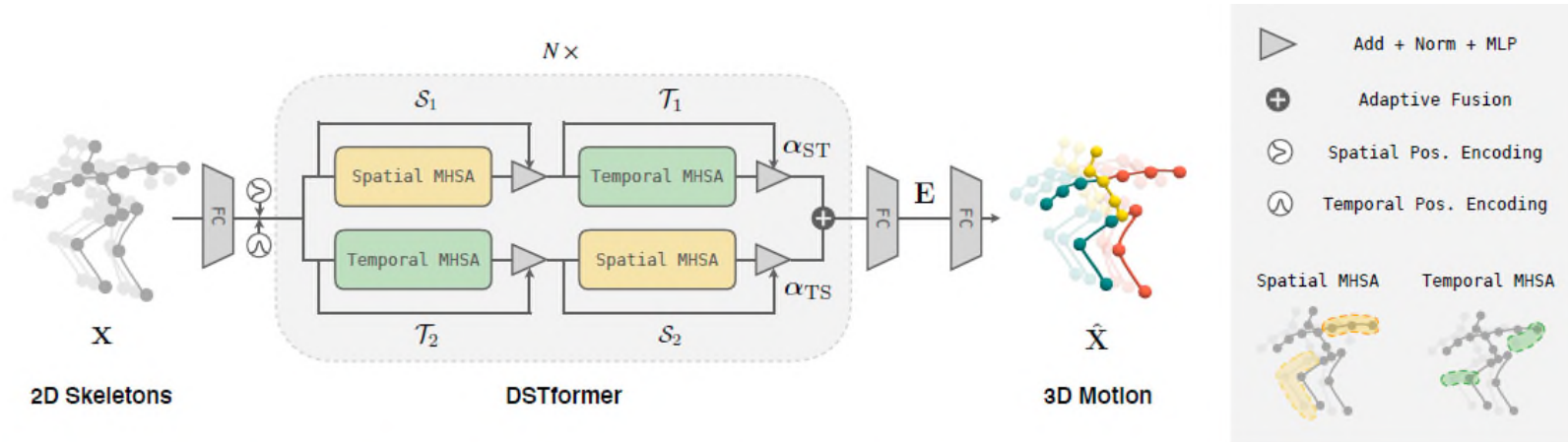


Figure 2. **Model architecture.** We propose the Dual-stream Spatio-temporal Transformer (DSTformer) as a general backbone for human motion modeling. DSTformer consists of  $N$  dual-stream-fusion modules. Each module contains two branches of spatial or temporal MHSA and MLP. The Spatial MHSA models the connection among different joints within a timestep, while the Temporal MHSA models the movement of one joint.



# Results

| Method                                 | $T$ | Dire.       | Disc.       | Eat         | Greet       | Phone       | Photo       | Pose        | Purch.      | Sit         | SitD        | Smoke       | Wait        | WalkD       | Walk        | WalkT       | Avg         |
|----------------------------------------|-----|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| Martinez <i>et al.</i> [78] ICCV'17    | 1   | 51.8        | 56.2        | 58.1        | 59.0        | 69.5        | 78.4        | 55.2        | 58.1        | 74.0        | 94.6        | 62.3        | 59.1        | 65.1        | 49.5        | 52.4        | 62.9        |
| Pavlakos <i>et al.</i> [88] CVPR'18    | 1   | 48.5        | 54.4        | 54.4        | 52.0        | 59.4        | 65.3        | 49.9        | 52.9        | 65.8        | 71.1        | 56.6        | 52.9        | 60.9        | 44.7        | 47.8        | 56.2        |
| LCN [28] ICCV'19                       | 1   | 46.8        | 52.3        | 44.7        | 50.4        | 52.9        | 68.9        | 49.6        | 46.4        | 60.2        | 78.9        | 51.2        | 50.0        | 54.8        | 40.4        | 43.3        | 52.7        |
| Xu <i>et al.</i> [121] CVPR'21         | 1   | 45.2        | 49.9        | 47.5        | 50.9        | 54.9        | 66.1        | 48.5        | 46.3        | 59.7        | 71.5        | 51.4        | 48.6        | 53.9        | 39.9        | 44.1        | 51.9        |
| VideoPose3D [89] CVPR'19               | 243 | 45.2        | 46.7        | 43.3        | 45.6        | 48.1        | 55.1        | 44.6        | 44.3        | 57.3        | 65.8        | 47.1        | 44.0        | 49.0        | 32.8        | 33.9        | 46.8        |
| Cai <i>et al.</i> [13] ICCV'19         | 7   | 44.6        | 47.4        | 45.6        | 48.8        | 50.8        | 59.0        | 47.2        | 43.9        | 57.9        | 61.9        | 49.7        | 46.6        | 51.3        | 37.1        | 39.4        | 48.8        |
| Yeh <i>et al.</i> [129] NeurIPS'19     | 243 | 44.8        | 46.1        | 43.3        | 46.4        | 49.0        | 55.2        | 44.6        | 44.0        | 58.3        | 62.7        | 47.1        | 43.9        | 48.6        | 32.7        | 33.3        | 46.7        |
| Liu <i>et al.</i> [67] CVPR'20         | 243 | 41.8        | 44.8        | 41.1        | 44.9        | 47.4        | 54.1        | 43.4        | 42.2        | 56.2        | 63.6        | 45.3        | 43.5        | 45.3        | 31.3        | 32.2        | 45.1        |
| * Cheng <i>et al.</i> [22] AAAI'20     | 128 | <u>36.2</u> | <u>38.1</u> | 42.7        | 35.9        | <b>38.2</b> | <u>45.7</u> | 36.8        | 42.0        | <b>45.9</b> | <b>51.3</b> | 41.8        | 41.5        | 43.8        | 33.1        | 28.6        | 40.1        |
| * UGCN [116] ECCV'20                   | 96  | 38.2        | 41.0        | 45.9        | 39.7        | 41.4        | 51.4        | 41.6        | 41.4        | 52.0        | 57.4        | 41.8        | 44.4        | 41.6        | 33.1        | 30.0        | 42.6        |
| † PoseFormer [135] ICCV'21             | 81  | 41.5        | 44.8        | 39.8        | 42.5        | 46.5        | 51.6        | 42.1        | 42.0        | 53.3        | 60.7        | 45.5        | 43.3        | 46.1        | 31.8        | 32.2        | 44.3        |
| * Wehrbein <i>et al.</i> [119] ICCV'21 | 200 | 38.5        | 42.5        | 39.9        | 41.7        | 46.5        | 51.6        | 39.9        | 40.8        | 49.5        | 56.8        | 45.3        | 46.4        | 46.8        | 37.8        | 40.4        | 44.3        |
| † MHFormer [56] CVPR'22                | 351 | 39.2        | 43.1        | 40.1        | 40.9        | 44.9        | 51.2        | 40.6        | 41.3        | 53.5        | 60.3        | 43.7        | 41.1        | 43.8        | 29.8        | 30.6        | 43.0        |
| *† MixSTE [134] CVPR'22                | 243 | 36.7        | 39.0        | <u>36.5</u> | 39.4        | <u>40.2</u> | <b>44.9</b> | 39.8        | 36.9        | 47.9        | 54.8        | <u>39.6</u> | 37.8        | 39.3        | 29.7        | 30.6        | 39.8        |
| † P-STMO [94] ECCV'22                  | 243 | 38.4        | 42.1        | 39.8        | 40.2        | 45.2        | 48.9        | 40.4        | 38.3        | 53.8        | 57.3        | 43.9        | 41.6        | 42.2        | 29.3        | 29.3        | 42.1        |
| † Ours (scratch)                       | 243 | 36.3        | 38.7        | 38.6        | <u>33.6</u> | 42.1        | 50.1        | <u>36.2</u> | <u>35.7</u> | 50.1        | 56.6        | 41.3        | <u>37.4</u> | <u>37.7</u> | <u>25.6</u> | <u>26.5</u> | <u>39.2</u> |
| † Ours (finetune)                      | 243 | <b>36.1</b> | <b>37.5</b> | <b>35.8</b> | <b>32.1</b> | 40.3        | 46.3        | <b>36.1</b> | <b>35.3</b> | <u>46.9</u> | <u>53.9</u> | <b>39.5</b> | <b>36.3</b> | <b>35.8</b> | <b>25.1</b> | <b>25.3</b> | <b>37.5</b> |
| Method                                 | $T$ | Dire.       | Disc.       | Eat         | Greet       | Phone       | Photo       | Pose        | Purch.      | Sit         | SitD        | Smoke       | Wait        | WalkD       | Walk        | WalkT       | Avg         |
| Martinez <i>et al.</i> [78] ICCV'17    | 1   | 37.7        | 44.4        | 40.3        | 42.1        | 48.2        | 54.9        | 44.4        | 42.1        | 54.6        | 58.0        | 45.1        | 46.4        | 47.6        | 36.4        | 40.4        | 45.5        |
| LCN [28] ICCV'19                       | 1   | 36.3        | 38.8        | 29.7        | 37.8        | 34.6        | 42.5        | 39.8        | 32.5        | 36.2        | 39.5        | 34.4        | 38.4        | 38.2        | 31.3        | 34.2        | 36.3        |
| Xu <i>et al.</i> [121] CVPR'21         | 1   | 35.8        | 38.1        | 31.0        | 35.3        | 35.8        | 43.2        | 37.3        | 31.7        | 38.4        | 45.5        | 35.4        | 36.7        | 36.8        | 27.9        | 30.7        | 35.8        |
| UGCN [116] ECCV'20                     | 96  | 23.0        | 25.7        | 22.8        | 22.6        | 24.1        | 30.6        | 24.9        | 24.5        | 31.1        | 35.0        | 25.6        | 24.3        | 25.1        | 19.8        | 18.4        | 25.6        |
| † PoseFormer [135] ICCV'21             | 81  | 30.0        | 33.6        | 29.9        | 31.0        | 30.2        | 33.3        | 34.8        | 31.4        | 37.8        | 38.6        | 31.7        | 31.5        | 29.0        | 23.3        | 23.1        | 31.3        |
| † MHFormer [56] CVPR'22                | 351 | 27.7        | 32.1        | 29.1        | 28.9        | 30.0        | 33.9        | 33.0        | 31.2        | 37.0        | 39.3        | 30.0        | 31.0        | 29.4        | 22.2        | 23.0        | 30.5        |
| † MixSTE [134] CVPR'22                 | 243 | 21.6        | 22.0        | 20.4        | 21.0        | 20.8        | 24.3        | 24.7        | 21.9        | 26.9        | 24.9        | 21.2        | 21.5        | 20.8        | 14.7        | 15.6        | 21.6        |
| † P-STMO [94] ECCV'22                  | 243 | 28.5        | 30.1        | 28.6        | 27.9        | 29.8        | 33.2        | 31.3        | 27.8        | 36.0        | 37.4        | 29.7        | 29.5        | 28.1        | 21.0        | 21.0        | 29.3        |
| † Ours (scratch)                       | 243 | <u>16.7</u> | <u>19.9</u> | <u>17.1</u> | <u>16.5</u> | <u>17.4</u> | <u>18.8</u> | <u>19.3</u> | <u>20.5</u> | <u>24.0</u> | <u>22.1</u> | <u>18.6</u> | <b>16.8</b> | <u>16.7</u> | <u>10.8</u> | <u>11.5</u> | <u>17.8</u> |
| † Ours (finetune)                      | 243 | <b>15.9</b> | <b>17.3</b> | <b>16.9</b> | <b>14.6</b> | <b>16.8</b> | <b>18.6</b> | <b>18.6</b> | <b>18.4</b> | <b>22.0</b> | <b>21.8</b> | <b>17.3</b> | <u>16.9</u> | <b>16.1</b> | <b>10.5</b> | <b>11.4</b> | <b>16.9</b> |
| Method                                 | $T$ | Dire.       | Disc.       | Eat         | Greet       | Phone       | Photo       | Pose        | Purch.      | Sit         | SitD        | Smoke       | Wait        | WalkD       | Walk        | WalkT       | Avg         |
| VideoPose3D [89] CVPR'19               | 243 | 3.0         | 3.1         | 2.2         | 3.4         | 2.3         | 2.7         | 2.7         | 3.1         | 2.1         | 2.9         | 2.3         | 2.4         | 3.7         | 3.1         | 2.8         | 2.8         |
| † PoseFormer [135] ICCV'21             | 81  | 3.2         | 3.4         | 2.6         | 3.6         | 2.6         | 3.0         | 2.9         | 3.2         | 2.6         | 3.3         | 2.7         | 2.7         | 3.8         | 3.2         | 2.9         | 3.1         |
| *† MixSTE [134] CVPR'22                | 243 | 2.5         | 2.7         | 1.9         | 2.8         | 1.9         | 2.2         | 2.3         | 2.6         | 1.6         | 2.2         | 1.9         | 2.0         | 3.1         | 2.6         | 2.2         | 2.3         |
| † Ours (scratch)                       | 243 | <u>1.8</u>  | <u>2.1</u>  | <u>1.5</u>  | <u>2.0</u>  | <u>1.5</u>  | <u>1.9</u>  | <u>1.8</u>  | <u>2.1</u>  | <u>1.2</u>  | <u>1.8</u>  | <u>1.5</u>  | <u>1.4</u>  | <u>2.6</u>  | <u>2.0</u>  | <u>1.7</u>  | <u>1.8</u>  |
| † Ours (finetune)                      | 243 | <b>1.7</b>  | <b>1.9</b>  | <b>1.4</b>  | <b>1.9</b>  | <b>1.4</b>  | <b>1.7</b>  | <b>1.7</b>  | <b>1.9</b>  | <b>1.1</b>  | <b>1.6</b>  | <b>1.4</b>  | <b>1.3</b>  | <b>2.4</b>  | <b>1.9</b>  | <b>1.6</b>  | <b>1.7</b>  |

Table 1. **Quantitative comparison of 3D human pose estimation on Human3.6M.** (Top) MPJPE (mm) using detected 2D pose sequences. (Middle) MPJPE (mm) using GT 2D pose sequences. (Bottom) MPJVE (mm) using detected 2D pose sequences.  $T$  denotes the clip length used by the method. We select the best results reported by each work. \* denotes using HRNet [103] for 2D detection. † denotes implemented with a spatio-temporal Transformer design. The best and second-best results are highlighted in bold and underlined formats.

# Visualization

---

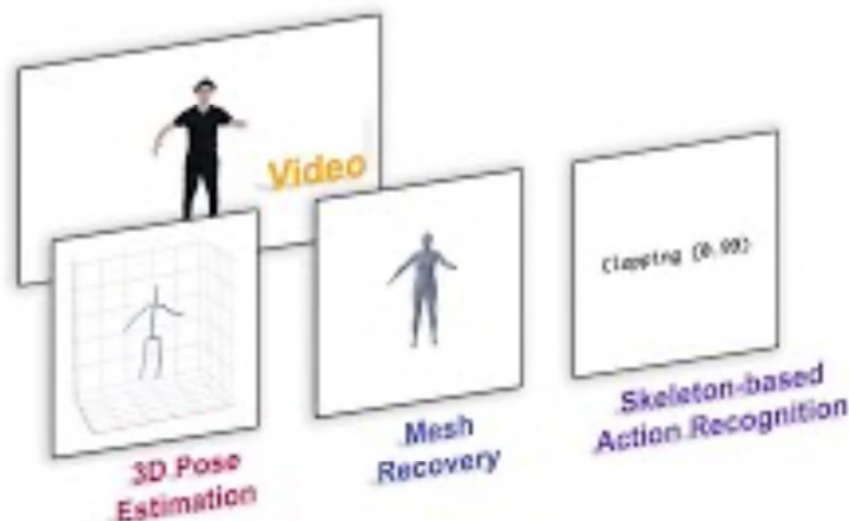
## Learning Human Motion Representations: A Unified Perspective

Wentao Zhu   Xiaoxuan Ma   Zhaoyang Liu   Libin Liu   Wayne Wu   Yizhou Wang

Peking University

SenseTime Research

Shanghai AI Laboratory



# Pose Estimation Summary

- This section covers the following:
  - Introduction
  - Single-Person Pose Estimation
  - Multi-Person Pose Estimation
  - Emerging / New Methods

# Video / Human Action Recognition (HAR)



# Human Action Recognition Overview

- Introduction
- Action Recognition Methods
  - Two-Stream Networks
  - 3D CNNs
  - Efficient Video Modelling
- New / Emerging Methods

# Objective

- To recognize human actions in a video.



Shaking hands



Cricket bowling



Skate boarding



Cutting in kitchen



Stretching leg



Riding a bike



Playing violin



Dribbling basketball

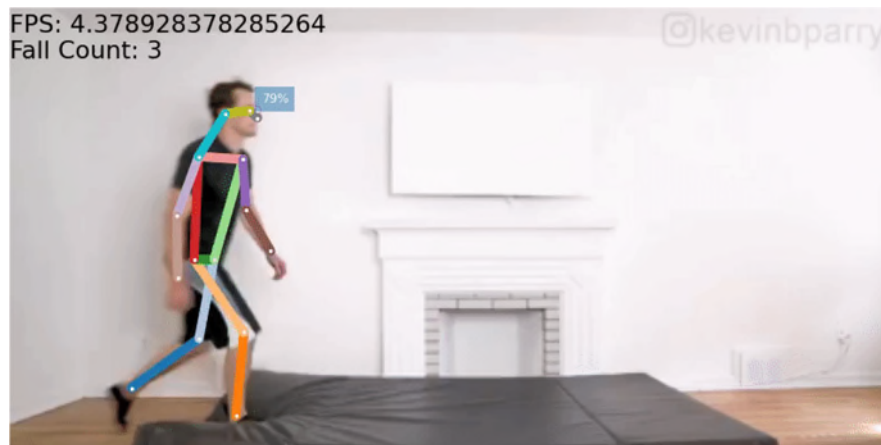
# Applications



Monitoring



Control

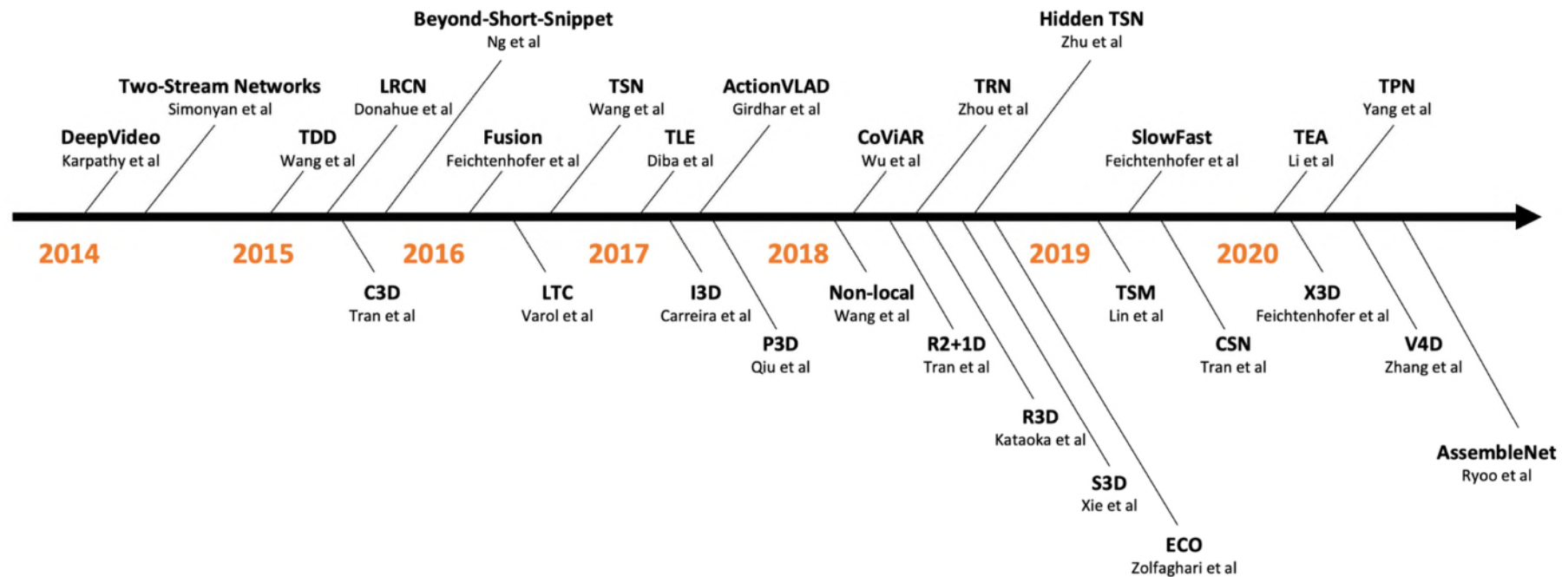


Danger detection



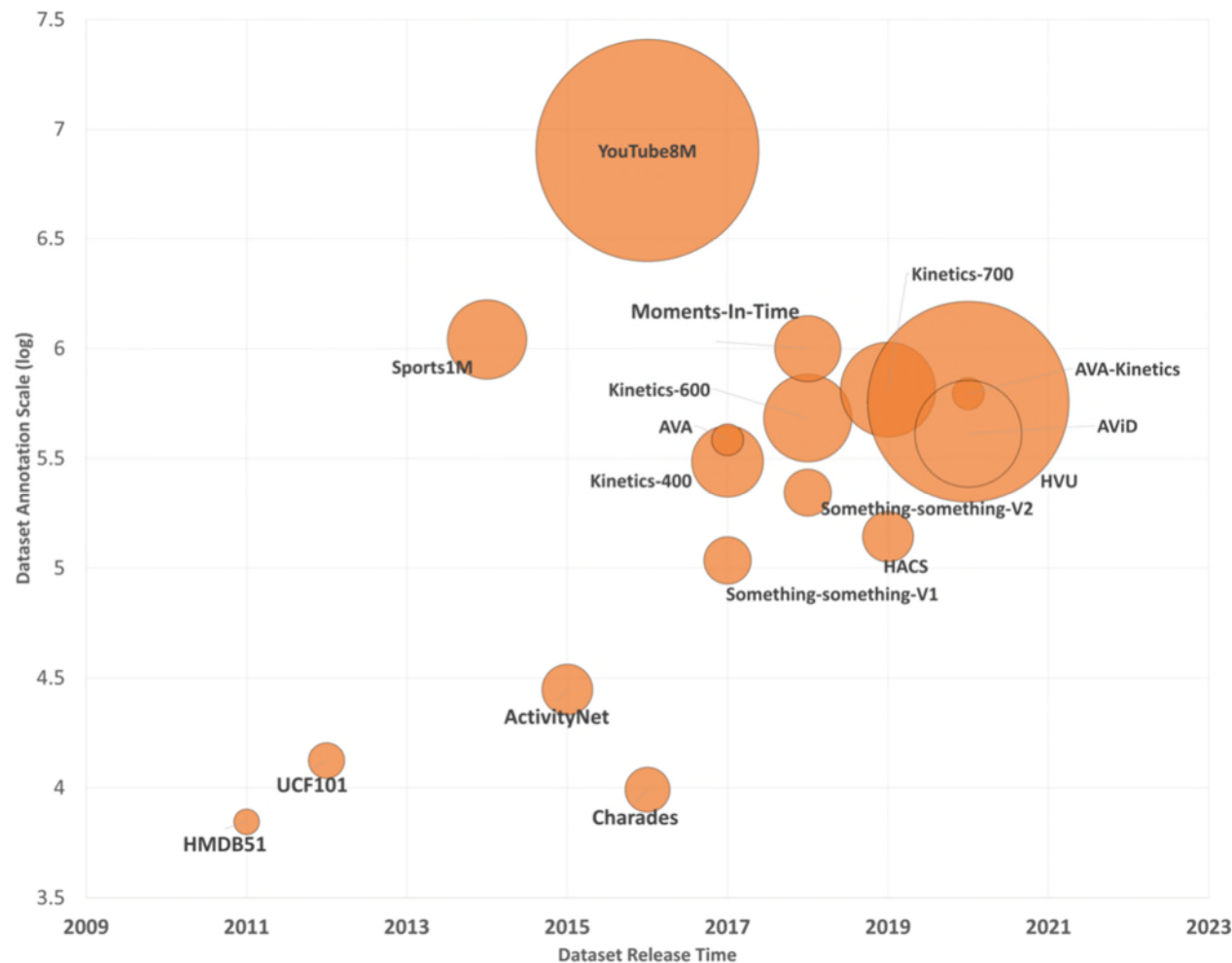
Event analysis

# Brief History & Milestones



# HAR Datasets

- High-quality large-scale action recognition datasets have emerged over the last decade.



# HAR Datasets

| Dataset                     | Year | # Samples | Ave. Len | # Actions | Comments                               |
|-----------------------------|------|-----------|----------|-----------|----------------------------------------|
| <a href="#">HMDB51</a>      | 2011 | 7K        | ~5s      | 51        | -                                      |
| <a href="#">UCF101</a>      | 2012 | 13.3K     | ~6s      | 101       | -                                      |
| <a href="#">Sports1M</a>    | 2014 | 1.1M      | ~5.5m    | 487       | First large-scale video action dataset |
| <a href="#">ActivityNet</a> | 2015 | 28K       | [5, 10]m | 200       | Human daily living actions             |
| <a href="#">YouTube8M</a>   | 2016 | 8M        | 229.6s   | 3862      | Largest-scale video dataset            |
| <a href="#">Charades</a>    | 2016 | 9.8K      | 30.1s    | 157       | Multi-label daily indoor activities    |
| Kinetics400                 | 2017 | 306K      | 10s      | 400       | Most widely adopted benchmark          |
| Kinetics600                 | 2018 | 482K      | 10s      | 600       |                                        |
| <a href="#">Kinetics700</a> | 2019 | 650K      | 10s      | 700       |                                        |



# HAR Datasets

| Dataset                        | Year | # Samples | Ave. Len | # Actions | Comments                                                                                   |
|--------------------------------|------|-----------|----------|-----------|--------------------------------------------------------------------------------------------|
| Sth-Sth V1                     | 2017 | 108.5K    | [2, 6]s  | 174       | Requires strong temporal modeling                                                          |
| <a href="#">Sth-Sth</a> V2     | 2017 | 220.8K    | [2, 6]s  | 174       |                                                                                            |
| <a href="#">AVA</a>            | 2017 | 385K      | 15m      | 80        | First large-scale spatio-temporal action detection dataset                                 |
| <a href="#">AVA-kinetics</a>   | 2020 | 624K      | 15m      | 80        |                                                                                            |
| <a href="#">Moment in Time</a> | 2018 | 1M        | 3s       | 339       | Designed for event understanding                                                           |
| <a href="#">HACS Clips</a>     | 2019 | 1.55M     | 2s       | 200       | Consist of segments                                                                        |
| <a href="#">HVV</a>            | 2020 | 572K      | 10s      | 739       | This dataset has six task categories: scene, object, action, event, attribute, and concept |
| <a href="#">AViD</a>           | 2020 | 450K      | [3, 15]s | 887       | Remove face identities to protect privacy of video makers                                  |
| <a href="#">NTU RGB+D (S)</a>  | 2016 | -         | -        | 120       | Introduce depth and skeleton information                                                   |

# Samples in HAR Datasets

**UCF101**



Cricket bowling



Skate boarding



Cutting in kitchen

**Kinetics400**



Riding a bike



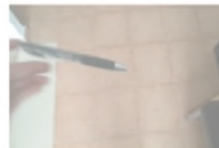
Salsa dancing



braiding hair

-----  
Dropping something

**Something  
-Something**



Picking something up  
-----

**Moments  
in time**



Climbing

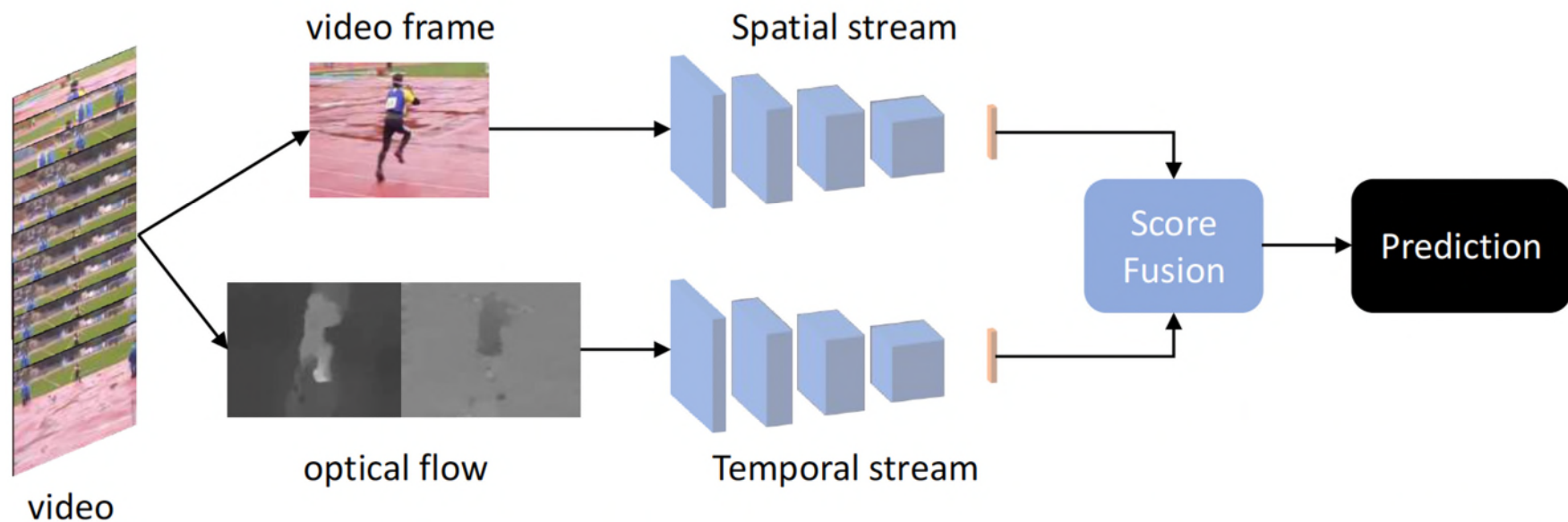
# HAR Methods

- Two-Stream Networks
- 3D CNNs
- Efficient Video Modelling
- Etc.

# Two-Stream Networks

# Two-Stream Networks

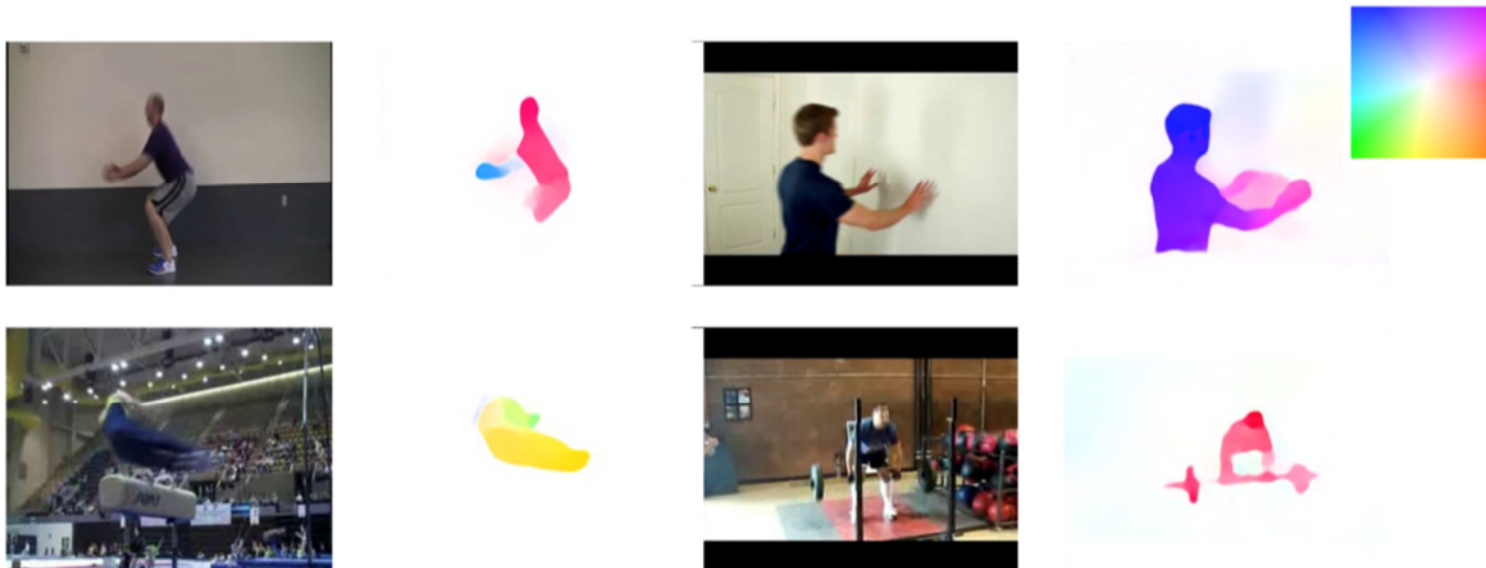
- Consist of a spatial stream, a temporal stream and a score fusion module:
  - Spatial stream captures appearance information from sampled RGB frames.
  - Temporal stream recognizes motion from optical flow.
  - Score fusion module combines scores from the spatial and temporal streams.



**Two-stream Networks**

# Optical Flow

- An effective motion representation to describe object/scene movement.
- Optical flow can describe the motion pattern of each action.
- Pros: provide orthogonal information to RGB image.
- Cons: Optical flow is computationally intensive & has large storage requirement.





# Two-Stream Networks

- Spatial stream
  - Sample video frame from video clips.
  - Input the sampled frame to a CNN network.
  - Output the spatial prediction score from a fully-connected layer.
- Temporal stream
  - Estimate the horizontal and vertical optical flows from stacked frames.
  - Concatenate the two optical flow images and input to a CNN.
  - Output the temporal prediction score from a fully-connected layer.
- Fusion
  - Use different fusion techniques (e.g., average, weighted average, etc.) to obtain final prediction score from both spatial and temporal scores.

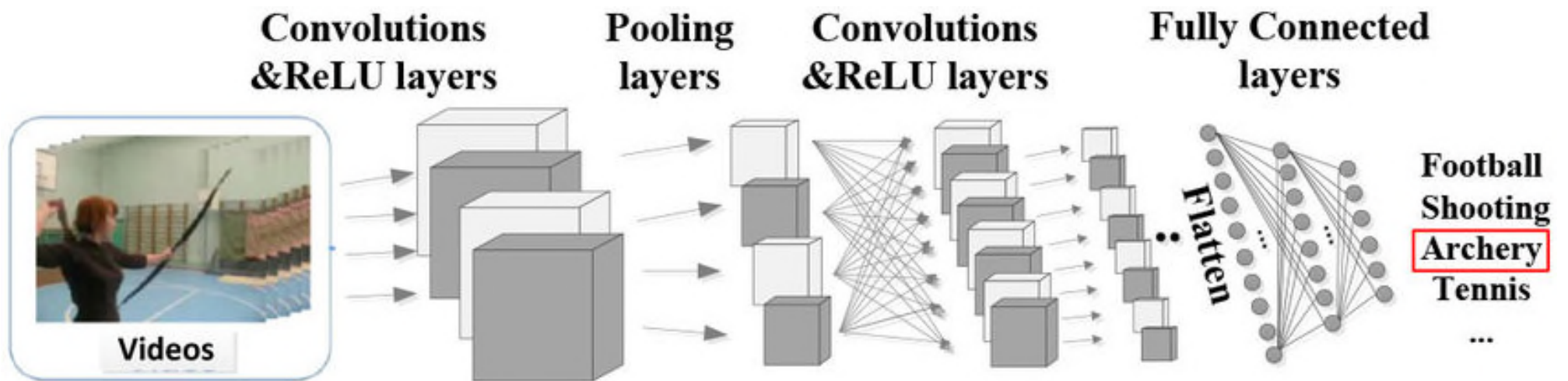
# Pros and Cons in Two-Stream Networks

- Pros
  - Simple network structure.
  - Optical flow is an accurate way to represent motion information.
- Cons
  - Optical flow is expensive to extract and store.

# 3D CNNs

# 3D CNNs

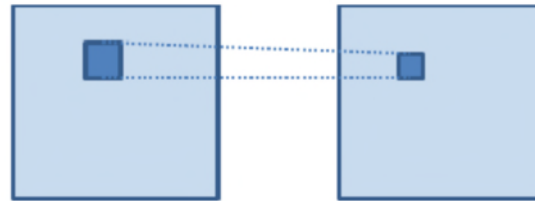
- 3D CNN treats a video as a 3D tensor with two spatial and one time dimension.



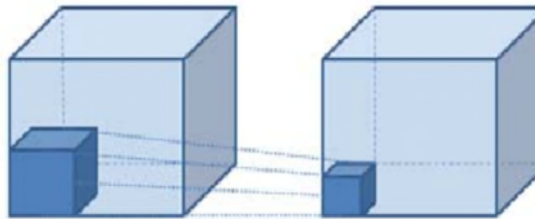
# 3D CNNs

- Designed to handle 3D tensors.
- Traditional 2D Convolution: dot product between input volume with different filters, where each filter and input are 2D matrices (ignore the depth dimension).
- 3D Convolution: dot product on 3D tensors.
- Representative work: SlowFast Network

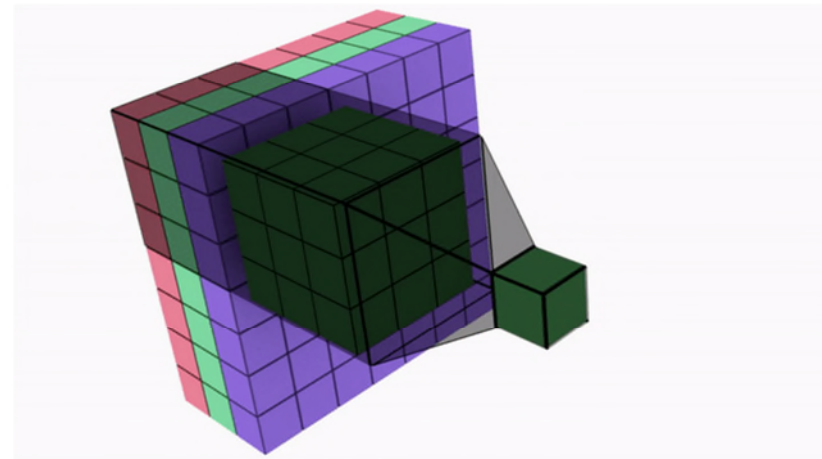
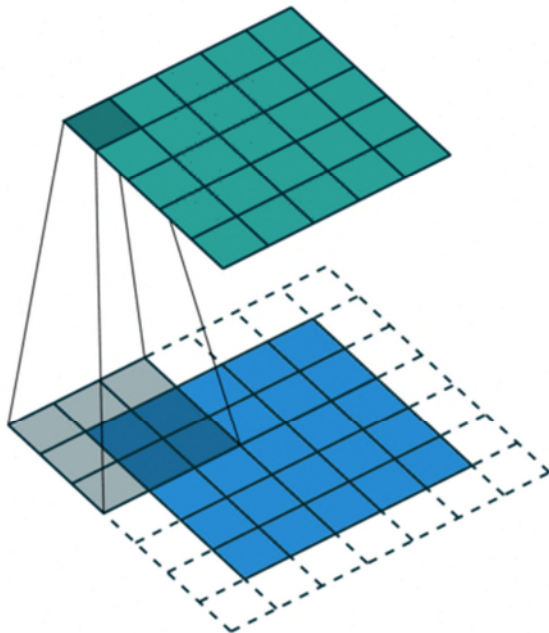
# 2D CNNs vs. 3D CNNs



(a) 2D convolution



(b) 3D convolution

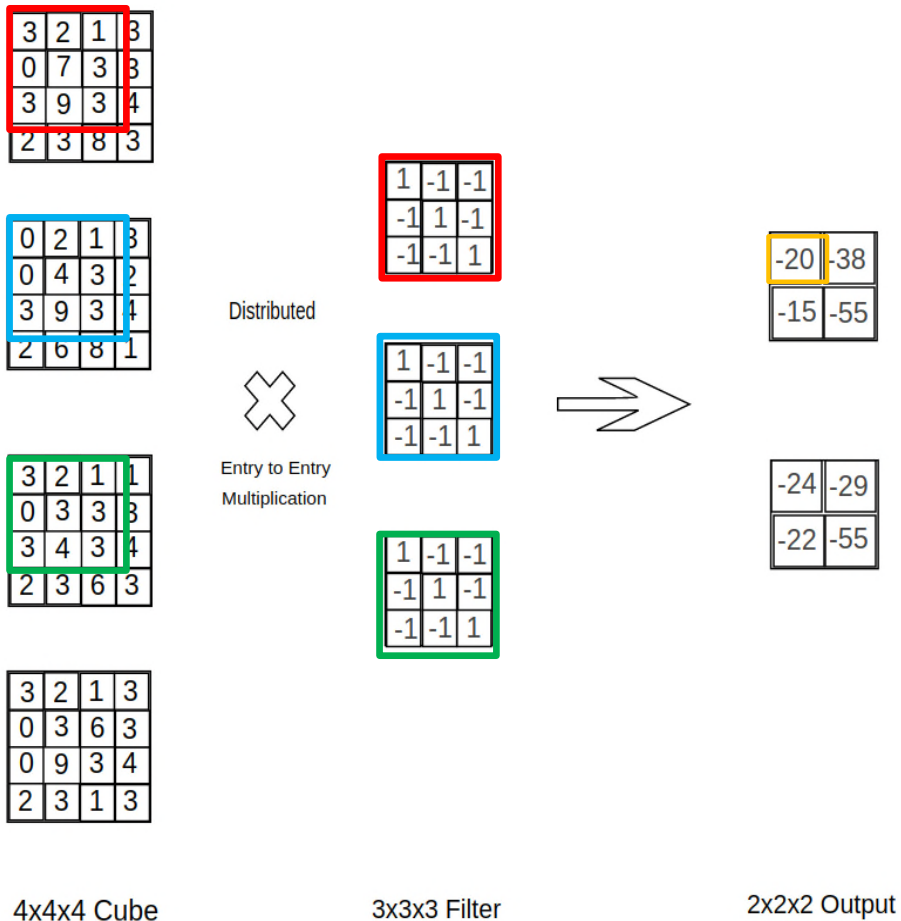




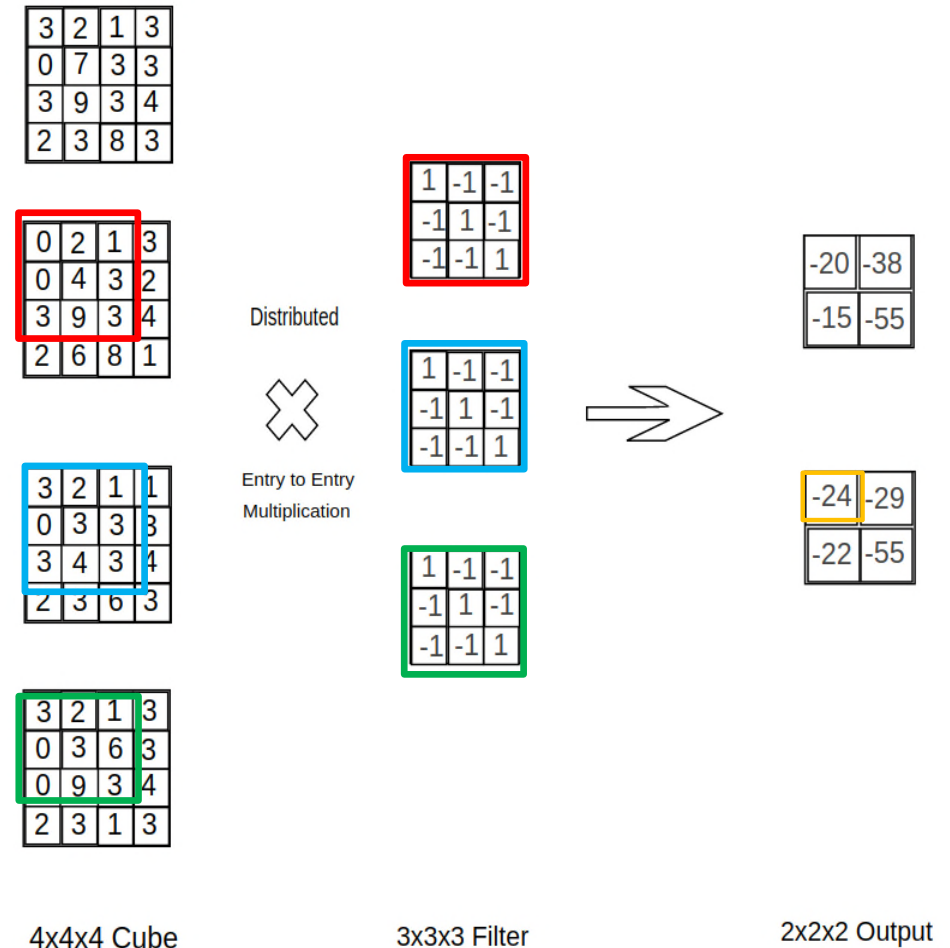
# 3D Convolution

- Assume depth channel = 1

3d Convolution



3d Convolution



# Pros and Cons

- Pros:
  - Feature extractor can obtain temporal and spatial information simultaneously.
- Cons:
  - Hard to train and deploy due to large model and low inference efficiency.

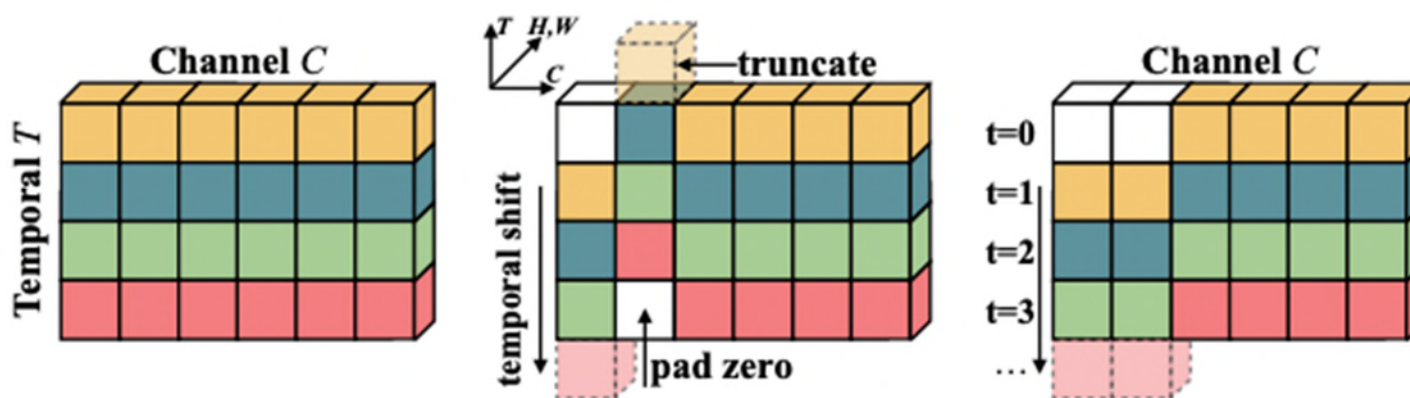
# Efficient Video Modelling

# Efficient Video Modelling

- Improve accuracy and efficiency at the same time for video action recognition.
- Features:
  - Encode motion information without using optical flow.
  - Temporal modeling without 3D convolution.
- Representative work: Temporal Shift Module (TSM)
- Pros
  - Fast and efficient.
- Cons
  - Not as accurate as 3D CNNs.

# Temporal Shift Module (TSM)

- Shifting Operation
  - It shifts part of the channels along the temporal dimension, thus facilitating information exchange among neighboring frames.
  - Two shift options: (1) bi-direction for offline settings and (2) uni-direction for online settings.



**(a)** The original tensor without shift.

**(b)** Offline temporal shift (bi-direction).

**(c)** Online temporal shift (uni-direction).

Emerging / New Methods



# Transformer-based Methods

- Existing approaches encode only short-range dependencies.
- Long-range dependency modelling is desirable in many applications such as action recognition.
- Representative work: Video Swin transformer
- Pros
  - Capture long-term dependencies.
- Cons
  - Computationally intensive and data hungry.

# Video Swin Transformer

- ViT-based methods require large computational resource as self-attention module consider all tokens.
- Video Swin Transformer aims to reduce model complexity while keeping the ability to capture long-term dependencies.

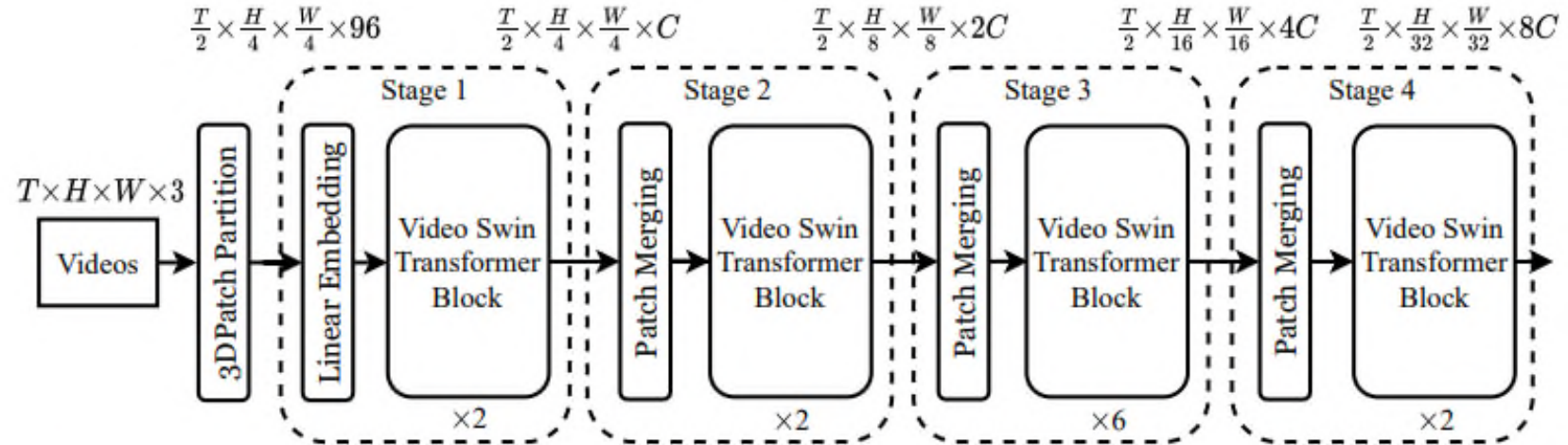
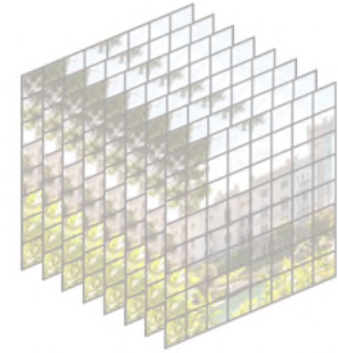


Figure 1: Overall architecture of Video Swin Transformer (tiny version, referred to as Swin-T).

# Video Swin Transformer



3D tokens:  $T' \times H' \times W' = 8 \times 8 \times 8$

- 3D Patch Partition
  - Input video size:  $T \times H \times W \times 3$ .
  - 3D token / patch size:  $2 \times 4 \times 4 \times 3$ .
  - $T' \times H' \times W'$ : number of tokens in a video clip =  $T/2 \times H/4 \times W/4$ .
- Linear Projection / Embedding
  - Use a linear layer to project the 3D tokens from number  $T' \times H' \times W' \times 96$  to  $T' \times H' \times W' \times C$  in channel dimension.

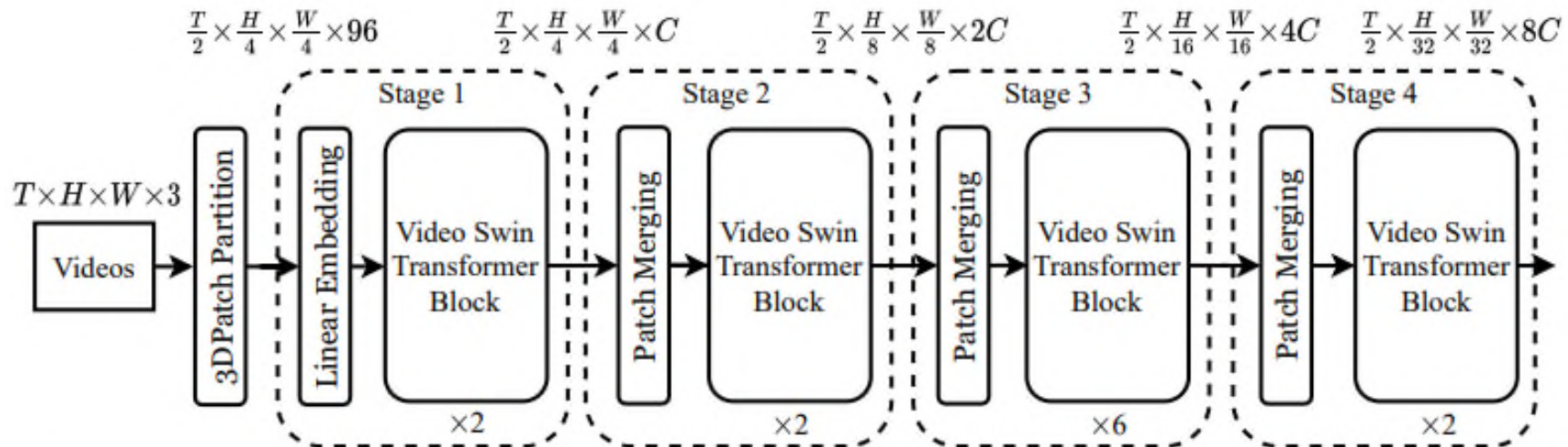


Figure 1: Overall architecture of Video Swin Transformer (tiny version, referred to as Swin-T).

# Video Swin Transformer Block

- In layer  $l$ , partition the 3D tokens into windows of size  $P \times M \times M$ , and perform self-attention in every local window.
- In layer  $l+1$ , shift the window with a certain step ( $P/2 \times M/2 \times M/2$ ), organize tokens into new windows, and calculate self-attention within each shift window.

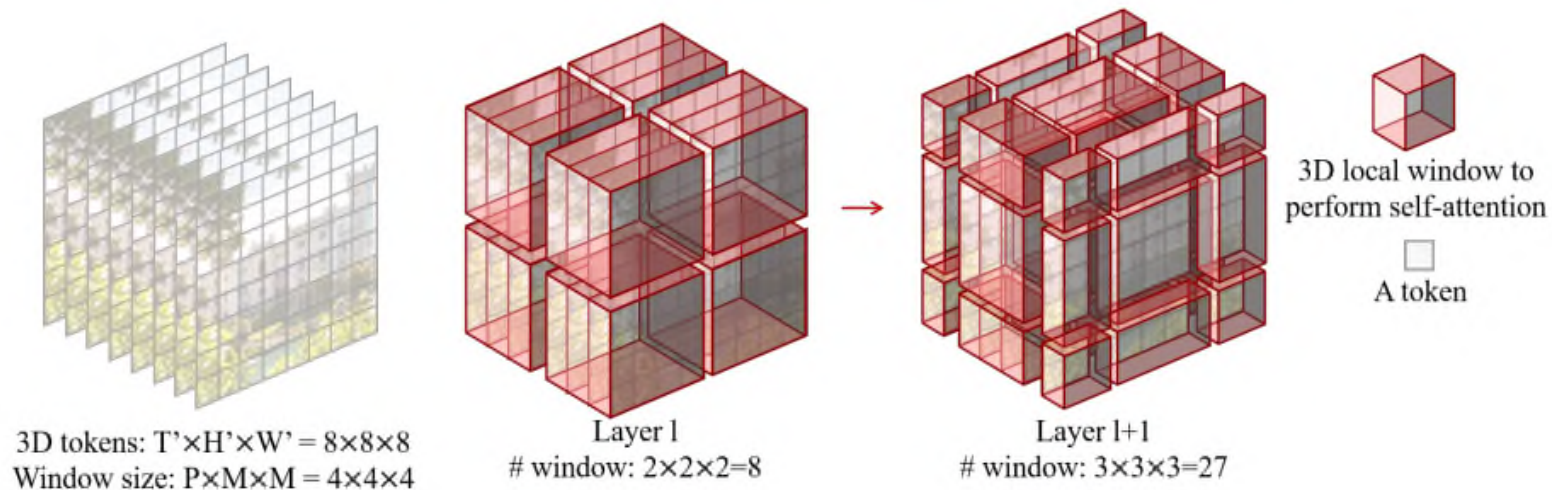


Figure 3: An illustrated example of 3D shifted windows. The input size  $T' \times H' \times W'$  is  $8 \times 8 \times 8$ , and the 3D window size  $P \times M \times M$  is  $4 \times 4 \times 4$ . As layer  $l$  adopts regular window partitioning, the number of windows in layer  $l$  is  $2 \times 2 \times 2 = 8$ . For layer  $l+1$ , as the windows are shifted by  $(\frac{P}{2}, \frac{M}{2}, \frac{M}{2}) = (2, 2, 2)$  tokens, the number of windows becomes  $3 \times 3 \times 3 = 27$ . Though the number of windows is increased, the efficient batch computation in [28] for the shifted configuration can be followed, such that the final number of windows for computation is still 8.



# Video Swin Transformer

- Patching merging
  - Perform  $2\times$  spatial downsampling and concatenate features of each  $2\times 2$  spatially neighboring patches.
  - Do not downsample along the temporal dimension.
  - Apply a linear layer to project the concatenated features.

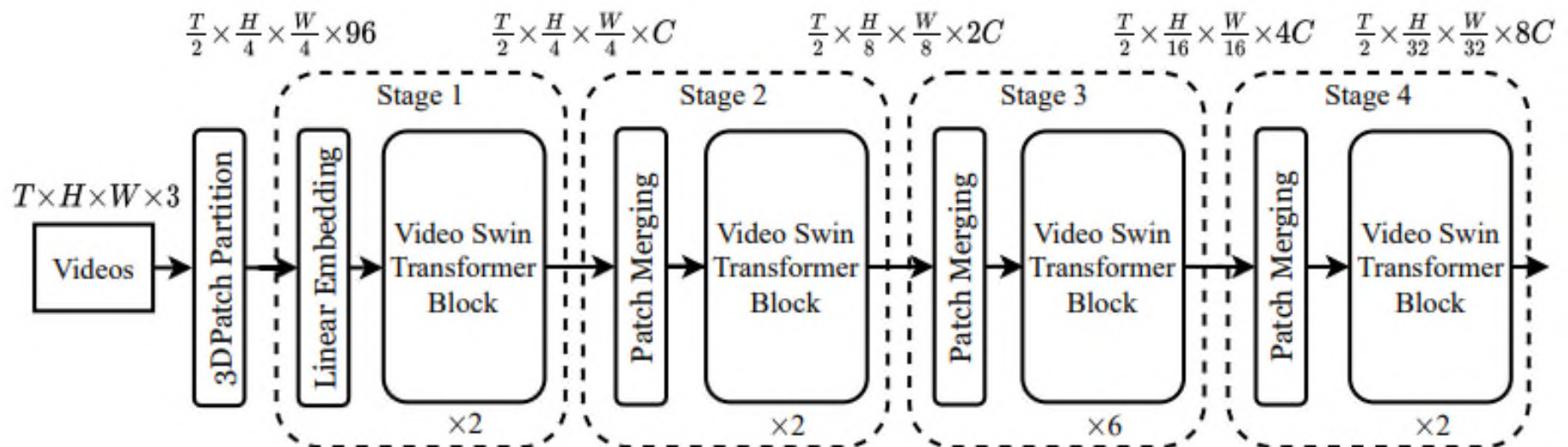


Figure 1: Overall architecture of Video Swin Transformer (tiny version, referred to as Swin-T).

# Architecture Variants

- Swin-T:  $C = 96$ , layer numbers =  $\{2, 2, 6, 2\}$
- Swin-S:  $C = 96$ , layer numbers =  $\{2, 2, 18, 2\}$
- Swin-B:  $C = 128$ , layer numbers =  $\{2, 2, 18, 2\}$
- Swin-L:  $C = 192$ , layer numbers =  $\{2, 2, 18, 2\}$

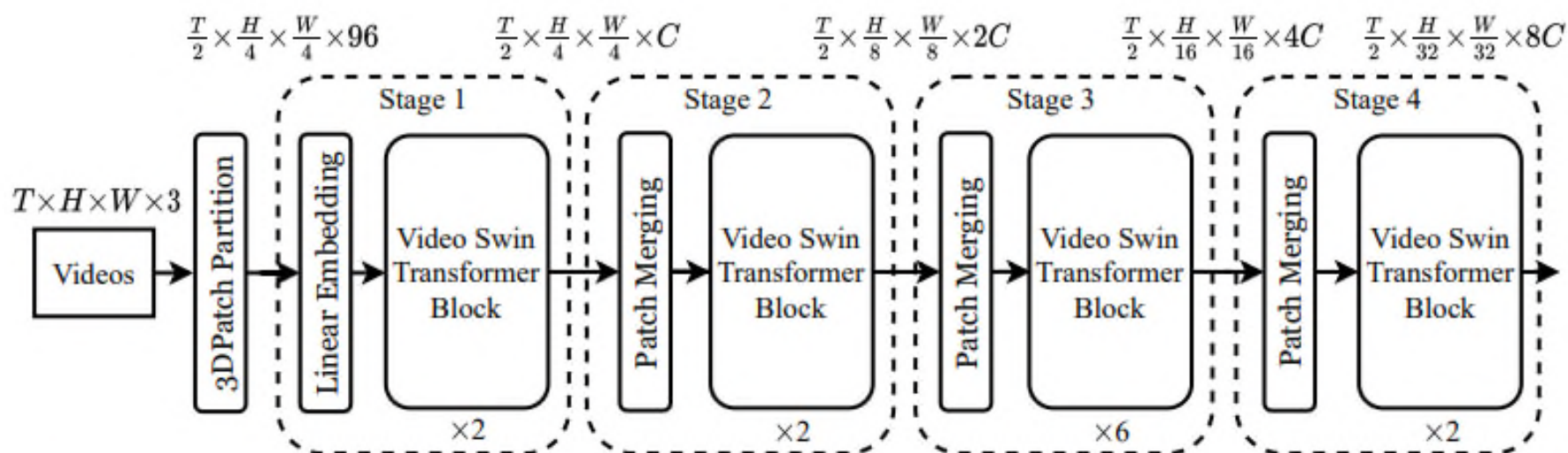


Figure 1: Overall architecture of Video Swin Transformer (tiny version, referred to as Swin-T).

# Performance Comparison

Table 2: Comparison to state-of-the-art on Kinetics-600.

| Method                         | Pretrain     | Top-1       | Top-5       | Views         | FLOPs | Param |
|--------------------------------|--------------|-------------|-------------|---------------|-------|-------|
| SlowFast R101+NL [13]          | -            | 81.8        | 95.1        | $10 \times 3$ | 234   | 59.9  |
| X3D-XL [12]                    | -            | 81.9        | 95.5        | $10 \times 3$ | 48    | 11.0  |
| MViT-B-24, $32 \times 3$ [9]   | -            | 83.8        | 96.3        | $5 \times 1$  | 236   | 52.9  |
| TimeSformer-HR [3]             | ImageNet-21K | 82.4        | 96          | $1 \times 3$  | 1703  | 121.4 |
| ViViT-L/ $16 \times 2$ 320 [1] | ImageNet-21K | 83.0        | 95.7        | $4 \times 3$  | 3992  | 310.8 |
| ViViT-H/ $16 \times 2$ [9]     | JFT-300M     | 85.8        | 96.5        | $4 \times 3$  | 8316  | 647.5 |
| Swin-B                         | ImageNet-21K | 84.0        | 96.5        | $4 \times 3$  | 282   | 88.1  |
| Swin-L (384 $\uparrow$ )       | ImageNet-21K | 85.9        | 97.1        | $4 \times 3$  | 2107  | 200.0 |
| Swin-L (384 $\uparrow$ )       | ImageNet-21K | <b>86.1</b> | <b>97.3</b> | $10 \times 5$ | 2107  | 200.0 |



# Skeleton-based Networks

- Utilize temporal pose information to predict actions.
- Methods
  - Extract pose information from videos.
  - Use techniques such as Graph Neural Network to classify actions.

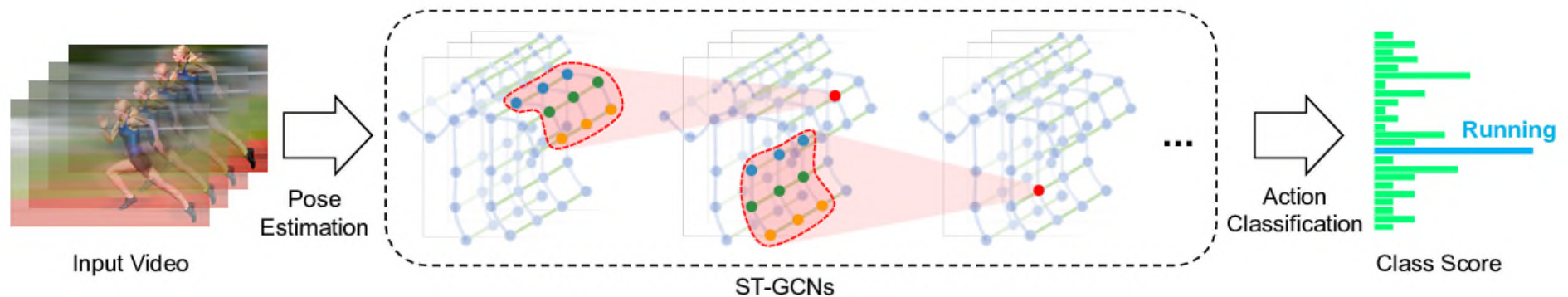
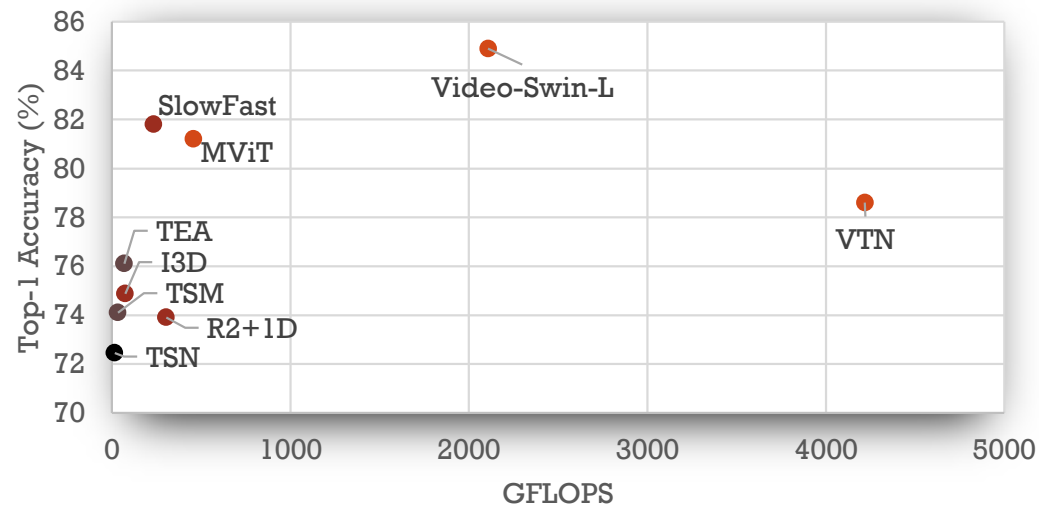
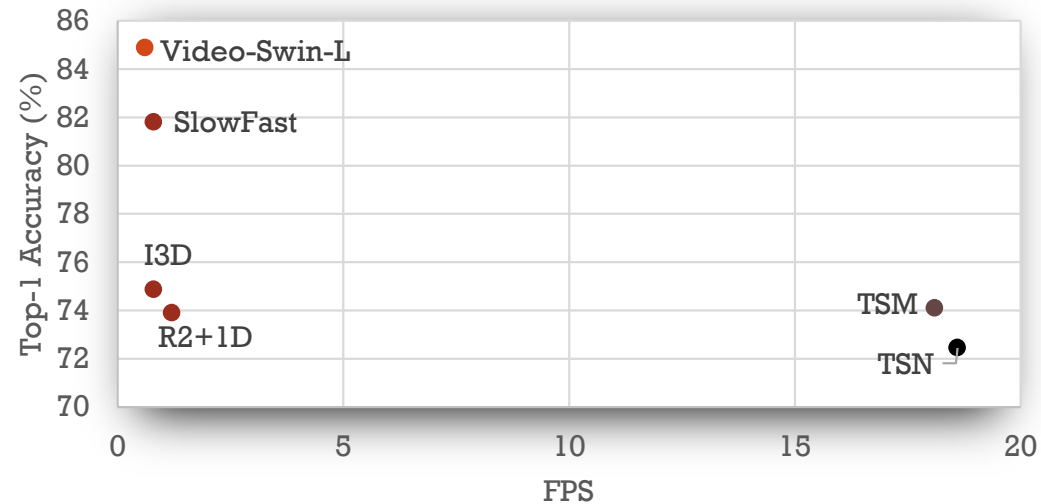


Figure 2: We perform pose estimation on videos and construct spatial temporal graph on skeleton sequences. Multiple layers of spatial-temporal graph convolution (ST-GCN) will be applied and gradually generate higher-level feature maps on the graph. It will then be classified by the standard Softmax classifier to the corresponding action category.

# Performance Comparison

| Methods                  | Representative Models |      |       | Input Size                           | GFLOPs × views          | Accuracy (Top1 %) (Kinetics 400)         | FPS  | Remarks                                                                          |
|--------------------------|-----------------------|------|-------|--------------------------------------|-------------------------|------------------------------------------|------|----------------------------------------------------------------------------------|
|                          | Model                 | Year | Venue |                                      |                         |                                          |      |                                                                                  |
| Two-stream networks      | TSN                   | 2016 | ECCV  | $8 \times 3 \times 224 \times 224$   | $16 \times 250$         | 72.45                                    | 18.6 | Simple design, significant computation and storage requirement for optical flows |
| 3D CNNs                  | I3D                   | 2017 | CVPR  | $32 \times 3 \times 224 \times 224$  | $108 \times \text{N/A}$ | 74.87                                    | 0.8  | Complex, very large computation consumption in training                          |
|                          | SlowFast              | 2019 | CVPR  | $32 \times 3 \times 224 \times 224$  | $234 \times 30$         | 81.8                                     | 0.8  |                                                                                  |
| Efficient video modeling | TSM                   | 2019 | ICCV  | $16 \times 3 \times 224 \times 224$  | $33 \times 30$          | 74.1                                     | 18.1 | Simple, efficient training, fast runtime, relatively accurate                    |
|                          | TDN                   | 2021 | CVPR  | $24 \times 3 \times 224 \times 224$  | $198 \times 30$         | 79.4                                     | -    |                                                                                  |
| Transformers             | VTN                   | 2021 | ICCV  | $250 \times 3 \times 224 \times 224$ | $4218 \times 1$         | 78.6                                     | -    | Accurate, large data requirement, high computational cost                        |
|                          | Video Swin-L          | 2022 | CVPR  | $32 \times 3 \times 384 \times 384$  | $2107 \times 50$        | 84.9                                     | 0.6  |                                                                                  |
| Skeleton based networks  | ST-GCN                | 2018 | AAAI  | -                                    | -                       | 30.7 (Kinetics 400)<br>86.9 (NTU60_XSub) | -    | Leverage on human pose, lower accuracy in broad domain applications.             |
|                          | PoseC3D               | 2021 | ArXiv | $8 \times 3 \times 224 \times 224$   | -                       | 47.4 (Kinetics 400)<br>94.3 (NTU60_XSub) | -    |                                                                                  |

# Performance Comparison



# Exercise: Human Action Recognition

- (d) Briefly describe how Temporal Shift Module (TSM) can achieve good computational efficiency in video action recognition.

(6 Marks)

# Solution

# Human Action Recognition Summary

- The section cover the following:
  - Introduction
  - Action Recognition Methods
    - Two-Stream Networks
    - 3D CNNs
    - Efficient Video Modelling
  - New / Emerging Methods



# Part 5 Summary

- The section covers the following topics:
  - Object Detection & Tracking
  - Pose Estimation
  - Video / Human Action Recognition